ПОНЯТЬ МОДЕЛЬ. НАПИСАТЬ КОД. ПРОВЕРИТЬ СЕБЯ.

JAVA
с нуля

До уверенного понимания языка,
объектов и небольших приложений.

20 глав / 180 упражнений / 6 проектов

Теория, примеры, ошибки, самостоятельная практика

```
исходник -> компилятор -> JVM
значения -> ссылки -> объекты
контракт -> реализация -> проверка
```

На русском / Java 17 / JDK 25 / Ubuntu / IntelliJ IDEA

Самостоятельная работа без готовых решений упражнений

vantaeonx · Издание 1.0 · 2026

Оглавление

Нажми название, чтобы перейти к разделу. Упражнения находятся сразу после теории своей главы; номера файлов сквозные.

Как учиться по этому пособию 4

Установка и первый запуск 6

Карта курса и условия перехода 8

01. Как Java-программа становится работающей программой 9

02. Переменные, типы и значения 20

03. Выражения, арифметика и логика 31

04. Ввод как договор о формате данных 42

05. Условия и полная система случаев 53

06. Циклы: повторение с доказуемой остановкой 64

07. Массивы и алгоритмы над последовательностями 75

08. Строки: содержание, Unicode и неизменяемость 86

09. Методы и передача значений 97

10. Класс, объект и состояние экземпляра 108

11. Конструкторы, инкапсуляция и инварианты 119

12. Наследование и полиморфизм 131

13. Интерфейсы и композиция 142

14. Исключения и работа с отказами 153

15. List и Set: порядок, дубликаты и изменение 164

16. Map, равенство и хеширование 175

17. Generics: связь типов в договоре 186

18. Файлы, пути и устойчивый текстовый формат 197

19. Отладка, проверки и слои приложения 208

20. Современные модели и сборка законченного приложения 219

Консольный пункт выдачи 231

Панель телеметрии 233

Бронирование рабочего пространства 235

Каталог мастерской 237

Файловый журнал расходов 239

Симулятор исследовательского рюкзака 241

Памятки после первого прохода 243

Словарь 245

Таблица прогресса 247

Источники и как ими пользоваться 258

Как учиться по этому пособию

Курс рассчитан на самостоятельное обучение с нуля. Если ты немного знаешь C и PHP, знакомые названия помогут, но правила преобразования типов, ссылок и методов нужно проверять именно для Java. В уроках используется Java 17; установленный JDK 25 подходит для компиляции с --release 17. Путь настройки Ubuntu и IntelliJ IDEA есть в README.

Один учебный цикл

1. Прочитай теорию до примера и перескажи её своими словами.
2. До запуска предскажи результат примера. Запусти и сравни.
3. Измени одно значение, объясни отличие: это проверяет модель, а не память картинки.
4. Прочитай задание, запиши вход, результат и границы. Сначала план, затем код.
5. Проверь обычный случай, точную границу, пустоту и отказ, если они разрешены договором.
6. Запиши причину исправленной ошибки и отметь прогресс.
7. Повтори без подсказок через 1, 7 и 21 день. При затруднении повтори главу и маленький опыт.

Что означает уровень

Уровень 1 вводит одну идею. Уровень 2 соединяет несколько приёмов. Уровень 3 требует самостоятельного алгоритма, аккуратных границ или проектирования договора. Сложное задание не обязано решаться за одно занятие. Разбей его на проверяемые шаги.

Основной маршрут: главы 01-20 по порядку, в каждой уровни 1 и 2. Сложные задания можно решать сразу или на втором проходе. Для уже знакомых переменных сначала выполни задачи на прогноз и преобразование типов; они покажут, можно ли ускорить повторение.

Пять форматов

- Написать код: обработка и вывод остаются тебе, данные уже подготовлены.
- Предсказать результат: сначала комментарий с прогнозом, потом запуск исходного кода.
- Найти ошибку: объясни причину, а не только предложи замену строки.
- Исправить программу: восстанови договор и проверь, что исправление работает на границах.
- Объяснить поведение: напиши связное объяснение и при необходимости короткий безопасный опыт.

У каждой задачи собственный ExNNN.java и main. Вложенные классы позволяют ООП-задачам оставаться независимыми. В настоящем приложении классы часто разносят по файлам; практика такой организации обсуждается после освоения модели.

Что считать освоением

Ты можешь без подсказки объяснить новую модель, решить короткую вариацию, найти причину ошибки и придумать проверку границы. Галочка за запуск недостаточна. Полезно записывать не «ошибка в цикле», а «счётчик не обновлялся перед continue, условие оставалось истинным».

Если застрял, уменьши данные до одного-двух элементов, нарисуй объекты и ссылки или таблицу итераций. Сравни условие со своим кодом по предложениям. Не ищи полное решение, пока не сформулировал конкретный вопрос о типе, состоянии или ходе выполнения.

Мини-проекты и следующий шаг

Шесть проектов объединяют темы постепенно: пункт выдачи, телеметрия, бронирование, каталог, файл расходов и рюкзак. У каждого есть этапы и проверяемые критерии. Сначала заверши минимальный договор, потом выбирай расширение. Финальный рюкзак - консольная модель для тренировки; он ещё не мод Minecraft.

В этой книге нет публичных готовых решений. Полные примеры теории объясняют механизм на отдельной ситуации. Проверочные результаты задач описывают требуемое поведение, а не раскрывают код. Удачной самостоятельной практики!

Установка и первый запуск

1. Скачай папку проекта, распакуй в доступное место. Имя внешней папки не важно.
2. В IDEA выбери **File → Open** и папку, в которой находятся `pom.xml` и этот README. Если предлагается импорт Maven, согласись. Первое скачивание плагина Maven требует сети.
3. Открой **File → Project Structure → Project**. Выбери установленный **JDK 25** как Project SDK, а **Language level - 17**, без preview. При необходимости укажи путь JDK через Add SDK. JDK, которым запущена сама IDEA, не обязательно является SDK проекта.
4. Если проект открылся без Maven, пометь `src/main/java` через **Mark Directory as → Sources Root**. Не помечай корнем `ch01`. В этой папке лежат все независимые упражнения.
5. Открой `src/main/java/ru/vantaeonx/javaguide/ch01/Ex001.java`. Прочитай условие, впиши автора, допиши только свою рабочую область, сохрани.
6. Нажми зелёный треугольник возле `main` → **Run 'Ex001.main()'**. Вывод и ввод находятся в панели Run, а не в редакторе исходника.

Пустой вывод у ещё не решённой заготовки нормален. Методы с `UnsupportedOperationException("TODO")` нужно реализовать **до вызова**. Примеры из теории лежат в `examples/Example01.java-Example20.java`; проще запускать их терминальной командой ниже. Они показывают другую ситуацию и не являются ответами задач.

Если IDEA пытается собирать весь модуль, а ты временно сломал другое упражнение, запусти нужное через `tools/run.sh`: он компилирует только выбранный файл. Не удаляй package и не переименовывай классы в Main.

Ubuntu 26.04: проверить инструменты

Открой терминал и выполни по отдельности:

ТЕРМИНАЛ

```
java -version
javac -version
```

Нужны обе команды. Если установлен JDK 25 и обе показывают 25, переустановка не нужна. Если `javac` отсутствует, установи JDK из доступных пакетов Ubuntu:

ТЕРМИНАЛ

```
sudo apt update
sudo apt install openjdk-25-jdk
```

Если в используемых репозиториях такого пакета нет, используй имеющийся JDK 17+ или официальный дистрибутив JDK. Для JDK 17 пакет обычно называется `openjdk-17-jdk`; сначала проверь `apt policy openjdk-17-jdk`. Курс не требует удалить JDK 25. При нескольких JDK можно проверить `update-alternatives --display java` и `update-alternatives --display javac`; версия терминала и SDK в IDEA настраиваются отдельно.

Команды ниже запускай из **корня java-guide-ru**, где лежит `pom.xml`:

ТЕРМИНАЛ

```
bash tools/run.sh Ex001
bash tools/run.sh Ex028
bash tools/run.sh Project01
bash tools/run.sh Example04
```

Последняя команда ждёт две строки: город, затем число. Упражнения на ввод содержат свой протокол и пример в комментариях. Цифры после точки вводи с точкой там, где это прямо указано. Аргументы main передаются после имени:

ТЕРМИНАЛ

```
bash tools/run.sh Ex180 sum 4 -2 7
```

До решения Ex180 ничего не выведет: это команда проверки **твоей будущей реализации**. `Ctrl+C` останавливает ошибочно написанный бесконечный цикл.

Без скриптов одно упражнение можно собрать так:

ТЕРМИНАЛ

```
mkdir -p build/classes
javac --release 17 -encoding UTF-8 -d build/classes
    > src/main/java/ru/vantaeonx/javaguide/ch01/Ex001.java
java -cp build/classes ru.vantaeonx.javaguide.ch01.Ex001
```

`--release 17` проверяет синтаксис, стандартные API и целевой байткод. Одна настройка Language level в редакторе этого полностью не заменяет. Для всех исходников сразу:

ТЕРМИНАЛ

```
bash tools/build.sh
```

Maven необязателен для упражнений. Если используешь его: `mvn compile`; настройки компилятора уже есть в `pom.xml`. В IDEA Maven может использовать встроенный Maven.

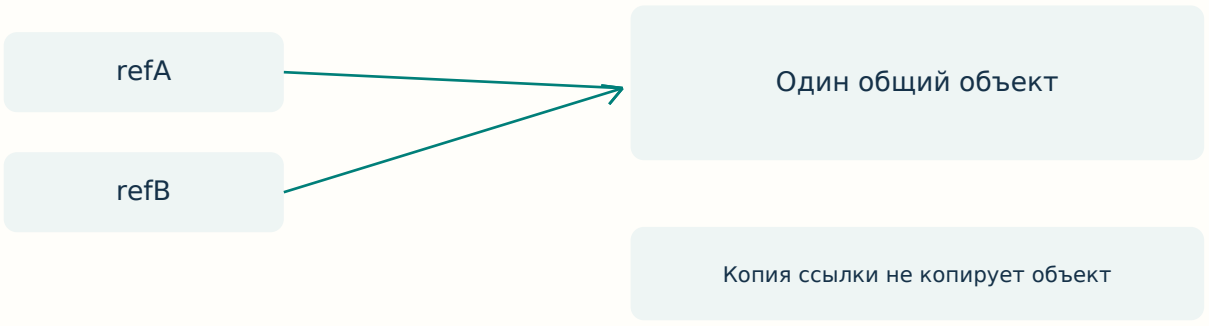
Карта курса и условия перехода

Главы	Что осваиваешь	Практика
01-05	Компиляция, типы, ввод и ветвления	Ex001-045 / проект 01
06-09	Алгоритмы, последовательности и методы	Ex046-081 / проект 02
10-14	Объекты, договоры и отказы	Ex082-126 / проект 03
15-18	Структуры данных, generics и файлы	Ex127-162 / проекты 04-05
19-20	Проверки и законченное приложение	Ex163-180 / проект 06

Перед новой темой проверь, можешь ли объяснить модель и пройти начальные проверки без подсказки. Не используй готовые ответы как меру освоения. Для сложной задачи сначала выдели минимальный рабочий шаг.

Как читать код в PDF

Точные исходники находятся в Java-файлах. Длинные строки в книге могут переноситься; продолжение отмечено >. Этот знак в начале перенесённой строки не является частью Java-кода. Для копирования используй исходный файл. Не перепечатывай весь каркас, чтобы начать упражнение.



ГЛАВА 01 / ТЕОРИЯ

01. Как Java-программа становится работающей программой

Цель и исходная модель

Программа - точное описание действий над данными. Компьютер не догадывается о намерении автора: неверное выражение может быть допустимой программой. Поэтому нужно различать три вопроса: правильно ли записан код, можно ли выполнить его на данных и соответствует ли результат условию.

Java-код хранится в текстовом файле `.java`. Компилятор `javac` проверяет грамматику, имена, типы и доступ, затем создаёт `.class` с байткодом. JVM загружает классы и выполняет байткод; реализация JVM может переводить часто исполняемый код в машинные инструкции. JDK содержит инструменты разработки и среду выполнения. IntelliJ IDEA помогает редактировать и запускать код, но выбранный в проекте JDK остаётся отдельной настройкой.

Путь выполнения: исходник -> проверка компилятором -> байткод -> загрузка JVM -> вызов main. Ошибка компиляции возникает до запуска. Ошибка выполнения появляется после старта. Логическая ошибка может не вызвать ни одного сообщения системы.

Почему здесь нужен класс

В этом курсе используется обычная структура Java 17. Публичный верхнеуровневый класс `Example01` хранится в `Example01.java`. Тело класса заключено в фигурные скобки. Внутри находится метод `main`: точка входа, которую запускает JVM. В более новых Java есть другие формы запуска, но сначала полезно освоить структуру, которая встречается в библиотеках и обычных проектах.

`public` разрешает доступ к методу извне, `static` позволяет вызвать его без создания объекта, `void` означает отсутствие возвращаемого значения. `String[] args` - массив аргументов командной строки. Пока это готовый контракт запуска; главы о методах и объектах объяснят каждую часть глубже. Аргументы не равны данным, которые пользователь вводит через Scanner после старта.

Инструкция обычно заканчивается `;`. Блок объединяет инструкции в `{ ... }`. Отступ делает вложенность видимой человеку, но блок задают скобки. Имена чувствительны к регистру: `main`, `Main` и `MAIN` различаются. Комментарий `//` продолжается до конца строки, `/* ... */` занимает блок. Комментарии не выполняются.

Отдельный учебный пример

JAVA

```
public class Example01 {
    public static void main(String[] args) {
        System.out.print("Маршрут: ");
        System.out.println("север");
        System.out.println("Длина: " + 8);
    }
}
```

ВЫВОД ПРИМЕРА

```
Маршрут: север
Длина: 8
```

`print` не переводит строку, `println` переводит. Выражение после + здесь формирует строку. Кавычки обозначают границы строкового литерала и не выводятся. Код выполняется сверху вниз внутри main. Запусти пример отдельно, затем замени один println на print и объясни изменение разметки вывода.

Пакет, путь и имя запуска

Пакет - пространство имён, а не просто декоративная папка. Упражнение `Ex001` находится в пакете `ru.vantaeonx.javaguide.ch01`; первая строка объявляет этот пакет. Путь от корня исходников повторяет его части: `ru/vantaeonx/javaguide/ch01/Ex001.java`. Полное имя класса - пакет и имя через точку.

При ручной компиляции `-d build/classes` задаёт корень байткода. При запуске `-cp build/classes` задаёт место поиска классов, после него пишется полное имя класса без `.java` и `.class`. Запуск случайного файла из другой папки - частая причина `ClassNotFoundException`. В IDEA корнем исходников является `src/main/java`, а не папка `ch01`.

Ошибки и способ диагностики

Если компилятор сообщает о пропущенной скобке в конце файла, причина может быть выше. Начинай с первого сообщения, сверяй пары скобок и кавычек. Если класс называется не как файл, исправь согласованно имя файла и объявления. Если программа ничего не печатает, проверь наличие вызова вывода и то, что запущен нужный main: пустая заготовка в курсе законно ничего не выводит.

Не исправляй все сообщения одновременно наугад. Сделай одно изменение, сохрани, собери снова. Так ты проверяешь гипотезу, а не подгоняешь текст под исчезновение красного цвета.

Проверка понимания

Объясни без подсказки различие JDK, JVM и IDEA. Расскажи, почему успешная компиляция не доказывает правильность вычислений. Найди корень исходников и полное имя первого упражнения. Перед практикой запусти пример и получи ровно две строки вывода.

ГЛАВА 01 / EX001

Табличка экспедиции

Уровень 1 · Написать код · Ex001.java

Перед началом

println завершает строку, а print продолжает её. Сначала спланируй разметку, затем вызовы.

Твоя задача

Выведи три строки: «Экспедиция Лира», «Сектор B-7», «Связь установлена». Используй переменные из заготовки и ровно три вызова println.

```java
String mission = "Экспедиция Лира";
String sector = "Сектор B-7";
String status = "Связь установлена";
```

Проверь себя

- Ровно три строки, без пустой строки между ними.
- Замена sector на «Сектор C-2» меняет только вторую строку.

План / прогноз / причина ошибки

ГЛАВА 01 / EX002

Где перевод строки

Уровень 1 · Предсказать результат · Ex002.java

Перед началом

Последовательность вызовов важнее числа строк исходника.

Твоя задача

До запуска запиши точный вывод, включая пробелы и переносы. Затем выполни код и объясни каждый перевод строки.

```java
System.out.print("A");
System.out.print(" B");
System.out.println("C");
System.out.println("D");
```

Проверь себя

- Прогноз записан до запуска.
- Отмечено, какие вызовы добавляют перевод строки; сам ответ в условии не дан.

План / прогноз / причина ошибки

ГЛАВА 01 / EX003

Класс потерял имя

Уровень 1 · Найти ошибку · Ex003.java

Перед началом

Внешнее имя файла и имя public-класса должны согласовываться.

Твоя задача

В комментарии дан отдельный файл Probe.java. Найди все причины, по которым он не компилируется. Исправленный вариант запиши в рабочей области без второго public-класса; результат main должен быть «Готово».

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
// Воображаемый файл Probe.java
public class probe {
    public static void Main(String[] args) {
        system.out.println("Готово")
    }
}
```

Проверь себя

- Укажи нарушения имени и регистра, а также синтаксиса.
- Исправленная рабочая область текущего файла компилируется и печатает «Готово».

План / прогноз / причина ошибки

ГЛАВА 01 / EX004

Квитанция терминала

Уровень 2 · Написать код · Ex004.java

Перед началом

Escape-последовательности входят в строковое значение и управляют его представлением.

Твоя задача

Выведи ровно две строки: «Файл: "route.txt"» и «Папка: C:\maps». Используй один println и \n внутри литерала. Не добавляй пустую третью строку.

Проверь себя

- Кавычки вокруг route.txt видны.
- Обратный слеш перед maps виден ровно один раз; строк ровно две.

План / прогноз / причина ошибки

ГЛАВА 01 / EX005

Что делает пакет

Уровень 2 · Объяснить поведение · Ex005.java

Перед началом

Имя пакета и корень classpath вместе определяют адрес класса.

Твоя задача

Своими словами объясни, почему пакет ru.vantaeonx.javaguide.ch01 не совпадает с абсолютным путём на диске. Запиши правильное полное имя Ex005 и команду запуска после сборки в build/classes.

Проверь себя

- Указан именно корень байткода в -cp.
- В имени запуска нет расширения файла и слешей.

План / прогноз / причина ошибки

ГЛАВА 01 / EX006

Лишняя точка

Уровень 2 · Исправить программу · Ex006.java

Перед началом

Компилятор проверяет форму записи до попытки печати.

Твоя задача

Исправь фрагмент, чтобы получить «Координата: 9». Сохрани переменную x и один println. Объясни первое сообщение компилятора при переносе исходного фрагмента.

```java
int x = 9;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
System.out.println("Координата: " + x;)
```

Проверь себя

- Печатается одна строка с одним пробелом после двоеточия.
- Имя переменной не подменено строковым литералом.

План / прогноз / причина ошибки

ГЛАВА 01 / EX007

Рамка паспорта

Уровень 3 · Написать код · Ex007.java

Перед началом

Строковое соединение позволяет вставлять данные в заранее заданную форму.

Твоя задача

Построй четыре строки: +------------+, | код: K-19 |, | тип: зонд |, +------------+. Не выравнивай общую ширину: воспроизведи именно эти строки. Код и тип бери из переменных.

```java
String code = "K-19";
String type = "зонд";
```

Проверь себя

- Выведены точно четыре заданные строки.
- При code = "M-20" изменяется только код во второй строке.

План / прогноз / причина ошибки

ГЛАВА 01 / EX008

Запуск с другого места

Уровень 3 · Написать код · Ex008.java

Перед началом

Рабочая папка и classpath решают разные задачи.

Твоя задача

Собери текущий класс терминальной командой --release 17 в build/classes. Из папки /tmp запусти его с абсолютным classpath. В main добавь вывод «Запуск независим от папки». В комментарии сохрани обе команды без личных секретов.

Проверь себя

- Сборка и запуск выполнены из разных папок.
- Объяснено, почему относительный build/classes из /tmp не нашёл бы классы.

План / прогноз / причина ошибки

ГЛАВА 01 / EX009

Два класса с одной ролью

Уровень 3 · Написать код · Ex009.java

Перед началом

Пакет устраняет столкновения коротких имён, а не меняет тело main.

Твоя задача

В комментариях спроектируй размещение двух классов с коротким именем Launcher: ru.demo.client и ru.demo.server. Для каждого укажи package, путь от src/main/java и полное имя запуска. В рабочем main выведи эти два полных имени.

Проверь себя

- Два пути различны и согласованы с package.
- В одном файле не объявлены два public-класса; вывод содержит два полных имени.

План / прогноз / причина ошибки

ГЛАВА 02 / ТЕОРИЯ

02. Переменные, типы и значения

Зачем тип известен заранее

Переменная - именованное место для значения определённого типа. Объявление `int count = 4` одновременно задаёт тип, имя и начальное значение. Присваивание меняет значение, а не смысл имени и не объявленный тип. В отличие от привычных сценариев PHP, строка с цифрами не превращается автоматически в целое при присваивании int.

Статическая проверка позволяет обнаружить несовместимость до запуска. Она не знает всего о реальности: int может хранить отрицательный возраст, если программа это разрешила. Тип ограничивает представление, а предметный смысл проверяет твой код.

Восемь примитивов

`byte`, `short`, `int`, `long` хранят целые числа со знаком, соответственно 8, 16, 32 и 64 бита. Для обычного счётчика подходит int, для больших сумм - long. `float` и `double` хранят приближённые двоичные значения; double обычно удобнее для учебных измерений. `char` хранит одну 16-битную кодовую единицу UTF-16, которая не всегда является целым символом для человека. `boolean` принимает только true или false. У boolean нет арифметического значения 0 или 1.

Литерал 7 имеет тип int, `7L` - long, `7.0` - double, `7.0f` - float. Суффикс L нужен до вычисления, а не после переполнения. Символьный литерал пишется в одинарных кавычках, строковый - в двойных: `'Q'` и `"Q"` имеют разные типы.

Преобразование и потеря информации

Расширение int в long сохраняет целое значение. Не всякое расширение сохраняет точность: большой long при переводе в double может округлиться. Сужение double в int отбрасывает дробную часть по направлению к нулю; это не округление до ближайшего. Явное приведение `(int)` говорит компилятору, что риск принят, но не делает результат математически точным.

В выражении байты и короткие целые обычно продвигаются до int. Поэтому сумма двух byte не обязана помещаться в byte даже при допустимых исходных значениях. Не ставь приведение только ради исчезновения ошибки: сначала оцени диапазон результата.

JAVA

```
public class Example02 {
    public static void main(String[] args) {
        int pages = 125;
        long totalPages = pages;
        double measure = -3.8;
        int truncated = (int) measure;
        final int DAYS = 7;
        System.out.println(totalPages);
        System.out.println(truncated);
        System.out.println(DAYS);
    }
}
```

ВЫВОД ПРИМЕРА

```
125
-3
7
```

Исходная measure не меняется при приведении: преобразованное значение записано в другую переменную. `final` запрещает повторное присваивание этой переменной. Позже мы увидим: final-ссылка не делает сам объект неизменяемым.

Инициализация, область видимости, ссылки

Локальную переменную нужно инициализировать до чтения. Компилятор требует, чтобы значение существовало на каждом возможном пути к использованию. Поля объектов получают значения по умолчанию, но это другой механизм, который не нужно переносить на локальные переменные.

Область видимости локального имени ограничена блоком объявления. Переменная внутри if не становится доступной после закрывающей скобки. Выбирай минимальную достаточную область: это уменьшает число мест, где значение можно случайно изменить.

String и массивы не примитивы. Переменная ссылочного типа хранит ссылочное значение, позволяющее обратиться к объекту, либо null. Нельзя использовать null как ноль или пустую строку. Присваивание ссылки не копирует объект. Эта разница станет центральной при изучении массивов и классов.

В Java 17 `var` разрешает вывод типа некоторых локальных переменных из инициализатора. Это не динамическая типизация: `var n = 2` всё равно создаёт int. В первых заданиях используй явные типы, чтобы выбор представления был осознанным.

Ошибки и проверка понимания

Не считай double универсальной заменой всех типов: он плохо подходит для точного учёта денежных единиц. Для учебных денежных задач используй целое число центов; BigDecimal можно изучить позднее. Не используй char как гарантированный «один эмодзи».

Перед практикой назови тип литералов 12, 12L, `12.0`, `'1'`, `"1"`. Объясни, какие сведения теряются при сужении и почему локальная переменная без значения не равна нулю.

ГЛАВА 02 / EX010

Паспорт датчика

Уровень 1 · Написать код · Ex010.java

Перед началом

Тип выбирают по диапазону и смыслу значения, а не по внешнему виду цифр.

Твоя задача

Объяви новые переменные подходящих типов для номера порта 2048, большой счётной величины 5000000000, температуры 18.75, признака готовности true и метки Q. Выведи каждое значение с подписью.

Проверь себя

- Большое целое хранится без потери и имеет суффикс L в литерале.
- Метка Q хранится как char; готовность как boolean.

План / прогноз / причина ошибки

ГЛАВА 02 / EX011

Сужение не округляет

Уровень 1 · Предсказать результат · Ex011.java

Перед началом

Приведение создаёт новое значение; оно не меняет источник.

Твоя задача

Предскажи три строки вывода и поясни направление отбрасывания дробной части.

JAVA

```
double source = -7.9;
int narrow = (int) source;
long wide = narrow;
System.out.println(source);
System.out.println(narrow);
System.out.println(wide);
```

Проверь себя

- Сначала записан прогноз.
- Для каждой переменной указаны тип и значение после присваивания.

План / прогноз / причина ошибки

ГЛАВА 02 / EX012

Байт слишком узок

Уровень 1 · Найти ошибку · Ex012.java

Перед началом

Результат арифметики над byte обычно имеет тип int.

Твоя задача

Найди причину ошибки компиляции. Предложи исправление без сужающего приведения и с сохранением точной суммы.

```java
byte left = 100;
byte right = 60;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
byte total = left + right;
System.out.println(total);
```

Проверь себя

- Сумма не оборачивается в отрицательное число.
- Объяснено, почему даже два byte не гарантируют результат byte.

План / прогноз / причина ошибки

ГЛАВА 02 / EX013

Граница целого

Уровень 2 · Написать код · Ex013.java

Перед началом

Строка и целое отличаются представлением, даже если показывают те же цифры.

Твоя задача

Из text получи int через Integer.parseInt. Выведи число плюс 1 и отдельно text плюс "1". Ввод не нужен.

```java
String text = "249";
```

Проверь себя

- Числовой результат 250.
- Текстовый результат 2491; исходная строка не изменена.

План / прогноз / причина ошибки

ГЛАВА 02 / EX014

Два нуля из разных миров

Уровень 2 · Объяснить поведение · Ex014.java

Перед началом

Автоматическая инициализация полей не распространяется на локальные переменные.

Твоя задача

Объясни, почему локальное int value без присваивания нельзя прочитать, а поле int в новом объекте изначально равно 0. Запиши минимальные фрагменты в комментариях и укажи, какой не компилируется.

Проверь себя

- Не утверждай, что все переменные Java автоматически обнуляются.
- Различены область видимости и время жизни.

План / прогноз / причина ошибки

ГЛАВА 02 / EX015

Повторное присваивание final

Уровень 2 · Исправить программу · Ex015.java

Перед началом

Final ограничивает присваивание переменной, а не все действия в программе.

Твоя задача

Исправь фрагмент: исходный limit должен оставаться 12, увеличенный результат храниться в отдельной переменной. Выведи оба.

```java
final int limit = 12;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
limit = limit + 1;
System.out.println(limit);
```

Проверь себя

- Вывод содержит 12 и 13.
- Final не удалён.

План / прогноз / причина ошибки

ГЛАВА 02 / EX016

Перевод объёма

Уровень 3 · Написать код · Ex016.java

Перед началом

Перед преобразованием оцени максимальный промежуточный результат.

Твоя задача

Для числа гигабайт вычисли число байт по правилу 1 GiB = 1024*1024*1024 байт. Используй long уже в вычислении. Выведи точное целое.

```java
int gib = 6;
```

Проверь себя

- 6 -> 6442450944.
- 0 -> 0; 3 -> 3221225472; промежуточное произведение не int.

План / прогноз / причина ошибки

ГЛАВА 02 / EX017

Цена потери точности

Уровень 3 · Написать код · Ex017.java

Перед началом

При переводе большого целого в double часть младших разрядов может исчезнуть.

Твоя задача

Преобразуй original в double и обратно в long. Выведи исходное, восстановленное и равенство. Повтори для 42. Объясни наблюдение, не используя строковое преобразование.

```java
long original = 9007199254740993L;
```

Проверь себя

- Большое целое после round-trip отличается.
- 42 сохраняется; объяснение связано с представлением double, а не случайной работой JVM.

План / прогноз / причина ошибки

ГЛАВА 02 / EX018

Область жизни счётчика

Уровень 3 · Написать код · Ex018.java

Перед началом

Объявление внутри блока не создаёт имя во внешнем блоке.

Твоя задача

В комментариях изобрази две вложенные области с переменными count и local. В рабочем коде сделай count доступным после внутреннего блока и увеличь его на local только внутри. Исходное count=2, local=5; выведи итог.

```java
int count = 2;
```

Проверь себя

- Результат 7.
- Local объявлена внутри блока и не используется после него; попытка такого использования объяснена в комментарии.

План / прогноз / причина ошибки

ГЛАВА 03 / ТЕОРИЯ

03. Выражения, арифметика и логика

Тип результата определяется выражением

Выражение вычисляет значение. Операция `a + b` не обязана иметь тип переменной, в которую её результат будет записан. В `double x = 7 / 2` деление выполняется над двумя int, даёт целое 3, и только затем оно превращается в 3.0. Для дробного деления хотя бы один операнд должен стать дробным до операции.

У целого деления дробная часть отбрасывается к нулю. Остаток `%` согласован с этим делением: для целых выполняется `a == (a / b) * b + a % b`, если b не равен нулю. Знак ненулевого остатка совпадает со знаком делимого. Это важно для отрицательных координат: остаток Java не всегда совпадает с математическим модулем; для циклических индексов существует Math.floorMod.

Порядок операций и побочные эффекты

Умножение и деление имеют более высокий приоритет, чем сложение. Скобки показывают замысел даже там, где технически не нужны. Операнды Java вычисляет слева направо; приоритет задаёт группировку, а не разрешение переставлять вызовы произвольно.

x++ возвращает старое значение и затем увеличивает x. ++x сначала увеличивает. Когда увеличение нужно само по себе, отдельная инструкция x++ яснее сложного выражения. Избегай нескольких изменений одной переменной в одном выражении: такая запись тренирует угадывание, а не проектирование.

Оператор + соединяет строки, если операция уже стала строковой. Поэтому `"Сумма: " + 2 + 3` последовательно приклеит две цифры. Группировка числовой части скобками меняет результат. Печать - тоже место, где нужно учитывать тип промежуточного значения.

JAVA

```
public class Example03 {
    public static void main(String[] args) {
        int minutes = 95;
        System.out.println(minutes / 60 + ":" + minutes % 60);
        System.out.println(9 / 4);
        System.out.println(9 / 4.0);
        System.out.println(-9 % 4);
        boolean safe = minutes > 0 && 120 / minutes >= 1;
        System.out.println(safe);
    }
}
```

ВЫВОД ПРИМЕРА

```
1:35
2
2.25
-1
true
```

Переполнение и точность

Целые примитивы имеют конечный диапазон. Обычное сложение int при переполнении не выдаёт исключение: результат оборачивается в пределах 32 бит. Если два int перемножаются до присваивания long, переполнение уже произошло. Расширь один операнд заранее либо используй подходящую проверяемую операцию Math.addExact или Math.multiplyExact.

Double представляет многие десятичные дроби приближённо. Результат 0.1 + 0.2 не обязан быть в точности 0.3. Форматирование до двух знаков меняет показ пользователю, но не внутреннее значение. Допуск для сравнения выбирают из смысла измерения, а не один магический порог для любой задачи.

Деление целого на ноль вызывает ArithmeticException. Дробное деление использует правила IEEE 754 и может дать Infinity или NaN. Для начинающего приложения лучше заранее определить допустимость знаменателя, чем ждать удобного поведения типа.

Булева логика

Сравнения ==, !=, <, <=, > и >= возвращают boolean. && требует истинности обеих частей, || - хотя бы одной, ! отрицает значение. Короткое замыкание означает, что правая часть && не вычисляется при false слева, а правая часть || - при true слева. Проверку безопасности ставят перед потенциально опасным выражением.

& и | с boolean вычисляют обе стороны. Для целых они означают побитовые операции. Не заменяй ими && и || случайно. Присваивание = и сравнение == выполняют разные действия; равенство ссылок не равно равенству содержимого строк.

Типичные ошибки и самопроверка

Если результат выглядит слишком маленьким, проверь целое деление и типы до операции. Если огромное произведение стало отрицательным, оцени диапазон промежуточного выражения. Если условие «между» записано как `0 < x < 10`, разбей его на два boolean-сравнения с &&.

Объясни, почему `(double) (a / b)` и `((double) a) / b` различаются. Назови ситуацию, где порядок двух частей && защищает программу. Перед практикой вручную вычисли промежуточные типы в строковом сложении.

ГЛАВА 03 / EX019

Пакеты энергии

Уровень 1 · Написать код · Ex019.java

Перед началом

Целое деление и остаток отвечают на разные вопросы упаковки.

Твоя задача

Определи число полных батарей по 24 единицы и остаток единиц. Выведи два целых.

```java
int energy = 137;
int perBattery = 24;
```

Проверь себя

- 137 -> 5 полных, остаток 17.

- 0 -> 0 и 0; 48 -> 2 и 0.

План / прогноз / причина ошибки

ГЛАВА 03 / EX020

Строка собирается слева

Уровень 1 · Предсказать результат · Ex020.java

Перед началом

Промежуточный тип определяет смысл следующего плюса.

Твоя задача

Предскажи точный вывод обеих строк и подпиши порядок группировки.

```java
System.out.println("Код " + 3 + 4);
System.out.println("Код " + (3 + 4));
```

Проверь себя

- Объясни момент перехода к строковому соединению.
- Не меняй исходный код до записи прогноза.

План / прогноз / причина ошибки

ГЛАВА 03 / EX021

Дробь стала целой

Уровень 1 · Найти ошибку · Ex021.java

Перед началом

Приведение после целого деления не возвращает потерянную дробную часть.

Твоя задача

Найди ошибку расчёта доли. Исправь выражение так, чтобы деление сразу было дробным.

JAVA

```
int used = 7;
int all = 8;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
double fraction = (double) (used / all);
System.out.println(fraction);
```

Проверь себя

- 7/8 -> 0.875.

- 1/4 -> 0.25; область задачи all > 0.

План / прогноз / причина ошибки

ГЛАВА 03 / EX022

Расстояние в сантиметрах

Уровень 2 · Написать код · Ex022.java

Перед началом

Точное целое представление удобно для единиц фиксированного масштаба.

Твоя задача

Из wholeMeters и extraCm получи общее число сантиметров как long. Разрешены wholeMeters >= 0 и extraCm от 0 до 99. Выведи целое.

```java
int wholeMeters = 30000000;
int extraCm = 75;
```

Проверь себя

- 30000000,75 -> 3000000075.
- 0,0 -> 0; умножение не переполняет int до расширения.

План / прогноз / причина ошибки

ГЛАВА 03 / EX023

Защитное условие

Уровень 2 · Объяснить поведение · Ex023.java

Перед началом

Короткое замыкание имеет значение для безопасности правой части.

Твоя задача

Объясни, почему выражение в комментарии безопасно при divisor=0, а замена && на & опасна. Ничего не дели при нуле в рабочем main.

```java
int divisor = 0;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
// Выражение для анализа, исходное && безопасно:
boolean ok = divisor != 0 && 10 / divisor > 1;
// Опасный вариант: divisor != 0 & 10 / divisor > 1
```

Проверь себя

- Указано, вычисляется ли 10/divisor.
- Объяснение различает булев результат и порядок вычисления.

План / прогноз / причина ошибки

ГЛАВА 03 / EX024

Переполнение до присваивания

Уровень 2 · Исправить программу · Ex024.java

Перед началом

Тип левой части присваивания не меняет уже выполненное произведение.

Твоя задача

Исправь вычисление площади без изменения a и b на long. Расширь один операнд до умножения.

```java
int a = 70000;
int b = 60000;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
long area = a * b;
System.out.println(area);
```

Проверь себя

- Площадь 4200000000.
- 1,1 -> 1; формула сохраняет long-результат.

План / прогноз / причина ошибки

ГЛАВА 03 / EX025

Циклический сектор

Уровень 3 · Написать код · Ex025.java

Перед началом

Остаток с отрицательным делимым не равен всегда неотрицательному индексу.

Твоя задача

Для любого int coordinate вычисли индекс 0..7 через Math.floorMod. Выведи его и объясни, почему coordinate % 8 недостаточно для отрицательных координат.

```java
int coordinate = -17;
```

Проверь себя

- -17 -> 7.

- 0 -> 0; 15 -> 7; -8 -> 0.

План / прогноз / причина ошибки

ГЛАВА 03 / EX026

Условие доступа

Уровень 3 · Написать код · Ex026.java

Перед началом

Комбинацию требований удобно проверять таблицей истинности.

Твоя задача

Вычисли canOpen: ключ есть И (есть питание ИЛИ включён ручной режим). Выведи boolean. Составь в комментариях все 8 комбинаций.

```java
boolean hasKey = true;
boolean powered = false;
boolean manual = true;
```

Проверь себя

- Исходный случай -> true.
- Без ключа всегда false; с ключом и обоими выключенными режимами false.

План / прогноз / причина ошибки

ГЛАВА 03 / EX027

Расчёт точных центов

Уровень 3 · Написать код · Ex027.java

Перед началом

Целые центы устраняют двоичную погрешность десятичных сумм.

Твоя задача

Вычисли общую цену quantity товаров unitCents, затем скидку discountCents один раз на заказ. Входы неотрицательны, скидка не превышает сумму. Выведи long-итог в центах; формат валюты не нужен.

```java
int quantity = 80000;
int unitCents = 45000;
int discountCents = 150;
```

Проверь себя

- Итог 3599999850.
- quantity=0 и discountCents=0 -> 0; произведение расширено заранее.

План / прогноз / причина ошибки

ГЛАВА 04 / ТЕОРИЯ

04. Ввод как договор о формате данных

Почему ввод сложнее готовой переменной

При присваивании `int amount = 6` данные уже проверил компилятор. В терминале пользователь передаёт текст: пустую строку, лишние пробелы, буквы, число вне диапазона. Программа должна понимать формат, преобразовать его и решить, что делать при нарушении договора. Пока обработка исключений не изучена, в первых заданиях формат гарантируется условием. В главе 14 вернёмся к ошибочному вводу.

Scanner оборачивает источник и читает из него данные. `System.in` представляет стандартный ввод. Импорт `java.util.Scanner` позволяет использовать короткое имя Scanner. Импорт не загружает стороннюю библиотеку: это класс стандартной платформы.

Токен и строка - разные единицы

`next()` читает следующий токен, обычно до пробельного разделителя. `nextInt()` читает токен и преобразует его в int. `nextLine()` читает остаток текущей строки до её конца и потребляет разделитель строки. После nextInt перевод строки часто ещё находится во входе. Немедленный nextLine тогда вернёт оставшуюся часть, часто пустую.

Есть два понятных протокола. Первый: читать все поля по строкам и явно преобразовывать числовые строки через Integer.parseInt. Второй: читать токены, а при переходе к построчному формату осознанно потреблять остаток строки. Не добавляй лишний nextLine везде «на всякий случай»: он может проглотить полезные данные.

JAVA

```
import java.util.Scanner;

public class Example04 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        String city = input.nextLine();
        int platforms = Integer.parseInt(input.nextLine().trim());
        System.out.println(city + ": " + (platforms + 1));
    }
}
```

СТАНДАРТНЫЙ ВВОД

```
Новая пристань
4
```

ВЫВОД ПРИМЕРА

```
Новая пристань: 5
```

Здесь город может содержать пробелы, а число занимает следующую строку. `trim()` удаляет края числовой строки, не меняя city. Пример не объявляет, что любое

содержимое терминала допустимо: пустая или нечисловая вторая строка вызовет ошибку преобразования. Контракт ввода нужно писать в условии, а позже проверять в коде.

Локаль и вывод

Scanner.nextDouble зависит от локали: разделителем дробной части может оказаться запятая или точка. Чтобы учебные проверки воспроизводились, договоримся о точке и используем `input.useLocale(Locale.ROOT)` для токенового чтения double. Double.parseDouble также принимает обычную десятичную запись с точкой. Не смешивай формат, который обещан пользователю, с настройками машины.

Для форматированного вывода `printf(Locale.ROOT, "%.2f%n", value)` задаёт две цифры после точки и платформенный перевод строки. Это представление результата, а не изменение его точности. Не округляй каждое промежуточное измерение без причины: ошибку округления можно накопить.

Источник данных и время жизни

Используй один Scanner над System.in на запуск программы. Несколько обёрток могут буферизовать один источник и мешать друг другу. Закрытие Scanner закрывает связанный источник, поэтому общий ввод не следует закрывать внутри вспомогательного метода, пока приложение ещё читает. Для короткого main окончание процесса освобождает ввод; для файловой обёртки нужно управляемое закрытие, которое изучим отдельно.

В заготовках на ввод оставлен комментарий о точном протоколе. Ты пишешь чтение самостоятельно. Если используются фиксированные исходные данные, задание прямо говорит, что ввод не нужен. Это два разных варианта упражнения, а не повод незаметно заменить один другим.

Ошибки и проверка понимания

Если «пропало имя», восстанови последовательность токенов и переводов строк. Если число не принимается, проверь разделитель и диапазон int. Если программа ждёт, возможно, ей ещё нужна строка по контракту; курс не требует сразу читать весь ввод одной командой.

Перед практикой нарисуй вход как символы с отметками `\n`. Покажи, какую часть забирает каждый вызов Scanner. Объясни различие между синтаксически верным целым и допустимым числом предметов. Вторую проверку программа выполняет отдельно.

ГЛАВА 04 / EX028

Название и количество

Уровень 1 · Написать код · Ex028.java

Перед началом

Построчный протокол сохраняет пробелы в названии.

Твоя задача

Прочитай название прибора из первой строки и целое количество из второй. Выведи «название: количество». Число корректное, >=0; лишних строк нет.

СТАНДАРТНЫЙ ВВОД

```
Северный маяк
3
```

Проверь себя

- Вход «Северный маяк», затем 3 -> «Северный маяк: 3».
- Вход «Зонд», затем 0 -> «Зонд: 0».

План / прогноз / причина ошибки

ГЛАВА 04 / EX029

Остаток строки

Уровень 1 · Предсказать результат · Ex029.java

Перед началом

После токенового чтения конец строки может оставаться во входе.

Твоя задача

Предскажи два вывода при заданном вводе. Объясни, что именно читает nextLine. Не исправляй код в этом задании.

JAVA

```
Scanner input = new Scanner(System.in);
int count = input.nextInt();
String title = input.nextLine();
System.out.println(count);
System.out.println("[" + title + "]");
```

СТАНДАРТНЫЙ ВВОД

```
5
Белая башня
```

Проверь себя

- Ввод строго соответствует приведённым двум строкам.
- Прогноз записан до запуска; указано состояние Scanner после nextInt.

План / прогноз / причина ошибки

ГЛАВА 04 / EX030

Название обрезалось

Уровень 1 · Найти ошибку · Ex030.java

Перед началом

Next читает токен, а не всю строку.

Твоя задача

Найди ошибку чтения полного названия. Исправь, сохранив договор: первая строка целиком является названием.

JAVA

```
Scanner input = new Scanner(System.in);
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
String title = input.next();
System.out.println(title);
```

СТАНДАРТНЫЙ ВВОД

```
Лунная станция
```

Проверь себя

- «Лунная станция» сохраняется целиком.
- Пробелы внутри названия не удаляются.

План / прогноз / причина ошибки

ГЛАВА 04 / EX031

Два измерения

Уровень 2 · Написать код · Ex031.java

Перед началом

Явная локаль делает десятичный протокол воспроизводимым.

Твоя задача

Прочитай два положительных double-токена с точкой, вычисли их сумму и выведи с двумя знаками после точки через printf(Locale.ROOT,...).

СТАНДАРТНЫЙ ВВОД

```
1.25 2.50
```

Проверь себя

- 1.25 2.50 -> 3.75.
- 0.10 0.20 -> 0.30; Scanner настроен на Locale.ROOT.

План / прогноз / причина ошибки

ГЛАВА 04 / EX032

Кто закрывает ввод

Уровень 2 · Объяснить поведение · Ex032.java

Перед началом

Закрытие обёртки влияет на общий источник.

Твоя задача

Объясни, почему вспомогательный метод, который создаёт и закрывает Scanner(System.in), может сломать последующий ввод main. Предложи договор владения одним Scanner.

Проверь себя

- Различены закрытие ресурса и конец локальной переменной.
- Предложено передавать Scanner или данные, а не создавать конкурирующие обёртки.

План / прогноз / причина ошибки

ГЛАВА 04 / EX033

Локаль не угадана

Уровень 2 · Исправить программу · Ex033.java

Перед началом

Настройки пользователя не должны случайно менять формат учебного ввода.

Твоя задача

Исправь фрагмент так, чтобы токен 2.75 принимался независимо от локали Ubuntu. Не заменяй точку во входе запятой.

JAVA

```
Scanner input = new Scanner(System.in);
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
input.useLocale(Locale.FRANCE);
double value = input.nextDouble();
System.out.println(value);
```

СТАНДАРТНЫЙ ВВОД

```
2.75
```

Проверь себя

- 2.75 -> вывод 2.75.
- Используется Locale.ROOT до nextDouble; договор о точке сохранён.

План / прогноз / причина ошибки

ГЛАВА 04 / EX034

Число затем описание

Уровень 3 · Написать код · Ex034.java

Перед началом

Смешивание токенов и строк требует явного перехода.

Твоя задача

Прочитай целое code через nextInt из первой строки, затем полное описание из второй. Остаток первой строки после числа пуст. Выведи code и описание на отдельных строках.

СТАНДАРТНЫЙ ВВОД

```
17
сектор дальнего наблюдения
```

Проверь себя

- 17 и «сектор дальнего наблюдения» сохраняются.
- Описание не пусто из-за забытого конца первой строки.

План / прогноз / причина ошибки

ГЛАВА 04 / EX035

Три строки протокола

Уровень 3 · Написать код · Ex035.java

Перед началом

Форматирование принадлежит границе вывода, а не арифметике.

Твоя задача

Прочитай название, целое число штук и double-массу одной штуки, каждое с новой строки. Выведи одну строку «название | N шт. | M кг», где M - общая масса с 3 знаками. Ввод корректный, числа неотрицательны.

СТАНДАРТНЫЙ ВВОД

```
Крепёж
4
0.125
```

Проверь себя

- «Крепёж», 4, 0.125 -> «Крепёж | 4 шт. | 0.500 кг».
- Ноль штук даёт 0.000 кг; название может содержать пробелы.

План / прогноз / причина ошибки

ГЛАВА 04 / EX036

Два источника параметров

Уровень 3 · Написать код · Ex036.java

Перед началом

Аргументы запуска и стандартный ввод поступают в разные моменты.

Твоя задача

Программа получает ровно один аргумент args[0]: имя оператора. Прочитай целое число смен из единственной строки стандартного ввода и выведи «имя: смены». Число корректное; проверять отсутствие аргумента в этом задании не нужно.

СТАНДАРТНЫЙ ВВОД

```
2
```

Проверь себя

- С аргументом Лея и вводом 2 -> «Лея: 2».
- С аргументом Рин и вводом 3 -> «Рин: 3»; имя не читается из Scanner.

План / прогноз / причина ошибки

ГЛАВА 05 / ТЕОРИЯ

05. Условия и полная система случаев

Ветка выбирает действие

`if` выполняет блок, когда boolean-условие истинно. `else` выполняется при ложном условии. В цепочке if / else if / else выбирается первая подходящая ветка, остальные не проверяются. Несколько отдельных if независимы: могут выполниться несколько блоков. Выбор между ними определяется тем, должны ли результаты исключать друг друга.

Полезно начать с таблицы случаев: какие входы допустимы, как они разделяются и что делать на границе. Если температура ровно равна порогу, программа должна отнести её к конкретной категории. Слова «больше», «не меньше» и «от ... до ... включительно» имеют разные переводы в операторы.

Порядок проверок

Сначала можно отвергать недопустимые входы, затем классифицировать допустимые. Внутри цепочки более узкие случаи должны предшествовать более широким, если иначе они станут недостижимыми. Например, проверка «значение положительное» поглотит «значение больше ста», если поставлена первой и обе находятся в одной цепочке.

Проверка диапазона состоит из двух сравнений. Логика «вне диапазона» использует ||, а «внутри» - &&. Отрицание составного условия можно получить по законам де Моргана: отрицание A && B равносильно !A || !B. Это инструмент проверки, а не обязанность писать максимально сложную формулу.

JAVA

```
public class Example05 {
    public static void main(String[] args) {
        int battery = 37;
        String state;
        if (battery < 0 || battery > 100) {
            state = "Некорректно";
        } else if (battery < 20) {
            state = "Зарядить";
        } else {
            state = "Можно ехать";
        }
        System.out.println(state);
    }
}
```

ВЫВОД ПРИМЕРА

```
Можно ехать
```

Все ветки присваивают state, поэтому после цепочки значение определено. Если убрать else, компилятор может обнаружить путь без присваивания. Эта проверка полезнее случайного начального значения, которое маскировало бы неполную систему случаев.

Switch и условное выражение

Switch удобен для выбора по дискретному значению: команде, номеру, enum. В Java 17 можно использовать стрелочную форму `case ... -> ...`; она не проваливается в следующую ветку. В старой форме с двоеточием часто нужен break. В курсе простые переключения записаны стрелочной формой; это синтаксис Java 17, не preview.

Switch-выражение должно дать результат для всех возможных случаев; switch-инструкция выбирает действие. Для произвольных числовых диапазонов обычно яснее if. В Java 17 switch по String не принимает null: проверяй такой вход заранее, если null разрешён.

Тернарное выражение `condition ? a : b` возвращает одно из двух значений. Оно удобно для короткого выбора подписи. Не вкладывай несколько таких выражений вместо читаемой системы веток. Ветка, которую не выбрали, не вычисляется.

Строки и осторожность со скобками

Для сравнения текстового содержимого используй equals. Оператор == проверяет совпадение ссылочных значений; некоторые литералы могут давать true из-за интернирования, создавая ложную уверенность в ошибочном подходе. Запись `"go".equals(command)` безопасна даже при command == null, если именно такое поведение нужно договору.

Пустая инструкция после if, например лишняя точка с запятой, делает условный блок пустым. Следующая строка выполняется независимо. Всегда ставь фигурные скобки: добавление второй инструкции позже не должно незаметно изменить её условность.

Границы и самопроверка

Для каждого порога проверь значение ниже, ровно на пороге и выше. Для двух независимых признаков составь четыре комбинации true/false. Если два условия перекрываются, запиши, кто имеет приоритет. Если не покрывают все входы, явно определи запасную ветку или ошибку.

Объясни различие двух if и if/else if. Покажи, почему `x != 1 || x != 2` истинно для любого int. Перед практикой составь таблицу решений на бумаге; код должен стать переводом этой таблицы.

ГЛАВА 05 / EX037

Сигнал уровня

Уровень 1 · Написать код · Ex037.java

Перед началом

Непересекающиеся интервалы дают одну подпись для каждого допустимого значения.

Твоя задача

Для int level от 0 до 100 выведи LOW при level<30, NORMAL при 30..79 и HIGH при 80..100. Недопустимые значения дают INVALID.

```java
int level = 80;
```

Проверь себя

- 80 -> HIGH; 29 -> LOW; 30 -> NORMAL.
- 79 -> NORMAL; -1 и 101 -> INVALID.

План / прогноз / причина ошибки

ГЛАВА 05 / EX038

Два независимых if

Уровень 1 · Предсказать результат · Ex038.java

Перед началом

Несколько if могут выполнить несколько действий.

Твоя задача

Предскажи строки при value=12. Затем объясни отличие от замены второго if на else if.

JAVA

```
int value = 12;
if (value > 0) { System.out.println("positive"); }
if (value > 10) { System.out.println("large"); } else { System.out.println("small"); }
```

Проверь себя

- Записаны прогноз и причина выбора ветвей.
- Код не изменён до первой проверки.

План / прогноз / причина ошибки

ГЛАВА 05 / EX039

Условие без тела

Уровень 1 · Найти ошибку · Ex039.java

Перед началом

Лишняя точка с запятой может создать пустую инструкцию.

Твоя задача

Найди ошибку: READY должно печататься только при ready=true. Исправь и проверь оба значения.

```java
boolean ready = false;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
if (ready); {
    System.out.println("READY");
}
```

Проверь себя

- false -> пустой вывод.
- true -> одна строка READY.

План / прогноз / причина ошибки

ГЛАВА 05 / EX040

Командный переключатель

Уровень 2 · Написать код · Ex040.java

Перед началом

Switch подходит для точного набора строковых команд.

Твоя задача

Для command выведи «Старт» при start, «Стоп» при stop, «Статус» при status и «Неизвестно» иначе. Используй стрелочный switch Java 17. Command не null.

```java
String command = "status";
```

Проверь себя

- status -> Статус.
- start, stop и пустая строка проверены; регистр значим.

План / прогноз / причина ошибки

ГЛАВА 05 / EX041

Всегда истинное или

Уровень 2 · Объяснить поведение · Ex041.java

Перед началом

Проверь формулу на всех возможных классах входа.

Твоя задача

Объясни, почему условие mode != 1 || mode != 2 не исключает ни 1, ни 2. Запиши корректное условие запрета обоих значений и таблицу для 0,1,2,3 в комментариях.

Проверь себя

- Для каждого входа рассмотрены обе части.
- Объяснено различие «не равен обоим» и «хотя бы одному».

План / прогноз / причина ошибки

ГЛАВА 05 / EX042

Широкая ветка раньше узкой

Уровень 2 · Исправить программу · Ex042.java

Перед началом

Первая истинная ветка цепочки поглощает последующие.

Твоя задача

Исправь классификацию: score>=90 -> gold, 60..89 -> silver, 0..59 -> bronze, вне 0..100 -> invalid.

```java
int score = 95;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
if (score >= 60) { System.out.println("silver"); }
else if (score >= 90) { System.out.println("gold"); }
else { System.out.println("bronze"); }
```

Проверь себя

- 95 -> gold; 60 -> silver; 59 -> bronze.
- -1,101 -> invalid; порядок ветвей объяснён.

План / прогноз / причина ошибки

ГЛАВА 05 / EX043

Проверка даты без библиотеки

Уровень 3 · Написать код · Ex043.java

Перед началом

Составной договор требует отдельно проверить год, месяц и день.

Твоя задача

Проверь григорианскую дату: год 1..9999, месяц 1..12, день в длине месяца. Високосность: делится на 400 либо на 4 и не на 100. Выведи VALID/INVALID. Date API не используй.

```java
int year = 2000;
int month = 2;
int day = 29;
```

Проверь себя

- 2000-02-29 VALID; 1900-02-29 INVALID.
- 2024-04-31 INVALID; год 0 INVALID; сначала проверяется месяц.

План / прогноз / причина ошибки

ГЛАВА 05 / EX044

Приоритет доступа

Уровень 3 · Написать код · Ex044.java

Перед началом

Сначала решается запрет, затем дополнительные возможности.

Твоя задача

Выведи DENIED, если blocked или !authenticated. Иначе ADMIN для admin, EDIT для editor, READ для остальных. Одновременно admin и editor -> ADMIN.

```java
boolean blocked = false;
boolean authenticated = true;
boolean admin = true;
boolean editor = true;
```

Проверь себя

- Исходный -> ADMIN.
- Blocked запрещает даже admin; неавторизованный editor -> DENIED.

План / прогноз / причина ошибки

ГЛАВА 05 / EX045

Тариф с границами

Уровень 3 · Написать код · Ex045.java

Перед началом

Кусочно заданная цена должна точно определить границы и недопустимые входы.

Твоя задача

Вычисли стоимость стоянки в центах: 0 минут бесплатно, 1..60 -> 200, 61..180 -> 500, 181..1440 -> 900. Вне 0..1440 выведи INVALID. Для допустимых выведи только число.

```java
int minutes = 61;
```

Проверь себя

- 0 -> 0; 60 -> 200; 61 -> 500; 180 -> 500.
- 181 и 1440 -> 900; -1 и 1441 INVALID.

План / прогноз / причина ошибки

ГЛАВА 06 / ТЕОРИЯ

06. Циклы: повторение с доказуемой остановкой

Повторяется тело, а не намерение

Цикл нужен, когда одно действие применяется к последовательности данных или выполняется до наступления события. В while сначала проверяется условие, затем тело. Поэтому тело может не выполниться ни разу. В do/while тело выполняется до первой проверки, хотя бы один раз. For объединяет инициализацию, условие и изменение счётчика в одном заголовке.

Для `for (int i = 0; i < n; i++)` порядок таков: создать i, проверить i < n, выполнить тело, увеличить i, снова проверить. После закрытия цикла локальное i из заголовка недоступно. Если n равно нулю, повторений нет. Условие `i <= n` добавляет ещё одно повторение и часто ломает доступ к массиву.

Состояние и инвариант

У цикла есть состояние: счётчик, накопленная сумма, число найденных элементов. Инвариант - утверждение, которое сохраняется при переходе к следующей итерации. Например, перед обработкой элемента с индексом i сумма уже содержит первые i элементов. Это объясняет начальное значение 0 и границу i < length.

Для остановки нужен прогресс: величина, которая приближает состояние к ложному условию. В обратном отсчёте уменьшается счётчик. В чтении до команды выхода источник должен когда-нибудь дать эту команду или закончиться. Если обновление забыто, изменение условия наугад не решает причину.

JAVA

```
public class Example06 {
    public static void main(String[] args) {
        int total = 0;
        for (int day = 1; day <= 4; day++) {
            total += day * 3;
        }
        System.out.println(total);
        int remaining = 3;
        while (remaining > 0) {
            System.out.print(remaining + " ");
            remaining--;
        }
        System.out.println("готово");
    }
}
```

ВЫВОД ПРИМЕРА

```
30
3 2 1 готово
```

В первом цикле до дня k сумма содержит вклад предыдущих дней. Во втором remaining не только печатается, но и уменьшается. Проследи вручную состояние перед проверкой, внутри тела и после изменения: это три разные точки времени.

Break, continue и вложенность

Break завершает ближайший цикл. Continue переходит к следующей итерации: в for выполняется шаг из заголовка, в while выполнение возвращается прямо к условию. Поэтому continue перед обновлением счётчика while способен создать бесконечный цикл.

Во вложенных циклах внутренний цикл запускается заново для каждого шага внешнего. Два прохода по n элементов не равны n*n: последовательные проходы дают около 2n действий, вложенные - около n*n. Эта разница становится заметной на больших данных. Сначала считай, сколько раз выполняется тело, затем обсуждай скорость.

Алгоритмы накопления

Для суммы начальное значение обычно 0, для произведения 1, для счётчика найденных 0. Для максимума неизвестных данных нельзя всегда брать 0: весь набор может быть отрицательным. Используй первый допустимый элемент или отдельно учитывай отсутствие данных. Пока массивы не изучены, это можно увидеть на последовательности значений из ввода.

Условие завершения не всегда является данными для обработки. Если договор задаёт 0 как сигнал остановки, этот 0 обычно не включают в статистику. Если ноль может быть полезным значением, выбери другой протокол: количество элементов заранее или отдельную команду.

Ошибки и проверка понимания

Частые ошибки: сброс суммы внутри тела; изменение не того счётчика; пропуск обновления на ветке continue; неверная граница; включение сигнала остановки в результат. Для проверки возьми минимальный вход: ноль повторений, одно повторение, затем два. Большой пример плохо показывает момент первой ошибки.

Перед практикой составь таблицу `итерация / условие / состояние до / состояние после`. Назови инвариант и причину остановки. Если не можешь их сформулировать, алгоритм ещё не ясен независимо от того, запускается ли он на одном входе.

ГЛАВА 06 / EX046

Метки маршрута

Уровень 1 · Написать код · Ex046.java

Перед началом

For удобен для известного количества повторений.

Твоя задача

Выведи все целые от 3 до limit включительно, каждое отдельной строкой. При limit<3 вывод пустой.

```java
int limit = 7;
```

Проверь себя

- 7 -> 3,4,5,6,7.
- 3 -> одна строка 3; 2 -> пусто.

План / прогноз / причина ошибки

ГЛАВА 06 / EX047

Состояние накопителя

Уровень 1 · Предсказать результат · Ex047.java

Перед началом

Обновление переменных имеет порядок внутри итерации.

Твоя задача

Предскажи итог sum и i. Запиши таблицу состояния после каждого тела.

JAVA

```
int i = 1;
int sum = 0;
while (i < 5) {
    sum += i;
    i += 2;
}
System.out.println(sum);
System.out.println(i);
```

Проверь себя

- Таблица начинается с состояния до первой проверки.
- Пояснено, почему последний i может не войти в сумму.

План / прогноз / причина ошибки

ГЛАВА 06 / EX048

Сумма всё забывает

Уровень 1 · Найти ошибку · Ex048.java

Перед началом

Накопитель должен переживать все итерации.

Твоя задача

Найди ошибку: программа должна суммировать 1..n включительно. Исправь место инициализации.

JAVA

```
int n = 4;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
int sum = 0;
for (int i = 1; i <= n; i++) {
    sum = 0;
    sum += i;
}
System.out.println(sum);
```

Проверь себя

- 4 -> 10; 1 -> 1; 0 -> 0.
- Накопитель не сбрасывается внутри цикла.

План / прогноз / причина ошибки

ГЛАВА 06 / EX049

До сигнала остановки

Уровень 2 · Написать код · Ex049.java

Перед началом

Сигнал завершения не является измерением.

Твоя задача

Читай int-токены до первого -1. Все остальные числа неотрицательны. Выведи количество и long-сумму измерений. Сигнал гарантирован.

СТАНДАРТНЫЙ ВВОД

```
4 0 7 -1
```

Проверь себя

- 4 0 7 -1 -> количество 3, сумма 11.
- -1 -> 0,0; сигнал не прибавлен.

План / прогноз / причина ошибки

ГЛАВА 06 / EX050

Continue пропустил шаг

Уровень 2 · Объяснить поведение · Ex050.java

Перед началом

В while переход к условию может обойти обновление.

Твоя задача

Объясни зависание фрагмента при i=2. Исправление предложи в комментариях, фрагмент пока не запускай.

```java
int i = 0;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
while (i < 4) {
    if (i == 2) { continue; }
    System.out.println(i);
    i++;
}
```

Проверь себя

- Названа первая итерация без прогресса.
- Показано, где должен выполняться шаг на любом пути.

План / прогноз / причина ошибки

ГЛАВА 06 / EX051

Отсчёт в неверную сторону

Уровень 2 · Исправить программу · Ex051.java

Перед началом

При истинном условии шаг должен приближать его к ложному.

Твоя задача

Исправь обратный отсчёт: при start>0 выведи start..1, затем DONE; при start<=0 только DONE.

JAVA

```
int start = 4;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
while (start > 0) {
    System.out.println(start);
    start++;
}
System.out.println("DONE");
```

Проверь себя

- 4 -> 4,3,2,1,DONE.

- 0 и -2 -> только DONE; исходный цикл намеренно не запускается.

План / прогноз / причина ошибки

ГЛАВА 06 / EX052

Цифровой контроль

Уровень 3 · Написать код · Ex052.java

Перед началом

Остаток и целое деление последовательно разбирают число.

Твоя задача

Для n от 0 до 1000000000 вычисли сумму цифр и количество цифр без String. Ноль имеет одну цифру. Сохрани исходное n.

```java
int n = 70204;
```

Проверь себя

- 70204 -> сумма 13, количество 5.
- 0 -> 0 и 1; 1000 -> 1 и 4.

План / прогноз / причина ошибки

ГЛАВА 06 / EX053

Ряды кирпичей

Уровень 3 · Написать код · Ex053.java

Перед началом

Во вложенном цикле внутренняя граница может зависеть от внешней.

Твоя задача

Для rows от 0 до 20 выведи треугольник: строка k содержит k символов # без пробелов, k от 1 до rows. Повторение строк через repeat не используй.

```java
int rows = 4;
```

Проверь себя

- 4 -> строки #, ##, ###, ####.
- 0 -> пустой вывод; нет лишних пробелов.

План / прогноз / причина ошибки

ГЛАВА 06 / EX054

Первый делитель

Уровень 3 · Написать код · Ex054.java

Перед началом

Ранний выход позволяет завершить поиск после достаточного доказательства.

Твоя задача

Для n от 2 до 1000000 найди минимальный делитель >1. Если делитель равен n, выведи PRIME; иначе выведи этот делитель. Не используй коллекции.

```java
int n = 221;
```

Проверь себя

- 221 -> 13; 49 -> 7; 2 -> PRIME.
- 997 -> PRIME; перебор не обязан идти до n, если квадрат кандидата больше n.

План / прогноз / причина ошибки

ГЛАВА 07 / ТЕОРИЯ

07. Массивы и алгоритмы над последовательностями

Объект фиксированного размера

Массив хранит элементы одного типа. `new int[4]` создаёт объект с четырьмя int, изначально нулевыми. Ссылочный массив изначально содержит null в каждой ячейке. Длина доступна через поле `length` и не меняется. Последний допустимый индекс равен length - 1. Пустой массив существует и имеет length == 0, но не имеет ни одного допустимого индекса.

Индекс - целое int-совместимое значение. Отрицательный индекс не означает отсчёт с конца. Выход за границы вызывает ArrayIndexOutOfBoundsException во время выполнения, даже если код успешно скомпилирован. Не путай количество элементов с номером последнего.

Копирование ссылки и данных

Присваивание `int[] b = a` копирует ссылку: a и b позволяют менять один массив. Для независимого массива нужна копия, например `Arrays.copyOf(a, a.length)`. Для массива объектов такая копия переносит ссылки на элементы, а не создаёт новые объекты элементов. Это называется поверхностной копией.

JAVA

```
import java.util.Arrays;

public class Example07 {
    public static void main(String[] args) {
        int[] heights = {3, 7, 2};
        int[] alias = heights;
        int[] copy = Arrays.copyOf(heights, heights.length);
        alias[0] = 9;
        System.out.println(Arrays.toString(heights));
        System.out.println(Arrays.toString(copy));
        int sum = 0;
        for (int value : heights) {
            sum += value;
        }
        System.out.println(sum);
    }
}
```

ВЫВОД ПРИМЕРА

```
[9, 7, 2]
[3, 7, 2]
18
```

Меняется ячейка общего объекта, а не «переменная heights целиком». Ссылка copy ведёт к другому массиву. Вывод Arrays.toString показывает элементы; прямой println массива не является удобной печатью его содержимого.

Обход и изменение

Цикл с индексом нужен, когда важна позиция или нужно записать новое значение в ячейку. Enhanced for удобен для чтения всех элементов. Его переменная получает значение очередного элемента. Присваивание этой локальной переменной не заменяет элемент массива; для примитивов это лишь изменение копии значения. Для объектов через скопированную ссылку можно менять объект, но это тоже не заменяет ячейку.

Линейный поиск последовательно проверяет элементы. Если нужен первый подходящий индекс, остановка после находки сохраняет смысл «первый». Для отсутствия часто выбирают -1, поскольку он не является допустимым индексом. Договор о таком результате нужно явно написать.

Фильтрация и агрегаты

У результата фильтрации заранее неизвестная длина. Без коллекций можно сначала посчитать подходящие элементы, создать точный массив, затем заполнить его вторым проходом. Это два линейных прохода. Альтернатива - временный массив максимального размера и последующая копия нужной части.

Среднее пустого массива не определено. Максимум пустого массива тоже требует отдельного договора. Не возвращай случайный ноль без объяснения: он может совпасть с законным результатом непустого входа. Условия заданий устанавливают, что делать с пустотой.

Двумерный массив Java является массивом ссылок на массивы. Строки могут иметь разную длину или быть null. Для прямоугольной таблицы используй внешний индекс строки и внутренний индекс столбца; для рваной таблицы читай длину текущей строки, а не первой.

Ошибки и самопроверка

Если меняется «копия» и неожиданно оригинал, проверь, копировались ли данные. Если первый отрицательный максимум оказался нулём, проверь инициализацию. Если сортировка повредила исходный порядок, помни: Arrays.sort меняет переданный массив.

Перед практикой нарисуй переменные как стрелки к одному или двум прямоугольникам массивов. Покажи допустимые индексы пустого массива и массива из одного элемента. Объясни, почему два прохода могут быть проще и надёжнее одного сложного.

ГЛАВА 07 / EX055

Температурный сдвиг

Уровень 1 · Написать код · Ex055.java

Перед началом

Для записи в каждую ячейку нужен индекс либо другой явный способ замены.

Твоя задача

Увеличь каждое значение samples на offset и выведи Arrays.toString результата. Меняй именно исходный массив.

```java
int[] samples = {-3, 0, 8};
int offset = 2;
```

Проверь себя

- [-3,0,8],2 -> [-1,2,10].
- Пустой массив остаётся пустым.

План / прогноз / причина ошибки

ГЛАВА 07 / EX056

Две ссылки один массив

Уровень 1 · Предсказать результат · Ex056.java

Перед началом

Присваивание ссылки не копирует ячейки.

Твоя задача

Предскажи оба массива после изменения. Нарисуй ссылки a и b.

```java
int[] a = {5, 6};
int[] b = a;
b[1] = 9;
System.out.println(Arrays.toString(a));
System.out.println(Arrays.toString(b));
```

Проверь себя

- Прогноз сделан до запуска.
- Указано число созданных объектов массива.

План / прогноз / причина ошибки

ГЛАВА 07 / EX057

Последний индекс

Уровень 1 · Найти ошибку · Ex057.java

Перед началом

Допустимые индексы заканчиваются перед length.

Твоя задача

Найди ошибку и исправь обход, чтобы напечатать все элементы без исключения.

JAVA

```
int[] data = {2, 4, 6};
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
for (int i = 0; i <= data.length; i++) {
    System.out.println(data[i]);
}
```

Проверь себя

- Три строки 2,4,6.
- Пустой массив даёт пустой вывод.

План / прогноз / причина ошибки

ГЛАВА 07 / EX058

Первое превышение

Уровень 2 · Написать код · Ex058.java

Перед началом

Результат отсутствия должен отличаться от любого законного индекса.

Твоя задача

Найди индекс первого элемента > threshold. Выведи -1 при отсутствии. Исходный массив не меняй.

```java
int[] data = {4, 9, 9, 2};
int threshold = 8;
```

Проверь себя

- Исходный -> 1.
- threshold=10 -> -1; пустой -> -1; первый подходящий индекс сохраняется.

План / прогноз / причина ошибки

ГЛАВА 07 / EX059

Почему for-each не изменил массив

Уровень 2 · Объяснить поведение · Ex059.java

Перед началом

Переменная обхода получает копию значения элемента.

Твоя задача

Объясни итог и предложи в комментарии способ изменения ячеек. Исходный код оставь для наблюдения.

```java
int[] values = {1, 2, 3};
for (int value : values) { value *= 10; }
System.out.println(Arrays.toString(values));
```

Проверь себя

- Объяснение различает локальную value и ячейку.
- Предложен индексный обход без готового полного решения задачи.

План / прогноз / причина ошибки

ГЛАВА 07 / EX060

Максимум ниже нуля

Уровень 2 · Исправить программу · Ex060.java

Перед началом

Начальное значение должно соответствовать реальному элементу либо отдельному состоянию пустоты.

Твоя задача

Исправь поиск максимума. Для пустого массива выведи EMPTY; иначе максимальный элемент.

```java
int[] values = {-7, -2, -11};
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
int max = 0;
for (int value : values) {
    if (value > max) { max = value; }
}
System.out.println(max);
```

Проверь себя

- Исходный -> -2.
- [-9] -> -9; [] -> EMPTY; [0,-1] -> 0.

План / прогноз / причина ошибки

ГЛАВА 07 / EX061

Положительные в новый массив

Уровень 3 · Написать код · Ex061.java

Перед началом

Фильтрация не должна менять порядок или исходный набор.

Твоя задача

Создай новый массив только из положительных значений, сохраняя порядок. Коллекции не используй; длина результата точная.

```java
int[] values = {0, 4, -2, 7, 4};
```

Проверь себя

- Результат [4,7,4]; исходный не изменён.
- [] и [0,-1] -> []; повтор 4 сохранён.

План / прогноз / причина ошибки

ГЛАВА 07 / EX062

Соседние слияния

Уровень 3 · Написать код · Ex062.java

Перед началом

Серия определяется сравнением соседних элементов, а не глобальной уникальностью.

Твоя задача

Сожми соседние одинаковые int до одного представителя каждой серии. Верни результат новым массивом точной длины; коллекции не используй.

```java
int[] values = {2, 2, 3, 2, 2, 2, 5};
```

Проверь себя

- Результат [2,3,2,5].
- [] -> []; [7,7] -> [7]; [1,2,1] сохраняется.

План / прогноз / причина ошибки

ГЛАВА 07 / EX063

Рваная таблица

Уровень 3 · Написать код · Ex063.java

Перед началом

Длина каждой строки двумерного массива может отличаться.

Твоя задача

Вычисли long-сумму всех элементов rows. Пустые строки разрешены, null-строки трактуй как отсутствие данных. Выведи сумму и количество обработанных элементов.

```java
int[][] rows = {{2, 3}, {}, null, {8}};
```

Проверь себя

- Исходный -> сумма 13, количество 3.
- Пустая внешняя таблица -> 0,0; ни одна строка не читается через rows[0].

План / прогноз / причина ошибки

ГЛАВА 08 / ТЕОРИЯ

08. Строки: содержание, Unicode и неизменяемость

String - объект с неизменяемым содержимым

Строка представляет последовательность UTF-16-кодовых единиц. String неизменяем: методы вроде trim, replace и toUpperCase возвращают результат, не переписывая исходное содержимое. Присваивание `text = text.trim()` меняет ссылку text на результат. Если забыть присваивание, исходная переменная останется прежней.

Строка "" существует, имеет нулевую длину и не равна null. Null означает отсутствие объекта; вызов метода через null вызывает NullPointerException. Пробельная строка не пуста: length учитывает пробелы. Метод isBlank проверяет пустоту или пробельное содержимое по правилам API.

Равенство и индексы

Equals сравнивает содержимое String. Оператор == сравнивает ссылочные значения. JVM может переиспользовать строковые литералы; поэтому == иногда даёт ожидаемый ответ случайно. Не выводи правило языка из единственного удачного опыта с литералами.

`charAt(i)` возвращает кодовую единицу, `substring(begin, end)` берёт полуинтервал: begin включён, end исключён. Длина результата равна end - begin. Для пустого диапазона begin == end результат пустой. Индекс end может равняться length, но charAt(length) недопустим.

JAVA

```
import java.util.Locale;

public class Example08 {
    public static void main(String[] args) {
        String label = "  Dock-12  ";
        String clean = label.trim().toLowerCase(Locale.ROOT);
        System.out.println(clean);
        System.out.println(label.length());
        System.out.println(clean.substring(0, 4));
        String face = "\uD83D\uDE00";
        System.out.println(face.length());
        System.out.println(face.codePointCount(0, face.length()));
    }
}
```

ВЫВОД ПРИМЕРА

```
dock-12
11
dock
2
1
```

Что значит «символ»

В литерале \uD83D\uDE00 явно записана суррогатная пара одного emoji. Emoji в примере занимает две UTF-16-единицы, хотя кодовая точка одна. Некоторые видимые символы вообще состоят из нескольких кодовых точек, например буква и комбинируемое ударение. Поэтому length не гарантирует количество видимых символов. В большинстве начальных задач алфавит явно ограничен ASCII или русскими буквами без комбинируемых последовательностей. Если задача разрешает любой Unicode, договор другой.

Для обхода кодовых точек можно использовать codePointAt и Character.charCount, либо поток codePoints после знакомства с потоковыми API. Не переворачивай произвольный Unicode-текст простым обменом char, не понимая суррогатных пар. Сложная пользовательская сегментация текста выходит за рамки обычного char-цикла.

Поиск, разбиение и сборка

IndexOf возвращает -1 при отсутствии. Перед substring на найденной позиции нужно проверить это значение. StartsWith и endsWith выражают конкретный вопрос яснее ручного извлечения кусочков.

Split принимает регулярное выражение. Точка означает любой символ, а не буквальную точку; для буквальной точки нужен экранированный шаблон. В `split(";", -1)` отрицательный limit сохраняет пустые поля в конце. Это важно для файлового формата: `name;` содержит второе пустое поле, а не одно поле.

При многократном добавлении текста в цикле удобен StringBuilder: изменяемый накопитель с append, после которого вызывают toString. Это отделяет процесс сборки от неизменяемого итогового значения. Не используй один общий builder для независимых сообщений без очистки.

Ошибки и самопроверка

Не применяй trim к полю, если пробелы по договору значимы. Не используй системную локаль для идентификаторов: правила регистра могут зависеть от языка. Locale.ROOT полезна для машинных ключей, но не решает все задачи естественного языка.

Перед практикой объясни разницу null, пустой строки и строки из пробелов. Назови пределы charAt. Покажи на примере, почему `text.replace(...)` без присваивания не меняет text. Уточняй в условии, что именно считается символом.

ГЛАВА 08 / EX064

Код без краёв

Уровень 1 · Написать код · Ex064.java

Перед началом

Trim возвращает результат, не меняя исходную строку.

Твоя задача

Удали только краевые пробелы trim, переведи латинский код в нижний регистр Locale.ROOT и выведи. Исходный code сохрани.

```java
String code = "  Dock-A  ";
```

Проверь себя

- Результат dock-a.
- " " -> пустая строка; внутренние пробелы сохраняются.

План / прогноз / причина ошибки

ГЛАВА 08 / EX065

Литерал и новый объект

Уровень 1 · Предсказать результат · Ex065.java

Перед началом

Равенство ссылок и содержимого отвечают на разные вопросы.

Твоя задача

Предскажи три boolean-результата и объясни каждый.

JAVA

```
String a = "map";
String b = new String("map");
String c = "map";
System.out.println(a == b);
System.out.println(a.equals(b));
System.out.println(a == c);
```

Проверь себя

- Прогноз до запуска.
- Упомянуто переиспользование литералов; == не рекомендован для содержимого.

План / прогноз / причина ошибки

ГЛАВА 08 / EX066

Последняя буква за границей

Уровень 1 · Найти ошибку · Ex066.java

Перед началом

Length - число кодовых единиц, а не допустимый последний индекс.

Твоя задача

Исправь получение последнего char. Для пустой строки выведи EMPTY. Здесь разрешены только ASCII-символы.

```java
String text = "harbor";
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
System.out.println(text.charAt(text.length()));
```

Проверь себя

- harbor -> r; X -> X.
- Пустая -> EMPTY; нет обращения по неверному индексу.

План / прогноз / причина ошибки

ГЛАВА 08 / EX067

Разбор метки

Уровень 2 · Написать код · Ex067.java

Перед началом

IndexOf позволяет сначала проверить наличие разделителя.

Твоя задача

Раздели text по первой двоеточии на key и value. Пустые части разрешены. Если двоеточия нет, выведи INVALID. Пробелы не удаляй.

```java
String text = "zone:B:7";
```

Проверь себя

- zone:B:7 -> key=zone, value=B:7.
- ":" -> обе части пусты; "zone" -> INVALID.

План / прогноз / причина ошибки

ГЛАВА 08 / EX068

Сколько здесь символов

Уровень 2 · Объяснить поведение · Ex068.java

Перед началом

UTF-16, кодовые точки и видимые символы не всегда совпадают.

Твоя задача

Выведи length и codePointCount для text и объясни различие в комментарии. Не называй codePointCount универсальным числом видимых символов.

```java
String text = "A\uD83D\uDE00Б";
```

Проверь себя

- Для исходной строки сравни обе метрики.
- Объяснение упоминает суррогатную пару и границы применимости.

План / прогноз / причина ошибки

ГЛАВА 08 / EX069

Замена без присваивания

Уровень 2 · Исправить программу · Ex069.java

Перед началом

Неизменяемый String требует сохранить возвращённое значение.

Твоя задача

Исправь замену всех дефисов на подчёркивания, чтобы printed text содержал новый результат.

```java
String text = "a-b-c";
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
text.replace("-", "_");
System.out.println(text);
```

Проверь себя

- a-b-c -> a_b_c.
- Пустая строка и строка без дефиса не ломаются.

План / прогноз / причина ошибки

ГЛАВА 08 / EX070

Проверка машинного имени

Уровень 3 · Написать код · Ex070.java

Перед началом

Ограниченный алфавит делает договор проверки точным.

Твоя задача

Проверь name: длина 3..12, первый символ латинская буква, остальные латинские буквы, цифры или _. Регистр разрешён любой. Выведи VALID/INVALID. Regex не используй.

```java
String name = "Rover_7";
```

Проверь себя

- Rover_7 VALID; 7Rover INVALID.
- ab INVALID; abc- INVALID; русские буквы INVALID; пустая строка обработана.

План / прогноз / причина ошибки

ГЛАВА 08 / EX071

Слова без пустых полей

Уровень 3 · Написать код · Ex071.java

Перед началом

Разделители и пустые части требуют явного алгоритма.

Твоя задача

Посчитай слова, разделённые одним или несколькими обычными пробелами U+0020. Краевые пробелы разрешены. Не используй split или regex; один проход.

```java
String text = "  красный   маяк у моря  ";
```

Проверь себя

- Исходный -> 4.
- "" и " " -> 0; "маяк" -> 1; прочие символы не считаются разделителями.

План / прогноз / причина ошибки

ГЛАВА 08 / EX072

Экранный отчёт

Уровень 3 · Написать код · Ex072.java

Перед началом

StringBuilder позволяет управлять разделителем без лишнего хвоста.

Твоя задача

Собери из items одну строку, разделяя элементы через " | ". Пустые элементы сохраняй; после последнего разделителя нет. Не используй String.join.

```java
String[] items = {"карта", "", "компас"};
```

Проверь себя

- Результат содержит «карта», затем пробел, |, два пробела, |, пробел, «компас».
- [] -> пустая строка; ["x"] -> x; исходный массив не меняется.

План / прогноз / причина ошибки

ГЛАВА 09 / ТЕОРИЯ

09. Методы и передача значений

Метод задаёт отдельный контракт

Метод объединяет действие под именем. Его объявление задаёт модификаторы, возвращаемый тип, имя, параметры и тело. Параметры - локальные переменные вызова; аргументы - значения, переданные при вызове. Метод с результатом обязан вернуть значение подходящего типа на каждом обычном пути выполнения. Void-метод результата не возвращает; печать на экран не заменяет return.

Полезный контракт отвечает на вопросы: какие входы разрешены, что возвращается, что изменяется снаружи, какие ошибки возможны. Например, метод площади не должен молча печатать случайное предупреждение и возвращать ноль, если вызывающий код ожидает результат или исключение.

Java всегда передаёт по значению

При вызове копируется значение аргумента. Для int это число. Для массива или объекта это ссылочное значение, а не сам объект. Поэтому метод может через копию ссылки изменить общий объект, но не может переназначить переменную вызывающего кода. Это не «передача по ссылке» в смысле изменения чужой переменной.

JAVA

```
public class Example09 {
    static int perimeter(int width, int height) {
        return 2 * (width + height);
    }

    static void repaint(int[] cells) {
        cells[0] = 8;
        cells = new int[] {1, 1};
    }

    public static void main(String[] args) {
        int[] board = {2, 3};
        repaint(board);
        System.out.println(board[0]);
        int border = perimeter(4, 6);
        System.out.println(border);
    }
}
```

ВЫВОД ПРИМЕРА

```
8
20
```

Первое присваивание в repaint меняет общий массив. Второе меняет только локальную ссылку cells. После возврата board по-прежнему указывает на первоначальный массив. Нарисуй два ссылочных значения в момент входа в метод, затем отдельный новый массив после переназначения.

Декомпозиция вместо длинного main

Main удобно оставить координатором: прочитать данные, вызвать вычисление, показать результат. Чистый вычислительный метод возвращает результат без чтения терминала и записи файла. Его можно проверить несколькими вызовами и использовать в другом интерфейсе.

Не дроби код по случайному числу строк. Выделяй действия с понятным именем и устойчивым договором. Если метод и читает пользователя, и считает, и сохраняет, тестировать отдельное вычисление трудно. Если параметров слишком много, возможно, не хватает объекта предметной области.

Static, перегрузка и область видимости

Static-метод принадлежит классу. Он не имеет this и не может напрямую читать состояние конкретного объекта. Пока работа идёт с данными аргументов, static-метод естественен. Позже методы объектов будут использовать поля собственного экземпляра.

Перегрузка - несколько методов с одним именем и разными списками типов параметров. Возвращаемого типа недостаточно для перегрузки. Выбор перегрузки происходит при компиляции по доступным типам аргументов; это отличается от переопределения в наследовании.

Локальные переменные разных вызовов независимы. Рекурсия создаёт новый вызов метода, который должен приближаться к базовому случаю. Для длинных последовательностей обычный цикл часто проще и не расходует стек на каждый шаг. В курсе рекурсия факультативна.

Ошибки и проверка понимания

Если метод только печатает, его результат нельзя использовать как число. Если часть веток не возвращает значение, сначала уточни контракт для этих случаев. Если метод неожиданно меняет массив, подпиши побочный эффект или верни копию. Не маскируй недописанный метод правдоподобным нулём: в заготовках методы могут явно бросать UnsupportedOperationException, пока не реализованы.

Перед практикой опиши один метод в трёх предложениях: вход, результат, изменение состояния. Объясни, какие значения копируются при передаче массива. Приведи пример, когда печать результата должна оставаться вне вычислительного метода.

ГЛАВА 09 / EX073

Площадь по договору

Уровень 1 · Написать код · Ex073.java

Перед началом

Return передаёт результат вызывающему коду без обязательной печати.

Твоя задача

Реализуй static long area(int width,int height), входы 0..100000. Main вызывает метод для данных и печатает результат; внутри метода вывода нет.

JAVA

```
int width = 70000;
int height = 60000;
```

JAVA

```
static long area(int width, int height) {
    // TODO: реализуй договор.
    throw new UnsupportedOperationException("TODO");
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- 70000,60000 -> 4200000000.
- 0,5 -> 0; промежуточное произведение не int.

План / прогноз / причина ошибки

ГЛАВА 09 / EX074

Копия числа и общая ячейка

Уровень 1 · Предсказать результат · Ex074.java

Перед началом

Аргументы передаются по значению; ссылочное значение тоже копируется.

Твоя задача

Предскажи две строки. Покажи, что меняет метод, а что остаётся у вызывающего кода.

JAVA

```
int n = 4;
int[] box = {4};
change(n, box);
System.out.println(n);
System.out.println(box[0]);
```

JAVA

```
static void change(int n, int[] box) {
    n = 9;
    box[0] = 9;
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Сначала прогноз и рисунок ссылок.
- Различены переназначение параметра и изменение ячейки.

План / прогноз / причина ошибки

ГЛАВА 09 / EX075

Void вместо результата

Уровень 1 · Найти ошибку · Ex075.java

Перед началом

Вывод не подставляется в арифметическое выражение как результат метода.

Твоя задача

Найди ошибку договора метода. Исправь метод так, чтобы main мог прибавить 1 к результату и вывести 13 для входов 5,7. Внутри метода ничего не печатай.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static void add(int a, int b) {
    System.out.println(a + b);
}
// В main:
int result = add(5, 7);
System.out.println(result + 1);
```

Проверь себя

- add(5,7) -> 12; внешний итог 13.
- add(-3,3) -> 0.

План / прогноз / причина ошибки

ГЛАВА 09 / EX076

Индекс без побочных эффектов

Уровень 2 · Написать код · Ex076.java

Перед началом

Метод поиска возвращает отсутствие по заранее заданному правилу.

Твоя задача

Реализуй static int firstIndex(int[] values,int target): первый индекс либо -1. Массив ненулевой, может быть пуст; его не меняй.

JAVA

```
int[] values = {6, 2, 6};
int target = 6;
```

JAVA

```
static int firstIndex(int[] values, int target) {
    throw new UnsupportedOperationException("TODO");
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Исходный -> 0; target=9 -> -1.
- [] -> -1; повтор возвращает первый индекс.

План / прогноз / причина ошибки

ГЛАВА 09 / EX077

Переназначение массива

Уровень 2 · Объяснить поведение · Ex077.java

Перед началом

Локальная ссылка не является переменной вызывающего кода.

Твоя задача

Объясни, почему reset не заменяет data. Запиши в комментарии два разных договора: вернуть новый массив и обнулить существующий.

JAVA

```
int[] data = {8, 9};
reset(data);
System.out.println(Arrays.toString(data));
```

JAVA

```
static void reset(int[] values) {
    values = new int[] {0, 0};
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Различены новый объект и старый массив.
- Оба предложенных договора явно описывают побочный эффект.

План / прогноз / причина ошибки

ГЛАВА 09 / EX078

Неполный return

Уровень 2 · Исправить программу · Ex078.java

Перед началом

Все обычные пути метода с результатом должны его возвращать.

Твоя задача

Исправь sign: для положительного 1, нуля 0, отрицательного -1. Используй int и полный договор, не печатай внутри.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static int sign(int value) {
    if (value > 0) { return 1; }
    if (value < 0) { return -1; }
}
```

Проверь себя

- 5 -> 1; 0 -> 0; -7 -> -1.

- Integer.MIN_VALUE -> -1; не используй value/Math.abs(value).

План / прогноз / причина ошибки

ГЛАВА 09 / EX079

Новая очищенная последовательность

Уровень 3 · Написать код · Ex079.java

Перед началом

Чистый метод можно повторно вызывать без изменения входа.

Твоя задача

Реализуй static int[] withoutZero(int[] values). Возвращается новый массив без нулей, порядок сохранён; коллекции не используй. Проверь, что даже без нулей возвращён другой объект массива.

JAVA
```
int[] values = {0, 5, 0, -2};
```

JAVA
```
static int[] withoutZero(int[] values) {
    throw new UnsupportedOperationException("TODO");
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Результат [5,-2], исходный [0,5,0,-2].
- [] -> новый []; [4] -> [4] с другой ссылкой.

План / прогноз / причина ошибки

ГЛАВА 09 / EX080

Парсер и вычислитель

Уровень 3 · Написать код · Ex080.java

Перед началом

Разделение чтения и вычисления позволяет проверить каждый этап отдельно.

Твоя задача

Напиши static int parseMinutes(String text) для корректной целой строки с краевыми пробелами и static long seconds(int minutes) для неотрицательного int. Main применяет оба к text, печатает секунды. Обработку ошибочных строк пока не делай.

JAVA

```
String text = " 125 ";
```

JAVA

```
static int parseMinutes(String text) {
    throw new UnsupportedOperationException("TODO");
}
static long seconds(int minutes) {
    throw new UnsupportedOperationException("TODO");
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- 125 -> 7500; 0 -> 0.
- 2147483647 -> 128849018820; методы ничего не читают из консоли.

План / прогноз / причина ошибки

ГЛАВА 09 / EX081

Сводка без глобальных переменных

Уровень 3 · Написать код · Ex081.java

Перед началом

Функциональная декомпозиция связывает небольшие проверяемые договоры.

Твоя задача

Напиши countValid и sumValid для int[]: допустимы значения 1..50. В main выведи число допустимых, число отброшенных и среднее с 2 знаками или EMPTY. Методы static, без вывода и изменения массива; сумма long.

JAVA

```
int[] samples = {0, 10, 60, 20, 30};
```

JAVA

```
static int countValid(int[] samples) {
    throw new UnsupportedOperationException("TODO");
}
static long sumValid(int[] samples) {
    throw new UnsupportedOperationException("TODO");
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Допустимых 3, отброшено 2, среднее 20.00.
- [] -> 0,0,EMPTY; [1,50] -> 2,0,25.50; не дели на полный размер.

План / прогноз / причина ошибки

ГЛАВА 10 / ТЕОРИЯ

10. Класс, объект и состояние экземпляра

От набора переменных к модели

Класс описывает допустимую структуру объектов: их поля и методы. Объект - конкретный экземпляр с собственным состоянием. Два объекта одного класса могут иметь разные значения полей. Если приложению нужны десять посылок, не создавай переменные weight1, weight2 и так далее: создай десять объектов одной модели.

Поля живут вместе с объектом и доступны его методам по правилам доступа. Локальные переменные принадлежат выполнению метода. Поля получают значения по умолчанию: 0 для целых, false для boolean, null для ссылок. Это не доказывает, что объект с таким состоянием допустим по смыслу; конструкторы исправят эту проблему в следующей главе.

Identity и общая ссылка

Выражение new создаёт объект и возвращает ссылочное значение. Несколько переменных могут указывать на один объект. Изменение через одну ссылку видно через другую. Две независимо созданные вещи с одинаковыми полями не обязаны быть равны по ==: это разные объекты.

JAVA

```
public class Example10 {
    static class Beacon {
        int level;

        void raise() {
            level++;
        }
    }

    public static void main(String[] args) {
        Beacon north = new Beacon();
        Beacon alias = north;
        Beacon south = new Beacon();
        alias.raise();
        System.out.println(north.level);
        System.out.println(south.level);
        System.out.println(north == alias);
        System.out.println(north == south);
    }
}
```

ВЫВОД ПРИМЕРА

```
1
0
true
false
```

Вложенный static-класс в примере позволяет сохранить всё в одном файле. Он не означает, что level общее для всех объектов. Ключевое слово static у вложенного класса лишь убирает необходимость иметь экземпляр внешнего Example10.

Поля и методы static

Поле экземпляра имеется у каждого объекта отдельно. Static-поле принадлежит классу и разделяется всеми экземплярами этого класса в обычной загрузке приложения. Счётчик всех созданных объектов может быть static; имя конкретной посылки - нет. Если сделать всё static, независимые объекты начнут мешать друг другу.

Метод экземпляра вызывается относительно объекта и имеет this - ссылку на текущий экземпляр. При совпадении имени параметра и поля `this.name = name` различает их. Присваивание `name = name` меняет параметр на него самого и оставляет поле прежним.

Поведение и ответственность

Объект полезен, когда он объединяет данные с поведением и правилами их изменения. Метод `consume(amount)` может контролировать остаток. Внешний код, который напрямую меняет поле, способен обойти это правило. Поэтому открытые поля в примере - промежуточная учебная модель, а не рекомендация для готового приложения.

Не каждый существующий предмет требует класса. Модель выбирается под задачу: для маршрута могут быть нужны координаты остановки, но не её цвет. Сначала выпиши операции, затем данные, необходимые для их поддержки. Избыточная иерархия классов усложняет понимание без пользы.

Жизненный цикл и память

Локальная ссылка может исчезнуть после выхода из метода, но объект остаётся доступным, если до него ведёт другая живая ссылка. Объект, недостижимый для работающей программы, может быть освобождён сборщиком мусора. Момент этого освобождения не следует использовать для закрытия файлов или записи важных данных.

Null не создаёт пустой экземпляр. Перед вызовом метода нужно либо гарантировать существование объекта контрактом, либо обработать отсутствие. Лучше уточнить, допускается ли отсутствие, чем расставлять проверки null повсюду без модели.

Ошибки и самопроверка

Частые причины: перепутанные static и поля экземпляра; ожидание автоматической глубокой копии; вызов метода через null; чтение начального 0 как законного состояния; забытый this. Для диагностики нарисуй объекты отдельно от переменных.

Перед практикой объясни, сколько объектов создают два new и одно присваивание ссылки. Покажи, какие поля принадлежат каждому объекту. Расскажи, почему сборщик мусора не заменяет дисциплину работы с внешними ресурсами.

ГЛАВА 10 / EX082

Карточка оборудования

Уровень 1 · Написать код · Ex082.java

Перед началом

Экземпляры одного класса имеют отдельное состояние.

Твоя задача

Определи вложенный static-класс Device с полями String label и int watts. Создай два объекта из данных, выведи каждый как «label: watts». Конструкторы пока не требуются.

JAVA

```
String firstLabel = "радар";
int firstWatts = 120;
String secondLabel = "зонд";
int secondWatts = 45;
```

Проверь себя

- Два независимых объекта, строки радар: 120 и зонд: 45.
- Изменение watts первого не меняет второго.

План / прогноз / причина ошибки

ГЛАВА 10 / EX083

Объект общий

Уровень 1 · Предсказать результат · Ex083.java

Перед началом

Alias обозначает ещё один путь к уже созданному объекту.

Твоя задача

Предскажи значения a.value, b.value, c.value и результаты ==.

JAVA

```
Cell a = new Cell();
a.value = 3;
Cell b = a;
Cell c = new Cell();
c.value = 3;
b.value = 8;
System.out.println(a.value + "," + b.value + "," + c.value);
System.out.println(a == b);
System.out.println(a == c);
```

JAVA

```
static class Cell { int value; }
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Сначала прогноз.
- Нарисованы два объекта и три ссылки.

План / прогноз / причина ошибки

ГЛАВА 10 / EX084

Static захватил общее состояние

Уровень 1 · Найти ошибку · Ex084.java

Перед началом

Static-поле принадлежит классу, а не каждой записи.

Твоя задача

Найди ошибку модели: у каждого Robot должен быть свой charge. Исправь её и продемонстрируй независимость двух роботов.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Robot { static int charge; }
// В main:
Robot a = new Robot();
Robot b = new Robot();
a.charge = 10;
b.charge = 20;
System.out.println(a.charge);
```

Проверь себя

- После установки первому 10, второму 20 первый остаётся 10.
- Объяснено, почему static здесь не подходит.

План / прогноз / причина ошибки

ГЛАВА 10 / EX085

Метод меняет свой экземпляр

Уровень 2 · Написать код · Ex085.java

Перед началом

This связывает вызов с конкретным объектом.

Твоя задача

Создай Meter с полем int steps и методом void advance(int amount), прибавляющим неотрицательный amount. Создай два счётчика и вызови advance для первого дважды, для второго один раз.

JAVA

```
int firstMove = 3;
int secondMove = 4;
int otherMove = 2;
```

Проверь себя

- Первый steps=7, второй=2.
- Метод не static; отрицательные аргументы пока исключены договором.

План / прогноз / причина ошибки

ГЛАВА 10 / EX086

Null не является пустым объектом

Уровень 2 · Объяснить поведение · Ex086.java

Перед началом

Ссылка без объекта не поддерживает доступ к его полям.

Твоя задача

Объясни, почему фрагмент падает, и предложи два разных смысла отсутствия устройства: обязательное создание или допустимое отсутствие с отдельной веткой.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
Device device = null;
System.out.println(device.label);
```

JAVA

```
static class Device { String label; }
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Назван NullPointerException.
- Не предлагается ловить исключение вместо уточнения договора.

План / прогноз / причина ошибки

ГЛАВА 10 / EX087

Поле осталось нулём

Уровень 2 · Исправить программу · Ex087.java

Перед началом

Совпадение имени параметра и поля требует различить получателя.

Твоя задача

Исправь метод setLevel, чтобы менялось поле текущего объекта. Создай Gauge и проверь установку 7.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Gauge {
    int level;
    void setLevel(int level) { level = level; }
}
```

Проверь себя

- После setLevel(7) поле level равно 7.
- Указана причина самоприсваивания параметра.

План / прогноз / причина ошибки

ГЛАВА 10 / EX088

Массив объектов без смешения

Уровень 3 · Написать код · Ex088.java

Перед началом

Ячейки ссылочного массива изначально не содержат созданных экземпляров.

Твоя задача

Определи Parcel с полями id и grams. Создай массив из трёх разных Parcel по данным и выведи long-сумму masses. Покажи в комментарии, почему одного new Parcel[3] недостаточно.

```java
String[] ids = {"P1", "P2", "P3"};
int[] masses = {1200, 800, 300};
```

Проверь себя

- Сумма 2300.
- Все три ячейки ненулевые и указывают на разные объекты.

План / прогноз / причина ошибки

ГЛАВА 10 / EX089

Снимок состояния

Уровень 3 · Написать код · Ex089.java

Перед началом

Копия полей создаёт отдельный объект, но ссылочные поля требуют отдельного решения.

Твоя задача

Для Point с int x,y напиши static Point copy(Point source). Создай original(2,5) через присваивание полей, копию, затем измени копии x на 9. Выведи обе точки и ==.

```java
int x = 2;
int y = 5;
```

Проверь себя

- Оригинал (2,5), копия (9,5), == false.
- Метод не возвращает source; null исключён договором.

План / прогноз / причина ошибки

ГЛАВА 10 / EX090

Журнал использования

Уровень 3 · Написать код · Ex090.java

Перед началом

Поведение объектов можно агрегировать без общего изменяемого static-поля.

Твоя задача

Создай Session с user и minutes и методом boolean isLong(), true при minutes>=60. По данным создай объекты, выведи long-сумму минут и число длинных сеансов. Поля экземплярные.

```java
String[] users = {"Лея", "Рин", "Лея"};
int[] minutes = {45, 60, 90};
```

Проверь себя

- Сумма 195, длинных 2.
- Повтор user не объединяет разные сеансы; пустой набор -> 0,0.

План / прогноз / причина ошибки

ГЛАВА 11 / ТЕОРИЯ

11. Конструкторы, инкапсуляция и инварианты

Объект должен начинать жизнь корректным

Конструктор выполняется при создании экземпляра. Его имя совпадает с именем класса; возвращаемый тип не пишется, даже void. Если конструкторы не объявлены, компилятор предоставляет конструктор без аргументов. Если объявлен хотя бы один, автоматический вариант без аргументов больше не добавляется.

Конструктор нужен, чтобы задать обязательные данные и проверить правила. Инвариант объекта - утверждение, которое должно быть истинно после завершения конструктора и каждого публичного изменения. Для измерителя объёма это может быть `0 <= amount && amount <= capacity`. Если объём недопустим, создание объекта должно завершиться отказом, а не скрытой подменой данных.

Private и единая точка проверки

Private-поле недоступно произвольному внешнему коду. Публичные методы дают контролируемые операции. Это инкапсуляция: устройство может меняться, пока договор методов остаётся прежним. Геттеры и сеттеры не являются обязательной парой для каждого поля. Метод `reserve` лучше выражает смысл, чем безусловный `setReserved`.

JAVA

```
public class Example11 {
    static final class Charge {
        private final int capacity;
        private int amount;

        Charge(int capacity) {
            if (capacity <= 0) {
                throw new IllegalArgumentException("capacity must be positive");
            }
            this.capacity = capacity;
        }

        void add(int units) {
            if (units < 0 || units > capacity - amount) {
                throw new IllegalArgumentException("invalid charge");
            }
            amount += units;
        }

        int amount() {
            return amount;
        }
    }

    public static void main(String[] args) {
        Charge charge = new Charge(10);
        charge.add(4);
        System.out.println(charge.amount());
    }
}
```

ВЫВОД ПРИМЕРА

```
4
```

Смысл throw будет подробно разобран в главе 14; здесь достаточно понимать, что неверная операция прерывается до изменения поля. Проверка через `capacity - amount` избегает переполнения суммы при допустимом инварианте. Все проверки выполняются до изменения состояния.

Перегрузка и делегирование конструкторов

Можно предложить несколько способов создания с разными параметрами. Вызов `this(...)` делегирует другому конструктору того же класса и в Java 17 должен быть первой инструкцией. Так одна основная проверка обслуживает все варианты создания. Дублирование проверки в нескольких местах со временем приводит к несовпадающим правилам.

This обозначает текущий экземпляр, а не родительский класс. Конструктор не является обычным методом и не вызывается как `object.ClassName()`. Присваивание полям после new не заменяет конструктор, если объект между этими действиями уже может попасть в чужой код.

Неизменяемые модели и defensive copy

Неизменяемый объект после создания не меняет наблюдаемое состояние. Обычно этому помогают final-класс, private final-поля, проверка входов и отсутствие методов изменения. Но final-ссылка на массив запрещает только переназначение ссылки, а содержимое массива остаётся изменяемым.

Если сохранить переданный массив напрямую, вызывающий код сможет поменять внутреннее состояние после проверки. Если вернуть внутренний массив из геттера, доступ откроется снова. Защитная копия на входе и выходе разрывает этот канал. Для массива изменяемых объектов одной поверхностной копии может быть недостаточно; нужно продумать владение элементами.

Доступ и пакеты

Public доступен извне, private ограничен классом и связанными вложенными контекстами, отсутствие модификатора даёт доступ в пакете. Protected связан с пакетом и наследованием; его тонкости рассмотрим в следующей главе. Пакет не является защитой от недоверенного кода, но помогает организовать внутренний API приложения.

Ошибки и самопроверка

Если сеттер принимает всё, private само по себе не защищает инвариант. Если ошибка возникает после изменения половины полей, объект может остаться противоречивым. Проверяй все условия до записи либо вычисляй новое состояние отдельно.

Перед практикой сформулируй инвариант будущего класса. Перечисли все пути изменения. Объясни, почему final-массив не равен неизменяемому массиву и почему скрывать поле полезнее, чем просто переименовать его.

ГЛАВА 11 / EX091

Обязательное имя

Уровень 1 · Написать код · Ex091.java

Перед началом

Конструктор устанавливает обязательные данные до выдачи объекта клиенту.

Твоя задача

Создай Badge с private final String name, конструктором Badge(String name) и методом name(). Запрещены null и пустое после trim имя; сохрани trim-результат. Для отказа IllegalArgumentException.

```java
String rawName = "  Лея  ";
```

Проверь себя

- name() -> Лея.
- null, "", " " отвергнуты; после создания имя не меняется.

План / прогноз / причина ошибки

ГЛАВА 11 / EX092

Конструктор делегирует

Уровень 1 · Предсказать результат · Ex092.java

Перед началом

This(...) вызывает другой конструктор до остального тела.

Твоя задача

Предскажи порядок строк при new Panel(). Объясни, какой конструктор выполнил какую строку.

JAVA

```
new Panel();
```

JAVA

```
static class Panel {
    Panel() { this(3); System.out.println("empty"); }
    Panel(int n) { System.out.println("size=" + n); }
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз записан до запуска.
- Пояснено, почему this(...) стоит первым.

План / прогноз / причина ошибки

ГЛАВА 11 / EX093

Массив вышел наружу

Уровень 1 · Найти ошибку · Ex093.java

Перед началом

Private-ссылка не защищает объект, если отдать её напрямую.

Твоя задача

Найди нарушение инкапсуляции, связанное с конструктором и геттером. Исправь оба пути утечки массива.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Inventory {
    private final int[] slots;
    Inventory(int[] slots) { this.slots = slots; }
    int[] getSlots() { return slots; }
}
```

Проверь себя

- Изменение входного массива после создания не влияет на Inventory.
- Изменение полученного getSlots массива не влияет на Inventory.

План / прогноз / причина ошибки

ГЛАВА 11 / EX094

Контролируемый резерв

Уровень 2 · Написать код · Ex094.java

Перед началом

Публичная операция должна сохранять инвариант после успеха и отказа.

Твоя задача

Создай Reserve(capacity) с private capacity и used. Capacity>0. Метод reserve(n) принимает n>=0 и только если хватает места; иначе IllegalArgumentException без изменения. Методы used() и free().

```java
int capacity = 8;
int request = 3;
```

Проверь себя

- После reserve(3): used=3, free=5.
- reserve(6) затем отвергнут и состояние 3,5; reserve(0) допустим.

План / прогноз / причина ошибки

ГЛАВА 11 / EX095

Final не замораживает объект

Уровень 2 · Объяснить поведение · Ex095.java

Перед началом

Final запрещает присваивание ссылке, но не метод изменения объекта.

Твоя задача

Объясни, почему код законен и что он показывает. Запиши, какое присваивание самой переменной было бы запрещено.

```java
final int[] labels = {1, 2};
labels[0] = 9;
System.out.println(labels[0]);
```

Проверь себя

- Различены ссылка и содержимое массива.
- Запрещённое присваивание записано только в комментарии.

План / прогноз / причина ошибки

ГЛАВА 11 / EX096

Проверка после записи

Уровень 2 · Исправить программу · Ex096.java

Перед началом

Отказ не должен оставлять сломанный инвариант.

Твоя задача

Исправь setAmount: допустимо 0..100; при отказе прежнее amount сохраняется. Создай объект, установи 40, попытайся 120 и проверь оставшиеся 40.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Tank {
    private int amount;
    void setAmount(int value) {
        amount = value;
        if (value < 0 || value > 100) {
            throw new IllegalArgumentException("amount");
        }
    }
}
```

Проверь себя

- Успех меняет состояние.
- Отказ бросает IllegalArgumentException до изменения поля.

План / прогноз / причина ошибки

ГЛАВА 11 / EX097

Неизменяемый диапазон

Уровень 3 · Написать код · Ex097.java

Перед началом

Конструктор проверяет связь полей, а методы раскрывают только договор.

Твоя задача

Создай final-класс Range с private final int start,end; требуй start<=end. Методы contains(int) и long lengthInclusive(). Границы включены.

```java
int start = -3;
int end = 4;
```

Проверь себя

- Длина 8; contains(-3),contains(4) true; contains(5) false.
- MIN_VALUE..MAX_VALUE -> 4294967296; обратные границы отвергнуты.

План / прогноз / причина ошибки

ГЛАВА 11 / EX098

Перевод между резервуарами

Уровень 3 · Написать код · Ex098.java

Перед началом

Связанная операция должна проверить оба объекта до любых изменений.

Твоя задача

Создай Tank с private capacity,amount, constructor(capacity,amount), amount(). Напиши transferTo(Tank other,int units): units>=0, хватает источника и места назначения; при отказе IllegalArgumentException, оба состояния прежние. Перевод самому себе - отсутствие изменения при units<=amount.

```java
int sourceCapacity = 10;
int sourceAmount = 7;
int targetCapacity = 8;
int targetAmount = 2;
```

Проверь себя

- Перевод 3 -> источник 4, цель 5.
- Перевод 9 отвергнут без изменений; нулевой допустим; constructor требует 0<=amount<=capacity.

План / прогноз / причина ошибки

ГЛАВА 11 / EX099

Каталог с защитной копией

Уровень 3 · Написать код · Ex099.java

Перед началом

Неизменяемость контейнера требует защитить вход и выход.

Твоя задача

Создай final Codes(String[] codes), сохраняющий копию. Null-массив и null-элементы запрещены; дубликаты разрешены. Метод values() возвращает новую копию. Проверь независимость трёх массивов: входа, внутреннего и результата.

```java
String[] inputCodes = {"AX", "BX"};
```

Проверь себя

- После inputCodes[0]="XX" values()[0] остаётся AX.
- Изменение массива, полученного values(), не влияет на следующую values(); [] допустим.

План / прогноз / причина ошибки

ГЛАВА 12 / ТЕОРИЯ

12. Наследование и полиморфизм

Когда один тип является разновидностью другого

Наследование extends описывает отношение типов. Объект подкласса может использоваться там, где ожидается базовый класс. Это обещание поведения, а не только способ скопировать поля и методы. Если подкласс нарушает договор базового типа, программа с общей ссылкой становится непредсказуемой для клиента.

Класс Java имеет один непосредственный базовый класс. Если extends не написан, обычный класс наследует Object. Конструкторы не наследуются. В Java 17 конструктор подкласса сначала вызывает this или super; если ничего не написано, вставляется super() при его доступности. Наличие только конструктора родителя с аргументом требует явного вызова.

Статический тип и реальный объект

Статический тип ссылки задаёт доступные операции при компиляции. Реальный класс объекта определяет, какое переопределение метода экземпляра будет выполнено. Именно это позволяет одному циклу работать с разными разновидностями объектов без цепочек instanceof.

JAVA

```
public class Example12 {
    static abstract class Route {
        abstract int minutes();
    }

    static class Walk extends Route {
        @Override
        int minutes() {
            return 18;
        }
    }

    static class Tram extends Route {
        @Override
        int minutes() {
            return 7;
        }
    }

    public static void main(String[] args) {
        Route[] routes = {new Walk(), new Tram()};
        for (Route route : routes) {
            System.out.println(route.minutes());
        }
    }
}
```

ВЫВОД ПРИМЕРА

```
18
7
```

В цикле переменная имеет тип Route, но не создаёт отдельный объект Route. Она получает ссылку на Walk, затем Tram. Abstract-класс нельзя создать напрямую; abstract-метод задаёт обязательную операцию без реализации в базовом классе.

Override не равен overload

Переопределение сохраняет совместимую сигнатуру метода экземпляра. Аннотация Override просит компилятор проверить намерение и ловит опечатки. Нельзя сузить видимость переопределяемого метода. Возвращаемый ссылочный тип может быть уточнён совместимым подтипом. Ограничения checked-исключений важны после главы 14.

Перегрузка выбирает между разными списками параметров при компиляции. Static-методы скрываются, а не переопределяются с динамическим выбором как методы экземпляра. Поля тоже не участвуют в таком полиморфном выборе. Поэтому не используй одинаковые имена полей родителя и потомка как замену переопределению поведения.

Super и состояние родителя

Super позволяет обратиться к реализации базового класса или его конструктору. Private-поле родителя нельзя напрямую читать из подкласса; для работы с состоянием предназначены методы базового класса. Protected не означает «открыто всем потомкам в любой ситуации»: за пределами пакета действуют дополнительные правила доступа через наследование. Сначала предпочитай private и небольшой ясный API.

Не вызывай переопределяемый метод из конструктора родителя без крайней необходимости. Он может попасть в код подкласса раньше, чем подкласс завершил инициализацию собственных полей. Объект ещё строится, а динамический выбор уже работает.

Композиция и безопасные границы

Наследование удобно, когда связь «является» устойчива и договор действительно общий. Для отношения «использует» лучше поле другого объекта: это композиция. Сервис доставки использует принтер, но не является принтером. Изменение стратегии тогда не требует перестройки родословной классов.

Приведение базовой ссылки к подклассу не превращает объект. Оно лишь утверждает, что объект уже подходящего типа. Неверное утверждение вызывает ClassCastException. Если приходится постоянно проверять конкретные подклассы, возможно, общему договору недостаёт операции.

Ошибки и самопроверка

Перед практикой укажи статический и фактический тип каждого объекта примера. Объясни, кто выбирает перегрузку и кто выбирает переопределение. Придумай нарушение договора базового типа: например, подкласс внезапно запрещает вход, разрешённый родителем. Поведение клиента должно оставаться осмысленным при подстановке любой законной разновидности.

ГЛАВА 12 / EX100

Два способа измерения

Уровень 1 · Написать код · Ex100.java

Перед началом

Общий базовый договор позволяет клиенту вызывать одну операцию.

Твоя задача

Создай abstract Sensor с abstract int read(); ConstantSensor(value) возвращает value. Создай два Sensor со значениями 11 и 25, выведи read каждого через Sensor-ссылку.

```java
int firstValue = 11;
int secondValue = 25;
```

Проверь себя

- Две строки 11 и 25.
- Override стоит на реализации; Sensor напрямую не создаётся.

План / прогноз / причина ошибки

ГЛАВА 12 / EX101

Выбор метода по объекту

Уровень 1 · Предсказать результат · Ex101.java

Перед началом

Статический тип и фактический класс участвуют в разных проверках.

Твоя задача

Предскажи результат и объясни выбор метода.

JAVA

```
Base value = new Child();
System.out.println(value.name());
```

JAVA

```
static class Base { String name() { return "base"; } }
static class Child extends Base {
    @Override String name() { return "child"; }
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Указаны статический тип ссылки и фактический тип объекта.
- Объяснение не сводит вызов к типу переменной.

План / прогноз / причина ошибки

ГЛАВА 12 / EX102

Опечатка вместо override

Уровень 1 · Найти ошибку · Ex102.java

Перед началом

Override проверяет совпадение договора с базовым методом.

Твоя задача

Найди причину ошибки аннотации и исправь класс так, чтобы через Speaker-ссылку получалось "loud". Не удаляй Override ради компиляции.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Speaker { String speak() { return "quiet"; } }
static class Loud extends Speaker {
    @Override String speek() { return "loud"; }
}
```

Проверь себя

- Вызов через Speaker работает полиморфно.
- Имя и параметры совпадают.

План / прогноз / причина ошибки

ГЛАВА 12 / EX103

Плата маршрута

Уровень 2 · Написать код · Ex103.java

Перед началом

Каждый подкласс реализует одну семантическую операцию с общей единицей измерения.

Твоя задача

Создай abstract Route с long costCents(int passengers). Bus: 250 центов за пассажира; Ferry: 400 плюс 150 за пассажира. Passengers>=0. Сложи стоимости двух Route для passengers=3.

```java
int passengers = 3;
```

Проверь себя

- Bus -> 750; Ferry -> 850; сумма 1600.
- 0 -> Bus 0, Ferry 400; большое int не переполняет произведение.

План / прогноз / причина ошибки

ГЛАВА 12 / EX104

Поля не полиморфны

Уровень 2 · Объяснить поведение · Ex104.java

Перед началом

Динамический выбор относится к переопределённым методам экземпляра.

Твоя задача

Предскажи два вывода и объясни, почему поле и метод могут показывать разные значения.

JAVA

```
Parent value = new Kid();
System.out.println(value.label);
System.out.println(value.label());
```

JAVA

```
static class Parent {
    String label = "parent";
    String label() { return label; }
}
static class Kid extends Parent {
    String label = "kid";
    @Override String label() { return label; }
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз до запуска.
- Различены скрытие поля и переопределение метода.

План / прогноз / причина ошибки

ГЛАВА 12 / EX105

Родителю нужен аргумент

Уровень 2 · Исправить программу · Ex105.java

Перед началом

Конструкторы не наследуются, super() может отсутствовать.

Твоя задача

Исправь конструктор Truck, чтобы он передавал plate родителю. Сохрани обязательность plate в Vehicle.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Vehicle {
    final String plate;
    Vehicle(String plate) { this.plate = plate; }
}
static class Truck extends Vehicle {
    Truck(String plate) { }
}
```

Проверь себя

- new Truck("T7") сохраняет T7 в родителе.
- Не добавлен фиктивный пустой конструктор Vehicle.

План / прогноз / причина ошибки

ГЛАВА 12 / EX106

Прямоугольники по общему договору

Уровень 3 · Написать код · Ex106.java

Перед началом

Базовый договор должен быть одинаково осмыслен для всех разновидностей.

Твоя задача

Создай abstract Figure с long area(); Rectangle(width,height) и Square(side), размеры 0..100000. Обойди Figure[] и выведи сумму площадей без instanceof.

```java
int width = 7;
int height = 4;
int side = 5;
```

Проверь себя

- Rectangle 28, Square 25, сумма 53.
- Размер 100000 не переполняет int; методы не сужают договор.

План / прогноз / причина ошибки

ГЛАВА 12 / EX107

Запрет опасного приведения

Уровень 3 · Написать код · Ex107.java

Перед началом

Приведение не создаёт новый тип объекта.

Твоя задача

Создай BaseDevice и два подкласса Lamp, Pump с общим методом description(). Обойди массив с обоими и выведи описания без приведений. В комментарии объясни, почему приведение Pump к Lamp через BaseDevice опасно.

Проверь себя

- В массиве есть обе разновидности и используются методы базового договора.
- Опасное приведение не выполняется; назван ClassCastException.

План / прогноз / причина ошибки

ГЛАВА 12 / EX108

Цена наследования

Уровень 3 · Написать код · Ex108.java

Перед началом

Клиент должен сохранять смысл при подстановке любой реализации.

Твоя задача

Спроектируй abstract Shipping с long quote(int grams), grams>=0. FixedShipping всегда 600; WeightedShipping 100 + 2*grams. Обойди оба для 350 и для 0. В комментарии объясни, почему подкласс, запрещающий grams=0, нарушил бы договор.

```java
int grams = 350;
```

Проверь себя

- 350 -> 600 и 800; 0 -> 600 и 100.
- Клиент без instanceof; long-арифметика.

План / прогноз / причина ошибки

ГЛАВА 13 / ТЕОРИЯ

13. Интерфейсы и композиция

Договор без общей родословной

Интерфейс описывает набор операций, которые обещает тип. Класс объявляет implements и реализует эти операции. Разные классы могут поддерживать один интерфейс, не наследуя общую реализацию. Класс может реализовать несколько интерфейсов, хотя базовый класс у него один.

Обычный абстрактный метод интерфейса публичен. Реализация должна быть public, даже если рядом все классы находятся в одном пакете. Поля интерфейса являются public static final-константами; это не поля будущего экземпляра. Для состояния реализации используй класс.

Клиент зависит от нужной возможности

Если форматтеру нужен способ отправить строку, ему не обязательно знать, отправляется ли она в консоль или накапливается в памяти. Передача зависимости через конструктор позволяет выбрать реализацию извне. Это делает устройство приложения видимым и даёт возможность проверять клиент без реального внешнего действия.

JAVA

```
public class Example13 {
    interface Output {
        void write(String message);
    }

    static class ConsoleOutput implements Output {
        @Override
        public void write(String message) {
            System.out.println(message);
        }
    }

    static class Greeting {
        private final Output output;

        Greeting(Output output) {
            this.output = output;
        }

        void welcome(String place) {
            output.write("Добро пожаловать: " + place);
        }
    }

    public static void main(String[] args) {
        Greeting greeting = new Greeting(new ConsoleOutput());
        greeting.welcome("мастерская");
    }
}
```

ВЫВОД ПРИМЕРА

```
Добро пожаловать: мастерская
```

Greeting содержит Output: это композиция. В примере зависимость считается ненулевой по контракту. В готовой публичной модели стоит проверить это в конструкторе. Для теста вместо ConsoleOutput можно передать накопитель сообщений; вычисление приветствия останется прежним.

Выбор размеров интерфейса

Интерфейс должен описывать цель клиента, а не все возможности огромного сервиса. Если клиент только читает, не заставляй его зависеть от удаления и записи. Небольшой договор проще реализовать и проверить. Но отдельный интерфейс для каждого метода без задачи тоже создаёт лишние имена.

Default-метод интерфейса может содержать реализацию, обычно через другие операции договора. Static-метод вызывают через имя интерфейса. Если два интерфейса дают несовместимые default-реализации, класс должен явно разрешить конфликт. Не используй default как скрытое место для состояния: экземплярных полей интерфейса нет.

Стратегия как заменяемое поведение

Стратегия - объект, который реализует один способ вычисления. Например, разные правила маршрутизации поддерживают одну операцию выбора стоимости. Клиент хранит ссылку интерфейсного типа и вызывает её. Выбор новой стратегии не меняет код клиента.

Это особенно полезно на границе внешнего мира: часы, хранилище, генератор случайных чисел, уведомления. Для проверки можно передать предсказуемую реализацию. Не нужно подключать сложный фреймворк, чтобы освоить принцип: обычного конструктора достаточно.

Интерфейс, абстрактный класс и окончательная модель

Интерфейс подходит для роли и возможного множества реализаций. Абстрактный класс полезен для действительно общего состояния и базовой реализации родственников. Final-класс запрещает наследование, когда расширение не предусмотрено договором. Эти средства отвечают на разные вопросы; ни одно не является универсально «более объектным».

Ошибки и самопроверка

Если реализация компилируется только после удаления Override, возможно, метод вовсе не реализует договор: проверь имя, параметры и public. Если клиент создаёт конкретную зависимость внутри каждого вызова, подменить её в проверке трудно. Если интерфейс содержит слишком много несвязанных операций, опиши несколько разных клиентов и их реальные потребности.

Перед практикой нарисуй два объекта и связь «содержит». Объясни, почему ConsoleOutput можно заменить, не меняя Greeting. Назови различие между переиспользованием реализации через наследование и переиспользованием поведения через поле.

ГЛАВА 13 / EX109

Один способ вывода

Уровень 1 · Написать код · Ex109.java

Перед началом

Реализация метода интерфейса должна быть public.

Твоя задача

Создай интерфейс Reporter с void report(String). ConsoleReporter печатает строку. В main храни реализацию в Reporter и передай message.

```java
String message = "Канал открыт";
```

Проверь себя

- Ровно строка Канал открыт.
- Метод реализации public, есть Override.

План / прогноз / причина ошибки

ГЛАВА 13 / EX110

Клиент хранит роль

Уровень 1 · Предсказать результат · Ex110.java

Перед началом

Поле интерфейсного типа ссылается на конкретный объект реализации.

Твоя задача

Предскажи вывод. Объясни, какие методы доступны через Action.

JAVA

```
Action action = new Bell();
action.run();
```

JAVA

```
interface Action { void run(); }
static class Bell implements Action {
    @Override public void run() { System.out.println("ring"); }
}
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз до запуска.
- Отмечено, что интерфейс сам по себе не создаёт объект.

План / прогноз / причина ошибки

ГЛАВА 13 / EX111

Доступ стал слабее

Уровень 1 · Найти ошибку · Ex111.java

Перед началом

Интерфейсный договор публичен по умолчанию.

Твоя задача

Найди причину ошибки и исправь реализацию без изменения Reader.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
interface Reader { String read(); }
static class FixedReader implements Reader {
    String read() { return "fixed"; }
}
```

Проверь себя

- Код после исправления компилируется.
- Объяснено, почему доступ по пакету недостаточен.

План / прогноз / причина ошибки

ГЛАВА 13 / EX112

Стратегия скидки

Уровень 2 · Написать код · Ex112.java

Перед началом

Разные политики можно подставлять без ветвления в клиенте.

Твоя задача

Создай Discount с long apply(long cents), cents>=0. NoDiscount возвращает исходную сумму; MinusHundred возвращает max(0,cents-100). Service получает Discount в конструкторе и считает итог.

```java
long priceCents = 350;
```

Проверь себя

- Для двух сервисов 350 и 250.
- При 50: 50 и 0; клиент не проверяет класс стратегии.

План / прогноз / причина ошибки

ГЛАВА 13 / EX113

Композиция вместо родства

Уровень 2 · Объяснить поведение · Ex113.java

Перед началом

Использование объекта не делает клиента его подтипом.

Твоя задача

Объясни, почему DeliveryService должен содержать Printer, а не наследоваться от Printer. Запиши схему классов и способ передать поддельный принтер для проверки в комментариях.

Проверь себя

- Названы обязанности клиента и зависимости.
- Объяснено, как проверить текст без настоящего принтера.

План / прогноз / причина ошибки

ГЛАВА 13 / EX114

Создание внутри лишило подмены

Уровень 2 · Исправить программу · Ex114.java

Перед началом

Явная зависимость через конструктор делает выбор реализации внешним решением.

Твоя задача

Перепиши фрагмент: Notifier должен получать Channel в конструкторе. Реализуй MemoryChannel, сохраняющий последнее сообщение в String и число вызовов в int. При notify("ok") последнее сообщение ok, вызов один.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
interface Channel { void send(String text); }
static class ConsoleChannel implements Channel {
    public void send(String text) { System.out.println(text); }
}
static class Notifier {
    void notify(String text) { new ConsoleChannel().send(text); }
}
```

Проверь себя

- Проверка не печатает в консоль из канала.
- Notifier не создаёт ConsoleChannel внутри метода.

План / прогноз / причина ошибки

ГЛАВА 13 / EX115

Детерминированные номера

Уровень 3 · Написать код · Ex115.java

Перед началом

Подмена источника делает сценарий воспроизводимым.

Твоя задача

Создай IdSource с int next(); SequenceSource(start) выдаёт start,start+1,... (входы малы). Registrar получает источник и создаёт строку "ID-номер". Проверь два вызова с началом 40.

```java
int start = 40;
```

Проверь себя

- ID-40, ID-41.
- Новый источник с 40 воспроизводит результат; глобального счётчика нет.

План / прогноз / причина ошибки

ГЛАВА 13 / EX116

Несколько правил проверки

Уровень 3 · Написать код · Ex116.java

Перед началом

Небольшие интерфейсы позволяют объединить независимые проверки.

Твоя задача

Создай Rule с boolean accepts(String). MinLength(3) и NoSpace проверяют ненулевую строку. Validator получает Rule[] и разрешает строку, только если все правила разрешают; пустой набор правил разрешает любую ненулевую строку.

```java
String candidate = "map7";
```

Проверь себя

- map7 true; "ab" false; "map 7" false.
- Пустой набор правил -> true; правила не изменяют строку.

План / прогноз / причина ошибки

ГЛАВА 13 / EX117

Хранилище за интерфейсом

Уровень 3 · Написать код · Ex117.java

Перед началом

Клиенту важен договор поиска, а не конкретная структура хранения.

Твоя задача

Создай Catalog с put(String id,String title) и String find(String id), отсутствие -> null. InMemoryCatalog хранит до 10 записей в двух String[] и счётчике размера; переполнение -> IllegalStateException. SearchService получает Catalog и возвращает title либо "NOT_FOUND". Id/title ненулевые.

```java
String id = "B7";
String title = "Навигация";
```

Проверь себя

- После put B7 -> Навигация; неизвестный -> NOT_FOUND.
- Повторный put заменяет название; два хранилища независимы; повтор при заполненном массиве разрешён.

План / прогноз / причина ошибки

ГЛАВА 14 / ТЕОРИЯ

14. Исключения и работа с отказами

Обычный результат и невозможность продолжить

Исключение сообщает, что операция не завершилась обычным результатом. Throw создаёт переход управления к подходящему обработчику. Выполнение оставшихся инструкций текущего блока прекращается; если обработчика здесь нет, поиск поднимается по цепочке вызовов. Stack trace показывает тип, сообщение и места вызовов, но причина не всегда находится в последней строке приложения.

Исключение не должно заменять обычное ветвление для ожидаемого выбора. Отсутствие элемента может быть нормальным результатом поиска. Повреждённый файл обязательных данных может означать отказ загрузки. Сначала реши, что является договором операции.

Checked и unchecked

RuntimeException и её потомки относятся к unchecked: компилятор не требует обязательного catch или throws. Это удобно для нарушений аргументов и инвариантов: IllegalArgumentException, IllegalStateException. Checked-исключения, например IOException, нужно обработать или объявить в throws. Throws не обрабатывает ошибку; оно передаёт обязанность вызывающему коду.

Error обычно относится к серьёзным проблемам среды. Начинающее приложение не должно ловить Throwable ради продолжения любой ценой. Обработчик выбирается по совместимости типа; более конкретные catch должны стоять перед общими.

JAVA

```
public class Example14 {
    static int readCount(String text) {
        int value = Integer.parseInt(text);
        if (value < 0) {
            throw new IllegalArgumentException("negative count");
        }
        return value;
    }

    public static void main(String[] args) {
        String text = "many";
        try {
            System.out.println(readCount(text));
        } catch (IllegalArgumentException error) {
            System.out.println("Некорректное количество");
        }
        System.out.println("Программа продолжилась");
    }
}
```

ВЫВОД ПРИМЕРА

```
Некорректное количество
Программа продолжилась
```

NumberFormatException является подклассом IllegalArgumentException, поэтому выбранный обработчик ловит ошибку преобразования. Он также ловит отрицательное число из проверки. Если для этих случаев нужны разные сообщения, обрабатывай их осознанно отдельно.

Где обрабатывать

Нижний вычислительный метод обычно не знает, как пользователь хочет увидеть отказ. Он может бросить исключение с контекстом. Верхний слой интерфейса решает: повторить ввод, показать сообщение, прекратить команду. Не печатай и не бросай ту же ошибку на каждом слое: получатся дубли и неудобный контракт.

Пустой catch скрывает отказ. Возврат 0 из catch может выдать ошибку за законное значение. Если исключение преобразуется в другое, сохраняй исходную причину конструктором с cause. Для пользовательского сообщения не обязательно показывать весь внутренний stack trace, но для диагностики его нужно сохранить.

Finally и ресурсы

Finally обычно выполняется при выходе из try/catch, в том числе при return и исключении. Не помещай туда return: он способен подавить предыдущий результат или ошибку. Аварийное завершение процесса и некоторые системные ситуации не дают гарантии выполнения finally.

Try-with-resources закрывает AutoCloseable-ресурсы даже при исключении. Ресурсы закрываются в обратном порядке объявления. Если одновременно ошибка возникла в теле и при закрытии, дополнительные ошибки могут быть сохранены как suppressed. Это надёжнее ручного close после чтения: до него выполнение может не дойти.

Безопасная практика

В заданиях опасные фрагменты помещены в комментарии с пометкой «НАМЕРЕННАЯ ОШИБКА». Перенеси их в рабочую область только при изучении соответствующего отказа. Логические ошибки, не приводящие к зависанию и разрушению данных, могут выполняться. Заготовка, которая успешно стартовала, ещё не является решённой задачей.

Перед практикой прочитай stack trace небольшого опыта: найди тип исключения, сообщение и первую строку своего кода. Объясни различие throw, throws и catch. Укажи слой, который должен решить, повторять ли ввод после отказа.

ГЛАВА 14 / EX118

Безопасное преобразование

Уровень 1 · Написать код · Ex118.java

Перед началом

Пользовательский слой может превратить ожидаемый отказ ввода в сообщение.

Твоя задача

Прочитай одну строку и преобразуй trim в int. При NumberFormatException выведи INVALID, иначе число. Не лови все Exception.

СТАНДАРТНЫЙ ВВОД

```
42
```

Проверь себя

- " 42 " -> 42; "forty" и "" -> INVALID.
- Число вне диапазона int -> INVALID.

План / прогноз / причина ошибки

ГЛАВА 14 / EX119

Путь после отказа

Уровень 1 · Предсказать результат · Ex119.java

Перед началом

После throw выполнение тела try не продолжается с места ошибки.

Твоя задача

Предскажи порядок строк.

JAVA

```
try {
    System.out.println("A");
    Integer.parseInt("x");
    System.out.println("B");
} catch (NumberFormatException error) {
    System.out.println("C");
} finally {
    System.out.println("D");
}
System.out.println("E");
```

Проверь себя

- Прогноз до запуска.
- Указано, какая строка недостижима на этом входе.

План / прогноз / причина ошибки

ГЛАВА 14 / EX120

Общий catch поглотил конкретный

Уровень 1 · Найти ошибку · Ex120.java

Перед началом

Обработчики проверяются сверху вниз по типу.

Твоя задача

Найди причину ошибки компиляции и переставь обработчики, сохранив оба.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
try { Integer.parseInt("x"); }
catch (IllegalArgumentException e) { System.out.println("argument"); }
catch (NumberFormatException e) { System.out.println("number"); }
```

Проверь себя

- Конкретный обработчик достижим.
- Объяснена связь NumberFormatException и IllegalArgumentException.

План / прогноз / причина ошибки

ГЛАВА 14 / EX121

Аргумент с предметным правилом

Уровень 2 · Написать код · Ex121.java

Перед началом

Парсер и проверка диапазона могут иметь разные причины отказа.

Твоя задача

Напиши parsePort(String): целое после trim, допустимо 1..65535; иначе IllegalArgumentException. Main проверяет 8080, 0 и "x", печатает значение либо INVALID.

```java
String[] inputs = {"8080", "0", "x"};
```

Проверь себя

- 8080 -> 8080; 0 и x -> INVALID.
- 1 и 65535 допустимы; 65536 отвергнут; null запрещён контрактом.

План / прогноз / причина ошибки

ГЛАВА 14 / EX122

Throws не ловит ошибку

Уровень 2 · Объяснить поведение · Ex122.java

Перед началом

Объявление возможности отказа не даёт способа восстановиться.

Твоя задача

Объясни различие IOException в throws метода и catch(IOException) у клиента. Опиши, где консольное приложение должно предложить повторить путь, а где просто передать отказ выше.

Проверь себя

- Названы checked-обязанность компилятора и решение интерфейса.
- Не предлагается делать пустой catch или возвращать случайный результат.

План / прогноз / причина ошибки

ГЛАВА 14 / EX123

Отказ превратился в ноль

Уровень 2 · Исправить программу · Ex123.java

Перед началом

Законный результат и ошибка не должны быть неразличимы без договора.

Твоя задача

Исправь divide: при b==0 бросай IllegalArgumentException с понятным сообщением. Иначе верни a/b (целое деление). Удали catch, скрывающий ошибку; main отдельно демонстрирует отказ.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static int divide(int a, int b) {
    try { return a / b; }
    catch (ArithmeticException e) { return 0; }
}
```

Проверь себя

- 6/3 -> 2; 0/3 -> 0.

- 6/0 -> ожидаемый IllegalArgumentException, а не значение 0.

План / прогноз / причина ошибки

ГЛАВА 14 / EX124

Номер строки в отказе

Уровень 3 · Написать код · Ex124.java

Перед началом

Полезное исключение сохраняет причину и контекст.

Твоя задача

Создай ParseFailure extends Exception с конструктором (String message, Throwable cause). Метод parse(String text,int line) преобразует int, а NumberFormatException оборачивает в ParseFailure с номером строки. Main обрабатывает ParseFailure.

```java
String text = "oops";
int line = 7;
```

Проверь себя

- Сообщение содержит 7, getCause() имеет тип NumberFormatException.
- "12",7 -> 12; исходный cause не потерян.

План / прогноз / причина ошибки

ГЛАВА 14 / EX125

Ресурс закрылся при ошибке

Уровень 3 · Написать код · Ex125.java

Перед началом

Try-with-resources закрывает AutoCloseable при обычном и аварийном выходе.

Твоя задача

Создай TraceResource implements AutoCloseable, close печатает CLOSED. В try-with-resources выведи BODY и намеренно брось IllegalStateException; catch печатает CAUGHT. Сначала запиши ожидаемый порядок в комментарии.

Проверь себя

- Порядок соответствует договору закрытия.
- Close вызывается ровно один раз; после catch программа завершается штатно.

План / прогноз / причина ошибки

ГЛАВА 14 / EX126

Повтор до корректного ввода

Уровень 3 · Написать код · Ex126.java

Перед началом

Повторение относится к пользовательскому сценарию, а не к вычислительному методу.

Твоя задача

Читай строки, пока не получишь целое 1..9. На неверную строку печатай RETRY, при успехе OK:n и заверши. Конец ввода до успеха -> END. Лови только ошибку преобразования; диапазон проверяй отдельно.

СТАНДАРТНЫЙ ВВОД

```
x
0
4
```

Проверь себя

- x,0,4 -> RETRY,RETRY,OK:4.
- Пустой ввод -> END; 10 затем EOF -> RETRY,END.

План / прогноз / причина ошибки

ГЛАВА 15 / ТЕОРИЯ

15. List и Set: порядок, дубликаты и изменение

Почему массива бывает недостаточно

Размер массива фиксирован. Когда число элементов меняется, коллекция берёт управление хранением на себя. List описывает последовательность с индексами и дубликатами. Set описывает множество без дубликатов по договору равенства элементов. Выбирай структуру по смыслу, а не по привычке использовать один тип всегда.

`List<String>` означает список строк. Параметр типа позволяет компилятору проверять добавление и чтение; полный механизм generics будет разобран в главе 17. Здесь достаточно знать: это не имя отдельного класса ListString. Для int в коллекциях используется Integer; boxing и unboxing связывают примитив и объектную обёртку.

Интерфейс и реализация

`List<String> names = new ArrayList<>()` разделяет роль и реализацию. Клиенту нужен договор List, хранение обеспечивает ArrayList. ArrayList удобно читать по индексу и добавлять в конец; вставка в начале сдвигает элементы. Не считай любую операцию одинаково дешёвой.

HashSet не обещает порядок обхода. LinkedHashSet сохраняет порядок вставки, TreeSet поддерживает сортированный порядок по сравнению. Случайный устойчивый порядок HashSet в одном запуске не является договором API.

JAVA

```
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class Example15 {
    public static void main(String[] args) {
        List<String> stops = new ArrayList<>();
        stops.add("порт");
        stops.add("парк");
        stops.add("порт");
        Set<String> unique = new LinkedHashSet<>(stops);
        System.out.println(stops.size());
        System.out.println(unique);
        List<Integer> levels = new ArrayList<>(List.of(4, 8, 4));
        levels.remove(Integer.valueOf(4));
        System.out.println(levels);
    }
}
```

ВЫВОД ПРИМЕРА

```
3
[порт, парк]
[8, 4]
```

Последний remove удаляет первое равное значение 4. Вызов remove(1) выбрал бы перегрузку с индексом. Это особенно важно для List<Integer>: выбор перегрузки зависит от типа аргумента.

Изменяемость не определяется одной ссылкой

List.of создаёт неизменяемую по структуре коллекцию и не допускает null. Попытка add вызывает UnsupportedOperationException. `new ArrayList<>(List.of(...))` создаёт изменяемую копию структуры. Объекты внутри копии могут остаться общими.

Arrays.asList возвращает список фиксированного размера, связанный с исходным массивом объектов: set возможен, add/remove размера - нет. Не называй все эти варианты просто «обычным списком». List.copyOf помогает получить неизменяемую структуру, но не глубокую копию изменяемых элементов.

Unboxing null в int вызывает NullPointerException. Обёртки могут быть null, примитив int - нет. Сравнивай значение Integer через equals или осознанное распаковывание при гарантированном ненулевом значении; == может зависеть от переиспользования объектов.

Удаление во время обхода

Удаление из ArrayList через список внутри enhanced for может нарушить итератор и вызвать ConcurrentModificationException. Это возможно даже в одном потоке: название не означает обязательную многопоточность. Используй Iterator.remove, removeIf после изучения predicate или отдельный список результата. Не полагайся на исключение как гарантированный детектор всех неправильных изменений.

Для поиска повторов множество удобно: результат add сообщает, появился ли новый элемент. Для сохранения всех вхождений нужен список, а не множество. Для подсчёта частот потребуется Map следующей главы.

Ошибки и самопроверка

Проверь пустую коллекцию, один элемент и повтор. Уточни, должен ли сохраняться порядок. Перед практикой объясни различие размера и максимального индекса. Покажи, почему final-ссылка не запрещает add в ArrayList, а List.of запрещает его без final. Выбери структуру для журнала событий и для набора уникальных разрешений.

ГЛАВА 15 / EX127

Очередь названий

Уровень 1 · Написать код · Ex127.java

Перед началом

ArrayList допускает изменение размера и сохраняет последовательность.

Твоя задача

Скопируй initial в изменяемый List<String>, добавь last, выведи размер и каждый элемент отдельной строкой. Initial не меняй.

```java
List<String> initial = List.of("порт", "парк");
String last = "склад";
```

Проверь себя

- Размер 3, порядок порт,парк,склад.
- Пустой initial -> один элемент; исходный размер остаётся 2.

План / прогноз / причина ошибки

ГЛАВА 15 / EX128

Remove выбирает перегрузку

Уровень 1 · Предсказать результат · Ex128.java

Перед началом

Int-аргумент может обозначать индекс, а Integer - значение.

Твоя задача

Предскажи списки после каждой операции, объясни различие перегрузок.

JAVA

```
List<Integer> values = new ArrayList<>(List.of(5, 1, 5, 9));
values.remove(1);
System.out.println(values);
values.remove(Integer.valueOf(5));
System.out.println(values);
```

Проверь себя

- Прогноз до запуска.
- Указано, удаляется ли первое или любое вхождение значения.

План / прогноз / причина ошибки

ГЛАВА 15 / EX129

Неизменяемый источник

Уровень 1 · Найти ошибку · Ex129.java

Перед началом

List.of не поддерживает добавление элементов.

Твоя задача

Найди причину отказа и исправь создание values, сохранив исходные два элемента и добавление третьего.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
List<Integer> values = List.of(2, 4);
values.add(6);
System.out.println(values);
```

Проверь себя

- Список содержит [2,4,6].
- List.of не модифицируется напрямую.

План / прогноз / причина ошибки

ГЛАВА 15 / EX130

Первые уникальные

Уровень 2 · Написать код · Ex130.java

Перед началом

LinkedHashSet сохраняет порядок первого добавления.

Твоя задача

Получи уникальные названия из values в порядке первого появления и выведи каждое отдельной строкой.

```java
List<String> values = List.of("медь", "железо", "медь", "уголь");
```

Проверь себя

- медь,железо,уголь.
- Пустой -> пусто; регистр значим; HashSet без гарантии порядка не подходит.

План / прогноз / причина ошибки

ГЛАВА 15 / EX131

Порядок множества

Уровень 2 · Объяснить поведение · Ex131.java

Перед началом

Наблюдаемый порядок одной реализации не становится обещанием всех множеств.

Твоя задача

Объясни, почему тест, ожидающий фиксированный текст HashSet.toString, хрупок. Предложи два корректных способа проверки: без порядка и с заданным порядком.

Проверь себя

- Упомянуты contains/размер либо равенство множеств.
- Для заданного порядка выбрана подходящая структура или сортировка.

План / прогноз / причина ошибки

ГЛАВА 15 / EX132

Удаление ломает обход

Уровень 2 · Исправить программу · Ex132.java

Перед началом

Структурное изменение через список может нарушить итератор.

Твоя задача

Исправь удаление отрицательных из values с сохранением остальных в прежнем порядке. Используй Iterator.remove либо отдельный результат, без lambda.

```java
List<Integer> values = new ArrayList<>(List.of(3, -1, -2, 4));
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
for (Integer value : values) {
    if (value < 0) { values.remove(value); }
}
```

Проверь себя

- Результат [3,4].

- [-1,-2] -> []; [] -> []; соседние отрицательные не пропущены.

План / прогноз / причина ошибки

ГЛАВА 15 / EX133

Стабильное пересечение

Уровень 3 · Написать код · Ex133.java

Перед началом

Множество ускоряет проверку принадлежности, список определяет порядок.

Твоя задача

Верни уникальные строки, встречающиеся и в first, и в second, в порядке первого появления в first. Исходники не меняй.

```java
List<String> first = List.of("B", "A", "B", "C");
List<String> second = List.of("C", "B", "B");
```

Проверь себя

- Результат [B,C].
- Пустой любой -> []; повторы не дублируют результат.

План / прогноз / причина ошибки

ГЛАВА 15 / EX134

Последние команды

Уровень 3 · Написать код · Ex134.java

Перед началом

Ограниченный журнал явно удаляет самые старые записи.

Твоя задача

Собери List<String> последних capacity команд из commands, сохраняя порядок. Capacity>=0; можно обрабатывать последовательно, удаляя первый элемент при переполнении.

```java
int capacity = 3;
List<String> commands = List.of("a", "b", "c", "d", "e");
```

Проверь себя

- Результат [c,d,e].
- Capacity=0 -> []; capacity=10 -> все пять; повторы сохраняются.

План / прогноз / причина ошибки

ГЛАВА 15 / EX135

Разделить поток повторов

Уровень 3 · Написать код · Ex135.java

Перед началом

Результат add множества позволяет отличать первое появление от повтора.

Твоя задача

Из values сформируй два списка: firstSeen содержит первые появления, repeated содержит все последующие вхождения в исходном порядке. Строки не null.

```java
List<String> values = List.of("x", "y", "x", "x", "z", "y");
```

Проверь себя

- firstSeen=[x,y,z], repeated=[x,x,y].
- Пустой -> два пустых; repeated не является множеством.

План / прогноз / причина ошибки

ГЛАВА 16 / ТЕОРИЯ

16. Map, равенство и хеширование

Ключ ведёт к одному значению

Map хранит пары ключ-значение. Это не список пар с дубликатами ключей: один ключ связан не более чем с одним значением. Put для уже существующего ключа заменяет значение и возвращает прежнее. Значения разных ключей могут совпадать.

HashMap не обещает порядок обхода. LinkedHashMap может сохранять порядок вставки, TreeMap - порядок ключей по сравнению. Если в условии требуется детерминированный отчёт, выбери подходящую структуру или отсортируй ключи перед выводом.

Get возвращает null при отсутствии ключа, но HashMap может также хранить null как значение. Для различения нужен containsKey либо договор, запрещающий null-значения. GetOrDefault подставляет значение при отсутствии записи, но не заменяет существующее значение null.

JAVA

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Example16 {
    public static void main(String[] args) {
        String[] tools = {"пила", "молоток", "пила"};
        Map<String, Integer> counts = new LinkedHashMap<>();
        for (String tool : tools) {
            int old = counts.getOrDefault(tool, 0);
            counts.put(tool, old + 1);
        }
        for (Map.Entry<String, Integer> entry : counts.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}
```

ВЫВОД ПРИМЕРА

```
пила: 2
молоток: 1
```

GetOrDefault не добавляет запись сам. Изменение происходит в put. Обход entrySet даёт ключ и значение вместе. В примере порядок первого появления сохраняется выбранной реализацией, а не свойством любого Map.

Что считает равными HashMap и HashSet

Object.equals по умолчанию сравнивает идентичность. Для собственного класса-значения часто нужно сравнивать значимые поля. Equals должен быть рефлексивным, симметричным, транзитивным, согласованным при неизменности значимых данных и возвращать false для null.

HashCode связан с equals: если два объекта равны, их хеш-коды обязаны совпадать. Обратное неверно: разные объекты могут иметь одинаковый хеш. Коллизия не является ошибкой; структура дополнительно проверяет равенство. Переопределяй equals и hashCode вместе и по одним значимым данным.

Зачем нужны оба? Хеш помогает быстро выбрать область поиска, equals определяет соответствие внутри неё. Если равные объекты имеют разные хеши, структура может искать не там и не находить запись, которая по смыслу существует.

Опасность изменяемого ключа

Если после вставки поменять поля ключа, участвующие в hashCode, объект останется размещён по прежнему хешу, а новый поиск будет использовать новое значение. Даже поиск тем же объектом может перестать работать. Для ключей подходят неизменяемые значения: String, enum, корректно устроенные собственные типы, некоторые record.

Не путай изменение значения Map с изменением ключа. Остаток товара можно менять, если идентификатор товара остаётся стабильным. Если изменился идентификатор, это операция удаления старой связи и создания новой, а не случайное редактирование поля.

Сравнение и сортированные структуры

TreeMap использует сравнение ключей; договор сравнения должен быть согласован с предполагаемым равенством. Результат 0 означает эквивалентный ключ для сортированной структуры. Если сравнить людей только по возрасту, два разных человека одинакового возраста могут слиться в одну запись. Для сортировки отчёта и для определения уникальности иногда нужны разные договоры.

Для Integer сравнения не вычитай значения ради результата comparator: вычитание может переполниться. Integer.compare выражает сравнение без этой ошибки. Comparator будет практически использован в главе о lambda.

Ошибки и самопроверка

Проверь повторный put, отсутствующий ключ, пустую карту и несколько ключей с одинаковым значением. Не полагайся на порядок HashMap и не проверяй его текстовый вывод как обязательный.

Перед практикой объясни, почему одинаковый hashCode не доказывает equals. Нарисуй последствия изменения ключа. Укажи, какая часть модели должна оставаться стабильной в телефонном справочнике и в реестре складских позиций.

ГЛАВА 16 / EX136

Склад по ключу

Уровень 1 · Написать код · Ex136.java

Перед началом

Put связывает один ключ с одним текущим значением.

Твоя задача

Создай LinkedHashMap<String,Integer> из данных, выведи значение для bolt и результат getOrDefault для неизвестного key с запасным 0.

```java
String[] keys = {"bolt", "nut"};
int[] amounts = {12, 8};
```

Проверь себя

- bolt -> 12; unknown -> 0.
- Карта содержит две записи, порядок вставки сохранён.

План / прогноз / причина ошибки

ГЛАВА 16 / EX137

Повторный put

Уровень 1 · Предсказать результат · Ex137.java

Перед началом

Замена значения не добавляет второй экземпляр ключа.

Твоя задача

Предскажи прежнее значение, размер и карту.

```java
Map<String, Integer> values = new LinkedHashMap<>();
values.put("A", 2);
Integer old = values.put("A", 7);
values.put("B", 7);
System.out.println(old);
System.out.println(values.size());
System.out.println(values);
```

Проверь себя

- Прогноз до запуска.
- Объяснено, почему одинаковые значения у разных ключей разрешены.

План / прогноз / причина ошибки

ГЛАВА 16 / EX138

Равенство без хеша

Уровень 1 · Найти ошибку · Ex138.java

Перед началом

Равные ключи обязаны иметь равные hashCode.

Твоя задача

Найди нарушение договора класса Key и исправь equals/hashCode согласованно. Сравнивай id; null-id запрещён конструктором по договору. Проверь поиск через новый равный Key.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static class Key {
    private final String id;
    Key(String id) { this.id = id; }
    @Override public boolean equals(Object other) {
        return other instanceof Key && id.equals(((Key) other).id);
    }
    // hashCode отсутствует намеренно
}
```

Проверь себя

- Два Key("A") равны и имеют равный хеш.
- После put первым ключом get вторым находит значение; коллизия не считается равенством.

План / прогноз / причина ошибки

ГЛАВА 16 / EX139

Частота событий

Уровень 2 · Написать код · Ex139.java

Перед началом

Счётчик по ключу обновляет прежнее число, а не перезаписывает единицей.

Твоя задача

Подсчитай частоту каждой строки в events. Выводи «ключ: число» в порядке первого появления.

```java
String[] events = {"join", "move", "move", "leave", "join"};
```

Проверь себя

- join: 2, move: 2, leave: 1.
- Пустой -> пустой отчёт; регистр значим.

План / прогноз / причина ошибки

ГЛАВА 16 / EX140

Ключ изменился после вставки

Уровень 2 · Объяснить поведение · Ex140.java

Перед началом

Хеширование требует стабильного ключа на время хранения.

Твоя задача

В комментариях объясни, почему изменение поля id ключа после put может сделать запись недоступной. Предложи неизменяемый ключ и опиши, как легально изменить идентификатор записи.

Проверь себя

- Не обещается гарантированный результат get на повреждённом договоре.
- Указаны удаление по старому ключу и вставка нового либо стабильный внешний идентификатор.

План / прогноз / причина ошибки

ГЛАВА 16 / EX141

GetOrDefault не записывает

Уровень 2 · Исправить программу · Ex141.java

Перед началом

Чтение запасного значения не обновляет Map автоматически.

Твоя задача

Исправь подсчёт, чтобы все повторения учитывались.

JAVA

```
String[] items = {"A", "A", "B"};
Map<String, Integer> counts = new LinkedHashMap<>();
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
for (String item : items) {
    int value = counts.getOrDefault(item, 0);
    value++;
}
System.out.println(counts);
```

Проверь себя

- A=2, B=1.
- Пустой набор даёт пустую карту.

План / прогноз / причина ошибки

ГЛАВА 16 / EX142

Группы по отделу

Уровень 3 · Написать код · Ex142.java

Перед началом

Значение карты тоже может быть коллекцией.

Твоя задача

Сгруппируй names по параллельному массиву departments. Порядок отделов - первое появление, имён внутри - исходный порядок. Выведи отдел и его List<String>.

```java
String[] names = {"Лея", "Рин", "Мира", "Тим"};
String[] departments = {"lab", "field", "lab", "field"};
```

Проверь себя

- lab -> [Лея,Мира], field -> [Рин,Тим].
- Пустой -> пусто; каждому отделу создан свой список; длины массивов равны по договору.

План / прогноз / причина ошибки

ГЛАВА 16 / EX143

Слияние остатков

Уровень 3 · Написать код · Ex143.java

Перед началом

Объединение карт требует отдельного правила для совпавших ключей.

Твоя задача

Объедини left и right в новую LinkedHashMap<String,Long>, суммируя значения общих ключей. Порядок: ключи left, затем новые из right. Исходники неизменны, переполнение long исключено.

```java
Map<String, Long> left = new LinkedHashMap<>();
left.put("A", 4L); left.put("B", 2L);
Map<String, Long> right = new LinkedHashMap<>();
right.put("B", 7L); right.put("C", 1L);
```

Проверь себя

- Результат A=4,B=9,C=1.
- Две пустые -> пустая; результат не является left или right.

План / прогноз / причина ошибки

ГЛАВА 16 / EX144

Двунаправленная уникальность

Уровень 3 · Написать код · Ex144.java

Перед началом

Уникальность ключей Map не гарантирует уникальность значений.

Твоя задача

Проверь, можно ли построить справочник id->name с уникальными id и уникальными name. При повторе любого выведи INVALID, иначе VALID. Регистр значим; массивы равной длины, элементы не null.

```java
String[] ids = {"U1", "U2", "U3"};
String[] names = {"Лея", "Рин", "Лея"};
```

Проверь себя

- Исходный INVALID из-за имени.
- Повтор id с другим именем тоже INVALID; пустой набор VALID; молчаливый put не скрывает повторы.

План / прогноз / причина ошибки

ГЛАВА 17 / ТЕОРИЯ

17. Generics: связь типов в договоре

Параметр типа сохраняет информацию

Generics позволяют описать алгоритм или контейнер один раз и связать типы его входов и выходов. В Box<T> буква T обозначает параметр типа, а Box<String> подставляет String. Компилятор проверяет, что туда не попало число, и знает тип результата get без ручного приведения.

Object мог бы хранить любую ссылку, но связь между «что положили» и «что достали» теряется. Ручное приведение переносит часть проверки на время выполнения. Raw type, например просто List, тоже теряет информацию и создаёт предупреждения. В курсе raw types не используются в рабочем коде.

JAVA

```
public class Example17 {
    static final class Slot<T> {
        private final T value;

        Slot(T value) {
            this.value = value;
        }

        T get() {
            return value;
        }
    }

    static <T> T chooseFirst(T first, T second) {
        return first;
    }

    public static void main(String[] args) {
        Slot<String> slot = new Slot<>("карта");
        String item = slot.get();
        System.out.println(item);
        System.out.println(chooseFirst(6, 9));
    }
}
```

ВЫВОД ПРИМЕРА

```
карта
6
```

Параметр метода <T> объявлен перед возвращаемым типом. Он не связан с одноимённым параметром другого класса. Ромб <> позволяет вывести аргумент типа конструктора из контекста. Для примитивов применяются обёртки: Slot<Integer>, а не Slot<int>.

Инвариантность

Integer является подтипом Number, но List<Integer> не является подтипом List<Number>. Иначе через ссылку List<Number> можно было бы добавить Double в исходный список Integer. Запрет защищает обещание списка о типе элементов.

Это называется инвариантностью параметризованного типа. Массивы имеют другую модель совместимости и проверяют часть записей при выполнении; не переноси правила массивов на generics.

Wildcard и направление данных

`List<? extends Number>` означает список некоторого неизвестного подтипа Number. Из него безопасно читать Number. Добавить произвольный Integer нельзя: фактическим списком мог быть List<Double>. Слово extends здесь не даёт право создавать новый неизвестный тип.

`List<? super Integer>` означает список некоторого супертипа Integer. В него безопасно добавить Integer, потому что это может быть List<Integer>, List<Number> или List<Object>. При чтении без дополнительной информации гарантирован только Object.

Правило PECS помогает выбрать границу: producer extends, consumer super. Это правило о потоке значений конкретной операции. Если метод и читает, и пишет строго связанный тип, нужен именованный параметр T либо конкретный тип, а не механическое применение wildcard.

Ограничения параметров

`<T extends Number>` разрешает использовать операции, известные Number, но не произвольную арифметику `a + b` над T. Операторы Java не становятся обобщёнными операторами чисел. Можно вызвать doubleValue и сознательно принять соответствующий тип и точность результата.

Один параметр связывает несколько мест договора. Например, метод копирования принимает источник `List<? extends T>` и приёмник `List<? super T>`. Два независимых wildcard такой связи автоматически не создают.

Стирание и пределы

Большая часть информации о параметрах generics стирается из обычного представления объектов при выполнении; компилятор вставляет необходимые проверки и приведения. Поэтому нельзя написать `new T()` или `new T[10]`, а `instanceof List<String>` не является обычной допустимой проверкой. Проверка `instanceof List<?>` возможна, но не доказывает тип всех элементов.

Не подавляй предупреждения, пока не понял, какое обещание обходишь. Перед практикой объясни запрет List<Integer> -> List<Number>, выбери extends/super для чтения и добавления и укажи, почему общий тип Object слабее связанного параметра T.

ГЛАВА 17 / EX145

Коробка заданного типа

Уровень 1 · Написать код · Ex145.java

Перед началом

Параметр типа связывает конструктор и возвращаемое значение.

Твоя задача

Создай final Box<T> с private final T value, конструктором и get(). Создай Box<String> и Box<Integer>, прочитай без приведений. Null допустим.

```java
String text = "компас";
Integer number = 7;
```

Проверь себя

- Получаются String компас и Integer 7.
- Box<String>(null) возвращает null; raw type не используется.

План / прогноз / причина ошибки

ГЛАВА 17 / EX146

Тип выводится из аргумента

Уровень 1 · Предсказать результат · Ex146.java

Перед началом

Обобщённый метод связывает вход и выход одним T.

Твоя задача

Предскажи вывод и укажи типы переменных a и b при компиляции.

JAVA

```
String a = echo("A");
Integer b = echo(9);
System.out.println(a);
System.out.println(b);
```

JAVA

```
static <T> T echo(T value) { return value; }
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз до запуска.
- Отмечены boxing и отсутствие ручного cast.

План / прогноз / причина ошибки

ГЛАВА 17 / EX147

Список чисел не ковариантен

Уровень 1 · Найти ошибку · Ex147.java

Перед началом

Инвариантность защищает список от добавления чужого подтипа.

Твоя задача

Объясни ошибку присваивания. Для задачи только чтения предложи тип ссылки, который принимает List<Integer> и позволяет читать Number.

```java
List<Integer> ints = new ArrayList<>(List.of(1, 2));
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
List<Number> numbers = ints;
```

Проверь себя

- Исправление не использует raw type или unchecked cast.
- Объяснено, что случилось бы при разрешённом добавлении Double.

План / прогноз / причина ошибки

ГЛАВА 17 / EX148

Первый или запасной

Уровень 2 · Написать код · Ex148.java

Перед началом

Параметр T сохраняет связь типа списка и запасного значения.

Твоя задача

Реализуй static <T> T firstOr(List<T> values,T fallback): первый элемент или fallback при пустоте. Null-список запрещён, null-элементы разрешены.

```java
List<String> values = List.of("карта", "схема");
String fallback = "нет";
```

Проверь себя

- Исходный -> карта; пустой -> нет.
- Список с первым null возвращает null, а не fallback; работает также для Integer.

План / прогноз / причина ошибки

ГЛАВА 17 / EX149

Почему нельзя добавить

Уровень 2 · Объяснить поведение · Ex149.java

Перед началом

Extends разрешает чтение общего типа, не запись произвольного подтипа.

Твоя задача

Объясни, почему List<? extends Number> можно читать как Number, но нельзя безопасно добавить Integer. Приведи в комментарии возможный фактический List<Double>.

Проверь себя

- Показан риск нарушить конкретный тип элементов.
- Не предложено снять ограничение cast-ом.

План / прогноз / причина ошибки

ГЛАВА 17 / EX150

Producer и consumer перепутаны

Уровень 2 · Исправить программу · Ex150.java

Перед началом

Приёмник должен принимать элементы источника, а не ограничивать их неизвестным подтипом.

Твоя задача

Исправь сигнатуру appendAll так, чтобы копировать из List<Integer> в List<Number>. Реализуй цикл, не очищая цель.

```java
List<Integer> source = List.of(2, 5);
List<Number> target = new ArrayList<>();
target.add(1.5);
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
static <T> void appendAll(List<? super T> source, List<? extends T> target) {
    // Эти границы намеренно неверны для копирования.
}
```

Проверь себя

- Результат [1.5,2,5].
- Источник не меняется; работают String->Object; unchecked предупреждений нет.

План / прогноз / причина ошибки

ГЛАВА 17 / EX151

Сумма любого Number

Уровень 3 · Написать код · Ex151.java

Перед началом

Ограничение Number даёт методы преобразования, а не оператор + для T.

Твоя задача

Реализуй static double sum(List<? extends Number> values) через doubleValue.
Null-элементы запрещены. Проверь Integer и Double без копирования в List<Number>.

```java
List<Integer> ints = List.of(2, 3, 4);
List<Double> doubles = List.of(0.5, 1.25);
```

Проверь себя

- Суммы 9.0 и 1.75.
- Пустой -> 0.0; в комментарии указана возможная потеря точности для больших целых.

План / прогноз / причина ошибки

ГЛАВА 17 / EX152

Обобщённая пара

Уровень 3 · Написать код · Ex152.java

Перед началом

Два параметра типа описывают независимые роли данных.

Твоя задача

Создай immutable Pair<K,V> с key(),value(). Напиши static <K,V> Pair<V,K> swapped(Pair<K,V>). Null-компоненты допустимы; исходная пара неизменна.

```java
String key = "dock";
Integer value = 12;
```

Проверь себя

- Pair<String,Integer> превращается в Pair<Integer,String> со значениями 12,dock.
- Никаких Object-приведений или raw types; null переносится.

План / прогноз / причина ошибки

ГЛАВА 17 / EX153

Заполнение приёмника

Уровень 3 · Написать код · Ex153.java

Перед началом

Super позволяет добавить определённый тип, но чтение даёт лишь Object.

Твоя задача

Напиши static void addRange(List<? super Integer> target,int from,int to), добавляя включительный диапазон при from<=to. Входы -1000..1000; обратный диапазон не добавляет ничего. Проверь List<Integer>, List<Number>, List<Object>.

```java
int from = 2;
int to = 4;
```

Проверь себя

- Во все три приёмника добавляются 2,3,4.
- Прежние элементы сохранены; from=4,to=2 не меняет цель.

План / прогноз / причина ошибки

ГЛАВА 18 / ТЕОРИЯ

18. Файлы, пути и устойчивый текстовый формат

Путь не равен содержимому файла

Path представляет путь в файловой системе. Само создание Path не открывает файл и не доказывает его существование. Относительный путь вычисляется от рабочей директории процесса, которая в IDEA может отличаться от папки исходника. Для диагностики полезно вывести path.toAbsolutePath, но не зашивать личный абсолютный путь в проект.

Files содержит операции чтения, записи и управления. В курсе для текста явно указывается StandardCharsets.UTF_8: русские данные не должны зависеть от системной кодировки. Расширение `.txt` само по себе не задаёт кодировку или формат полей.

JAVA

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class Example18 {
    public static void main(String[] args) throws IOException {
        Path directory = Files.createTempDirectory("java-guide-");
        Path file = directory.resolve("routes.txt");
        try {
            Files.write(file, List.of("порт;12", "парк;5"),
                    StandardCharsets.UTF_8);
            List<String> lines = Files.readAllLines(file,
                    StandardCharsets.UTF_8);
            System.out.println(lines.size());
            System.out.println(lines.get(0));
        } finally {
            Files.deleteIfExists(file);
            Files.deleteIfExists(directory);
        }
    }
}
```

ВЫВОД ПРИМЕРА

```
2
порт;12
```

Пример использует временную папку и удаляет её. Throws передаёт IOException вызывающей среде; это демонстрация операции, а не конечный пользовательский интерфейс. В мини-проекте ошибки должны получить понятное сообщение на уровне команды.

Чтение целиком и потоковое чтение

ReadString и readAllLines удобны для маленьких файлов. Они загружают содержимое в память. Для большого журнала нужен BufferedReader или поток строк. Поток от Files.lines владеет открытым ресурсом: используй try-with-resources и не возвращай его из метода, который уже закрыл ресурс.

Проверка Files.exists перед чтением не гарантирует успех: файл может исчезнуть между действиями. Лучше попытаться выполнить нужную операцию и обработать IOException. Проверка существования иногда полезна для пользовательского сценария, но не заменяет обработку отказа.

Формат - часть договора

Пусть запись имеет вид `name;count`. Нужно определить: разрешён ли `;` внутри имени, допустимы ли пустые поля, что означает пробел, какой диапазон count, как представлены переводы строки. В учебных задачах разделитель внутри имени запрещён. Это простой формат с ограничением, а не полноценный CSV.

При разборе используй split с сохранением пустого последнего поля и проверяй количество полей до индексации. Номер строки и причина отказа важнее общего «не удалось прочитать». Выбери политику: прекратить загрузку целиком или пропустить ошибочные строки с отчётом. Нельзя молча потерять записи.

Запись и сохранность данных

Обычная запись может создать файл или обрезать существующий. Режим добавления задаётся явно через StandardOpenOption.APPEND и CREATE, если это требуется. Родительские каталоги не всегда создаются автоматически; Files.createDirectories решает отдельную задачу.

Для сохранения важных данных сначала подготовь полное новое содержимое, запиши во временный файл в том же каталоге, затем замени целевой через move. AtomicMove может быть не поддержан файловой системой; политика запасного варианта должна быть явной. Даже такая схема не равна универсальной транзакционной базе данных и не гарантирует защиту от всех сбоев питания без дополнительных мер.

В заданиях запись выполняется только в `build/sandbox` или во временную папку. Не проверяй удаление и перезапись на личных файлах. Пути из пользовательского ввода требуют отдельного ограничения, если приложение должно оставаться внутри выделенного каталога.

Ошибки и самопроверка

Проверь пустой файл, отсутствие, русскую строку, повреждённую запись и последний перевод строки. Отделяй ошибку файловой операции от ошибки формата: IOException и NumberFormatException говорят о разных причинах.

Перед практикой объясни, откуда считается относительный путь. Назови момент открытия ресурса. Напиши словами договор строки данных и политику отказа до того, как начнёшь разбирать её кодом.

ГЛАВА 18 / EX154

Первая запись UTF-8

Уровень 1 · Написать код · Ex154.java

Перед началом

Path задаёт адрес, Files выполняет операцию.

Твоя задача

Создай build/sandbox/ex154, запиши в message.txt строку text в UTF-8, прочитай и выведи обратно. Родительские папки создай. Ошибку IOException покажи как IO_ERROR.

```java
String text = "Привет, маяк!";
Path directory = Path.of("build", "sandbox", "ex154");
```

Проверь себя

- Прочитанная строка совпадает с исходной.
- Повторный запуск заменяет содержимое только своего message.txt; личные пути не используются.

План / прогноз / причина ошибки

ГЛАВА 18 / EX155

Разрешение пути

Уровень 1 · Предсказать результат · Ex155.java

Перед началом

Нормализация убирает синтаксические части пути, но не читает файловую систему.

Твоя задача

Предскажи вывод на Ubuntu и объясни, какой путь относительный.

```java
Path base = Path.of("build", "sandbox");
Path target = base.resolve("logs/../data.txt").normalize();
System.out.println(target);
System.out.println(target.isAbsolute());
```

Проверь себя

- Прогноз до запуска.
- Не утверждай, что normalize проверяет существование или разрешает симлинки.

План / прогноз / причина ошибки

ГЛАВА 18 / EX156

Потерянное последнее поле

Уровень 1 · Найти ошибку · Ex156.java

Перед началом

Split по умолчанию может удалить пустые поля в конце.

Твоя задача

Найди ошибку в разборе строки из ровно двух полей, где второе разрешено пустым. Исправь и проверяй длину до обращения.

```java
String line = "lamp;";
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
String[] fields = line.split(";");
System.out.println(fields[1]);
```

Проверь себя

- Для lamp; два поля: lamp и пустое.
- lamp без разделителя INVALID; lamp;4;5 INVALID.

План / прогноз / причина ошибки

ГЛАВА 18 / EX157

Сумма строк файла

Уровень 2 · Написать код · Ex157.java

Перед началом

Чтение файла и разбор чисел имеют разные причины отказа.

Твоя задача

Создай свой временный UTF-8 файл с lines, затем прочитай. Каждая строка корректное целое; вычисли long-сумму. Удали временный файл в finally. При IOException выведи IO_ERROR.

```java
List<String> lines = List.of("12", "-2", "0");
```

Проверь себя

- Сумма 10.
- Пустой файл -> 0; временный файл удалён при успехе; запись только в temp.

План / прогноз / причина ошибки

ГЛАВА 18 / EX158

Проверка существования не гарантия

Уровень 2 · Объяснить поведение · Ex158.java

Перед началом

Между exists и read состояние внешнего мира может измениться.

Твоя задача

Объясни, почему if(Files.exists(path)) не отменяет catch(IOException). Назови минимум три причины отказа чтения и предложи место пользовательского сообщения.

Проверь себя

- Упомянуты исчезновение, права или тип пути/ошибка устройства.
- Обработка файловой ошибки отделена от ошибки формата строки.

План / прогноз / причина ошибки

ГЛАВА 18 / EX159

Поток остался открытым

Уровень 2 · Исправить программу · Ex159.java

Перед началом

Files.lines открывает ресурс, который нужно закрыть.

Твоя задача

Исправь фрагмент через try-with-resources. Файл создай самостоятельно во временной папке, заполни тремя строками и удали после проверки. Не возвращай закрытый stream наружу.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
long count = Files.lines(path, StandardCharsets.UTF_8).count();
System.out.println(count);
```

Проверь себя

- Подсчёт даёт 3.
- Пустой файл даёт 0; ресурс закрывается до удаления.

План / прогноз / причина ошибки

ГЛАВА 18 / EX160

Загрузка без молчаливой потери

Уровень 3 · Написать код · Ex160.java

Перед началом

Ошибочные строки требуют отчёта с номером.

Твоя задача

Разбери каждую строку как id;count: id непустой, без краевых пробелов; count целое >=0. Ровно два поля; пустые строки считаются ошибкой. Выведи VALID:n и для каждой ошибки ERROR:номер, нумерация с 1. Данные запиши и прочитай из своего temp-файла.

```java
List<String> lines = List.of("A;3", "B;", "C;-1", "D;7", "oops");
```

Проверь себя

- VALID:2; ошибки 2,3,5.
- Пустой файл VALID:0; A;0 допустимо; IOException обрабатывается отдельно.

План / прогноз / причина ошибки

ГЛАВА 18 / EX161

Журнал с добавлением

Уровень 3 · Написать код · Ex161.java

Перед началом

Режим добавления нужно выбрать явно вместо усечения прежних данных.

Твоя задача

В своём build/sandbox/ex161/events.txt запиши initial с заменой, затем добавь extra через APPEND. Прочитай строки и выведи количество и последнюю. Каждый запуск начинается с initial, не накапливает старую проверку.

```java
List<String> initial = List.of("BOOT", "READY");
List<String> extra = List.of("SAVE");
```

Проверь себя

- Три строки, последняя SAVE.
- Повторный запуск тоже три; все строки в UTF-8; папка создана.

План / прогноз / причина ошибки

ГЛАВА 18 / EX162

Сначала подготовить затем заменить

Уровень 3 · Написать код · Ex162.java

Перед началом

Сохранение не должно уничтожать прежний файл на этапе подготовки.

Твоя задача

Спроектируй и реализуй save(Path target,List<String> lines): temp-файл в родительском каталоге, UTF-8 запись, move с ATOMIC_MOVE и REPLACE_EXISTING; если AtomicMoveNotSupportedException, явно сообщи отказ и оставь старый target, без тихого неатомарного fallback. В finally убери temp. Проверяй только свой sandbox.

```java
List<String> oldLines = List.of("OLD");
List<String> newLines = List.of("NEW", "READY");
```

Проверь себя

- При успехе target содержит новые строки.
- До move целевой файл не обрезается; неподдержанный atomic move имеет отдельную политику; не обещана защита от всех аппаратных сбоев.

План / прогноз / причина ошибки

ГЛАВА 19 / ТЕОРИЯ

19. Отладка, проверки и слои приложения

Успех запуска ещё не доказательство

Программа может завершиться без исключения и посчитать неверную стоимость. Проверка должна сравнить наблюдаемое поведение с условием. Один пример подтверждает только один случай. Для диапазона нужны границы, для коллекции - пустота, один элемент и повторы, для отказа - вход, который обязан быть отклонён.

Сначала разделяй наблюдение и гипотезу. «При входе 0 получено -1» - наблюдение. «Проблема в начальном максимуме» - гипотеза. Измени один фактор или проверь промежуточное значение, чтобы отличить её от других объяснений.

Отладчик и минимальный воспроизводимый пример

В IDEA поставь breakpoint на исполняемой строке. Debug остановит выполнение перед ней. Step Over выполняет текущую операцию без входа внутрь метода, Step Into входит в вызываемый метод, Resume продолжает до следующей остановки. Наблюдай значения и стек вызовов. Некоторые выражения в Evaluate Expression способны менять состояние; не вызывай случайные методы только ради просмотра.

Для цикла найди первую итерацию, где инвариант перестаёт выполняться. Для объекта проследи создание и последний метод изменения. Если проблема исчезает на маленьком входе, увеличивай его по одному элементу. Это быстрее, чем перечитывать весь файл без конкретного вопроса.

JAVA

```
public class Example19 {
    static int clamp(int value, int low, int high) {
        if (low > high) {
            throw new IllegalArgumentException("invalid bounds");
        }
        return Math.max(low, Math.min(value, high));
    }

    static void check(boolean condition, String name) {
        if (!condition) {
            throw new AssertionError(name);
        }
    }

    public static void main(String[] args) {
        check(clamp(-3, 0, 9) == 0, "below");
        check(clamp(4, 0, 9) == 4, "inside");
        check(clamp(12, 0, 9) == 9, "above");
        System.out.println("3 проверки пройдены");
    }
}
```

ВЫВОД ПРИМЕРА

```
3 проверки пройдены
```

Это простой проверочный стенд без библиотек. AssertionError здесь явно создаётся проверяющим методом. Он отличается от ключевого слова assert, которое по умолчанию может быть выключено и требует -ea. Не используй отключаемый assert для проверки пользовательского ввода.

Полезные тесты

Тест должен проверять договор, а не повторять реализацию той же формулой с теми же ошибками. Для маленького примера ожидаемый результат можно вывести вручную. Для сортировки полезны свойства: элементы не потеряны, порядок не убывает, повторная сортировка не меняет результат. Для перевода в формат и обратно полезен round-trip, но он не доказывает соответствие внешнему формату, если обе стороны одинаково ошибаются.

Отказы тоже проверяются: неверный аргумент должен быть отвергнут, состояние после отказа должно сохранять инвариант. Не засчитывай любой пойманный Exception как нужный отказ: проверь конкретный тип и точку, которую тестировал.

Разделение обязанностей

Небольшое приложение удобно делить на три части. Модель содержит данные и правила. Сервис выполняет сценарий над моделью. Адаптеры читают консоль, печатают и работают с файлом. В начале это могут быть методы или вложенные классы; количество файлов не является целью.

Если вычисление зависит от терминала, файла и текущего времени одновременно, его трудно воспроизвести. Передавай входные данные и зависимости явно. Для накопителя сообщения можно подменить вывод реализацией интерфейса. Для счётчика дней можно передать дату вместо обращения к реальным часам внутри каждой функции.

Регрессия и границы автоматизации

После исправления добавь минимальный случай, который ловил ошибку. Затем выполни связанные проверки. Компиляция всех стартовых файлов проверяет каркас курса, но не правильность ещё не написанных решений. Автоматическая сверка условий и данных обнаруживает рассинхронизацию, но не заменяет педагогическое чтение текста.

Перед практикой запиши ожидаемое поведение до реализации. Для одного случая определи обычный вход, границу и отказ. Объясни, что именно доказывает каждая проверка и какое поведение остаётся непроверенным.

ГЛАВА 19 / EX163

Три проверки контракта

Уровень 1 · Написать код · Ex163.java

Перед началом

Ожидаемое поведение задаётся до реализации метода.

Твоя задача

Напиши static int absSmall(int n) для -1000..1000 и отдельный check-метод, бросающий AssertionError при несовпадении. Проверь отрицательное, ноль и положительное. Assert-ключевое слово не используй.

```java
int negative = -7;
int zero = 0;
int positive = 9;
```

Проверь себя

- Ожидаются 7,0,9.
- При специально ошибочной реализации одна проверка падает; main сообщает OK только после всех проверок.

План / прогноз / причина ошибки

ГЛАВА 19 / EX164

Граница проверяется отдельно

Уровень 1 · Предсказать результат · Ex164.java

Перед началом

Неполный набор тестов может пропустить ошибку включённой границы.

Твоя задача

Предскажи результаты двух вызовов и назови недостающий граничный тест для условия «допустимо 1..10 включительно».

JAVA

```
System.out.println(allowed(5));
System.out.println(allowed(11));
```

JAVA

```
static boolean allowed(int n) { return n >= 1 && n < 10; }
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз до запуска.
- В комментарии назван вход, который обнаружит дефект.

План / прогноз / причина ошибки

ГЛАВА 19 / EX165

Тест копирует ошибку

Уровень 1 · Найти ошибку · Ex165.java

Перед началом

Проверка, повторяющая ту же ошибочную формулу, не независима.

Твоя задача

Найди дефект вычисления среднего и дефект теста. Исправь метод и запиши ожидаемый результат для 1 и 2 числом, полученным вручную. Метод должен корректно считать среднее любых двух int без переполнения суммы.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
static double average(int a, int b) { return (a + b) / 2; }
// В тесте:
boolean ok = average(1, 2) == (1 + 2) / 2;
```

Проверь себя

- Среднее 1 и 2 равно 1.5.
- Тест не повторяет целое деление реализации; проверен ещё случай -2 и 2. MAX_VALUE,MAX_VALUE даёт 2147483647.0.

План / прогноз / причина ошибки

ГЛАВА 19 / EX166

Отказ тоже проверяется

Уровень 2 · Написать код · Ex166.java

Перед началом

Проверять нужно конкретный отказ, а не любое исключение.

Твоя задача

Реализуй requirePositive(int n), возвращающий n при n>0 и бросающий IllegalArgumentException иначе. Напиши проверки для 1,0,-1; если отказ не возник, тест должен сам упасть.

Проверь себя

- Ловится именно IllegalArgumentException.
- Реализация, принимающая 0, обнаруживается; проверки не зависят от -ea.

План / прогноз / причина ошибки

ГЛАВА 19 / EX167

Отладочный план

Уровень 2 · Объяснить поведение · Ex167.java

Перед началом

Наблюдение отделяют от предположения о причине.

Твоя задача

Составь план диагностики: программа теряет последний элемент отчёта. Укажи минимальный вход, breakpoint, наблюдаемые переменные и гипотезу. Никаких случайных исправлений до наблюдения.

Проверь себя

- Названы индекс, length/size и условие цикла.
- Минимальный вход включает один элемент; есть способ опровергнуть гипотезу.

План / прогноз / причина ошибки

ГЛАВА 19 / EX168

Сообщение внутри расчёта

Уровень 2 · Исправить программу · Ex168.java

Перед началом

Печать вычислителем усложняет переиспользование и проверку.

Твоя задача

Перепиши totalCents как чистый метод с long-результатом без вывода. Main отвечает за форматирование. Quantity и unitCents >=0, произведение помещается в long.

```java
int quantity = 3;
int unitCents = 125;
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
static void totalCents(int quantity, int unitCents) {
    System.out.println("TOTAL:" + quantity * unitCents);
}
```

Проверь себя

- Метод возвращает 375; main печатает TOTAL:375.
- Метод не читает Scanner и не пишет файл.

План / прогноз / причина ошибки

ГЛАВА 19 / EX169

Свойства сортировки

Уровень 3 · Написать код · Ex169.java

Перед началом

Проверка свойства дополняет конкретный ожидаемый пример.

Твоя задача

Напиши тестовый стенд для Arrays.sort над int[]: результат не убывает, частоты элементов сохранены, повторная сортировка не меняет результат. Используй копии и Map частот, без готового альтернативного сортировщика.

```java
int[] values = {4, -1, 4, 0};
```

Проверь себя

- Проверены пустой, один элемент и повторы.
- Перестановка элементов проходит, потеря элемента или нарушение порядка обнаруживаются.

План / прогноз / причина ошибки

ГЛАВА 19 / EX170

Сервис без терминала

Уровень 3 · Написать код · Ex170.java

Перед началом

Явная зависимость позволяет проверить сценарий в памяти.

Твоя задача

Создай Storage с save(String), MemoryStorage с List<String> и NoteService, получающий Storage. addNote удаляет краевые пробелы, запрещает пустое и сохраняет один раз. Проверь без чтения терминала и без файлов.

```java
String note = "  проверить люк  ";
```

Проверь себя

- Хранилище содержит «проверить люк».
- Пустая заметка отвергнута и размер не меняется; отказ IllegalArgumentException.

План / прогноз / причина ошибки

ГЛАВА 19 / EX171

Регрессия после исправления

Уровень 3 · Написать код · Ex171.java

Перед началом

Тест должен воспроизводить прежнюю ошибку на минимальном входе.

Твоя задача

Реализуй medianOfThree(int a,int b,int c) без массивов и без суммирования трёх чисел. Добавь проверки всех порядков 1,2,3, повторов и предельных int. В комментарии объясни, какой тест ловит переполнение наивной формулы.

```java
int a = Integer.MAX_VALUE;
int b = Integer.MIN_VALUE;
int c = 0;
```

Проверь себя

- Исходный -> 0; 2,2,9 -> 2.
- Все шесть перестановок 1,2,3 -> 2; нет переполнения промежуточной суммы.

План / прогноз / причина ошибки

ГЛАВА 20 / ТЕОРИЯ

20. Современные модели и сборка законченного приложения

Enum вместо произвольной строки

Когда набор состояний закрыт, enum ограничивает его: `NEW`, `ACTIVE`, `DONE`. Опечатка в имени становится ошибкой компиляции, а не странной веткой во время запуска. Enum может иметь поля и методы. Внешний текст нужно явно преобразовать в допустимое состояние; valueOf требует точного имени и бросает исключение для неизвестного.

Не выбирай enum, если набор определяется пользователем и растёт из файла без пересборки. Представление должно соответствовать источнику правил. Переходы между состояниями тоже нужно проверять: наличие трёх значений не означает, что любой переход допустим.

Record как модель значения

Record в Java 17 - краткая форма класса для данных с компонентами. Он получает конструктор, методы доступа по именам компонентов, equals, hashCode и toString. Record final и не предназначен для наследования от другого класса. Можно добавить проверку в компактный конструктор и обычные методы.

Неизменяемость record поверхностна. Компонент List может ссылаться на изменяемый список; проверка конструктора и List.copyOf помогают защитить структуру. Record хорош для координат или результата вычисления, но не заменяет любую модель с жизненным циклом и управляемым изменением.

JAVA

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Example20 {
    enum Priority { LOW, HIGH }

    record Ticket(String title, Priority priority) {
        Ticket {
            if (title == null || title.isBlank() || priority == null) {
                throw new IllegalArgumentException("invalid ticket");
            }
        }
    }

    public static void main(String[] args) {
        List<Ticket> tickets = new ArrayList<>(List.of(
                new Ticket("Мост", Priority.HIGH),
                new Ticket("Арка", Priority.LOW)));
        tickets.sort(Comparator.comparing(Ticket::title));
        tickets.forEach(ticket -> System.out.println(ticket.title()));
    }
}
```

ВЫВОД ПРИМЕРА

```
Арка
Мост
```

Lambda и ссылка на метод

Функциональный интерфейс имеет один абстрактный метод. Lambda задаёт его реализацию короткой записью. В forEach параметр ticket получает очередной элемент; тело выполняется для каждого. `Ticket::title` - ссылка на метод, здесь предоставляющая ключ сравнения. Сначала умей записать то же поведение обычным классом с интерфейсом, тогда краткая форма не будет магией.

Локальная переменная, используемая lambda, должна быть final или effectively final: не переназначаться после инициализации. Это не делает все захваченные объекты неизменяемыми. Побочные эффекты в цепочках вычислений ухудшают проверяемость; ясный цикл часто лучше сложной цепочки. Потоки Stream, параллелизм и конкурентные коллекции требуют отдельного изучения.

Минимальное реальное приложение

У законченного консольного приложения есть сценарии, формат входа, модель, команды, сохранение, обработка отказов и способ запуска. README должен объяснять действие пользователя, а не устройство каждой внутренней переменной. Данные для демонстрации должны быть безопасными и воспроизводимыми.

Упаковать выбранную точку входа можно в JAR с манифестом main-класса. JAR - контейнер классов и ресурсов, а не отдельная JVM. Для запуска нужен совместимый runtime. `--release 17` ограничивает синтаксис, доступные API и целевой байткод при компиляции современным JDK. Это сильнее, чем одно `-source 17`, но запуск тестов именно на Java 17 остаётся отдельной проверкой.

Для примера упражнение Ex001 после сборки можно упаковать так:

ТЕРМИНАЛ

```
jar --create --file build/demo.jar \
    --main-class ru.vantaeonx.javaguide.ch01.Ex001 \
    -C build/run/Ex001 .
java -jar build/demo.jar
```

Сначала собери выбранный файл через tools/run.sh. В JAR попадут его классы из отдельного каталога, включая вложенные. Для другого упражнения замени имя и пакет согласованно. Заготовка без твоего вывода и после упаковки останется пустой.

Maven в проекте задаёт структуру и release; shell-скрипты позволяют учиться без установки Maven. Позднее инструмент зависимостей понадобится для библиотек и модов. Не нужно внедрять веб-фреймворк до понимания границ модели и ввода-вывода.

Путь к Minecraft

Переносимые знания здесь - классы, интерфейсы, коллекции, события как вызовы, состояние и чтение документации API. Требования к Java у конкретной версии игры и загрузчика нужно проверять по их официальной документации перед созданием мода. Нельзя считать установленный JDK 25 автоматически подходящим для любой версии Minecraft.

Следующий учебный шаг - выбрать одну версию игры и один загрузчик, собрать минимальный пример по их документации и добавить одно наблюдаемое поведение. Не начинай с огромной системы или нескольких несовместимых версий. Курс даёт языковой фундамент; API игры, многопоточность и публикация мода требуют собственной практики.

Перед итоговым проектом объясни, где приложение хранит истину о состоянии, кто проверяет данные и кто сообщает об отказе. Если модель можно проверить без консоли и диска, ты уже построил полезную границу ответственности.

ГЛАВА 20 / EX172

Состояние как enum

Уровень 1 · Написать код · Ex172.java

Перед началом

Закрытый набор значений уменьшает пространство опечаток.

Твоя задача

Создай enum Status { NEW, ACTIVE, DONE }. Для каждого значения выведи русскую подпись через switch: Новая, В работе, Завершена. Не используй строку вместо enum внутри модели.

Проверь себя

- Три подписи в порядке объявления.
- Switch охватывает все значения; null не передаётся.

План / прогноз / причина ошибки

ГЛАВА 20 / EX173

Record сравнивает компоненты

Уровень 1 · Предсказать результат · Ex173.java

Перед началом

Record задаёт семантику значения для своих компонентов.

Твоя задача

Предскажи equals и == для двух независимо созданных записей.

JAVA

```
Position a = new Position(2, 5);
Position b = new Position(2, 5);
System.out.println(a.equals(b));
System.out.println(a == b);
```

JAVA

```
record Position(int x, int y) { }
```

Объявления уже есть в Java-файле. Реализуй TODO перед вызовом; для анализа исходные методы оставлены рабочими.

Проверь себя

- Прогноз до запуска.
- Объяснено различие равного значения и одного объекта.

План / прогноз / причина ошибки

ГЛАВА 20 / EX174

Захваченная переменная меняется

Уровень 1 · Найти ошибку · Ex174.java

Перед началом

Lambda требует final или effectively final локальную переменную.

Твоя задача

Найди причину ошибки и исправь так, чтобы offset задавался один раз и использовался для вывода чисел плюс 10.

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

JAVA

```
int offset = 10;
List<Integer> values = List.of(1, 2);
values.forEach(value -> System.out.println(value + offset));
offset = 20;
```

Проверь себя

- Для [1,2] вывод 11 и 12.
- Нет обходного изменяемого массива ради захвата.

План / прогноз / причина ошибки

ГЛАВА 20 / EX175

Record с проверкой

Уровень 2 · Написать код · Ex175.java

Перед началом

Краткая модель всё равно нуждается в предметном инварианте.

Твоя задача

Создай record Coordinate(int x,int y) с компактным конструктором: обе координаты -100..100. Метод long distanceSquared() возвращает x*x+y*y. Неверный аргумент -> IllegalArgumentException.

```java
int x = 3;
int y = 4;
```

Проверь себя

- distanceSquared() -> 25.

- 100,100 -> 20000; 101,0 отвергнут; equals одинаковых координат true.

План / прогноз / причина ошибки

ГЛАВА 20 / EX176

Поверхностная неизменяемость

Уровень 2 · Объяснить поведение · Ex176.java

Перед началом

Final-компонент record может ссылаться на изменяемую коллекцию.

Твоя задача

Объясни, почему record Log(List<String> lines) без защитного конструктора может менять наблюдаемое состояние. Предложи List.copyOf в компактном конструкторе и уточни его отношение к изменяемым элементам.

Проверь себя

- Различены структура списка и глубокая неизменяемость.
- Назван запрет null у List.copyOf; для String-элементов содержимое неизменяемо.

План / прогноз / причина ошибки

ГЛАВА 20 / EX177

Сравнение переполнилось

Уровень 2 · Исправить программу · Ex177.java

Перед началом

Вычитание двух int не является безопасным универсальным comparator.

Твоя задача

Исправь сортировку levels по возрастанию через Integer.compare или comparingInt. Проверь крайние значения.

```java
List<Integer> levels = new ArrayList<>(List.of(Integer.MAX_VALUE, 0, Integer.MIN_VALUE));
```

НАМЕРЕННАЯ ОШИБКА / ФРАГМЕНТ ДЛЯ АНАЛИЗА

В стартовом файле фрагмент закомментирован. Переноси его в рабочую область только по условию: методы и классы вне main, инструкции внутри.

```java
levels.sort((left, right) -> left - right);
```

Проверь себя

- Порядок MIN_VALUE,0,MAX_VALUE.
- Повторы сохраняются; comparator не вычитает операнды.

План / прогноз / причина ошибки

ГЛАВА 20 / EX178

Разрешённые переходы

Уровень 3 · Написать код · Ex178.java

Перед началом

Enum задаёт состояния, отдельный договор задаёт переходы.

Твоя задача

Определи в этом же файле enum Status { NEW, ACTIVE, DONE }. Создай Task с private Status и transitionTo(Status): NEW->ACTIVE, ACTIVE->DONE, повтор того же состояния не меняет ничего; остальные переходы IllegalStateException. Конструктор начинает с NEW.

```java
String label = "Проверить шлюз";
```

Проверь себя

- NEW->ACTIVE->DONE допустимо.
- NEW->DONE отвергнуто; DONE->NEW отвергнуто; состояние после отказа сохранено.

План / прогноз / причина ошибки

ГЛАВА 20 / EX179

Сортировка отчёта

Уровень 3 · Написать код · Ex179.java

Перед началом

Составной comparator выражает стабильное правило для равных ключей.

Твоя задача

Создай record Score(String name,int points). Отсортируй по points по убыванию, при равных - name по обычному String.compareTo по возрастанию. Имена не null. Выведи name:points.

```java
List<String> names = List.of("B", "A", "C");
List<Integer> points = List.of(8, 8, 12);
```

Проверь себя

- C:12, A:8, B:8.
- При равных name и points записи не теряются; исходные параллельные списки не меняются.

План / прогноз / причина ошибки

ГЛАВА 20 / EX180

Релиз одной команды

Уровень 3 · Написать код · Ex180.java

Перед началом

Запуск законченного приложения требует точки входа и ясного договора пользователя.

Твоя задача

В main реализуй команду sum: args[0] должен быть sum, остальные - целые int; вычисли long-сумму. Нет команды или ошибка числа -> USAGE. Упакуй только этот класс в исполняемый JAR и запиши команды сборки/запуска в комментарии. При sum без чисел сумма 0.

Проверь себя

- java -jar app.jar sum 4 -2 7 -> 9.
- sum -> 0; add 1 и sum x -> USAGE; перед USAGE не печатается частичная сумма.

План / прогноз / причина ошибки

ПОСЛЕ ГЛАВЫ 05 / МИНИ-ПРОЕКТ 01

Консольный пункт выдачи

Одна операция оформления отправления: ввод, арифметика, проверки и ветвления.

Договор приложения

Прочитай четыре строки: получатель (непустой, без краевых пробелов), масса в граммах (целое 1..30000), зона (local или remote), признак fragile (yes или no). На этом этапе числовая строка гарантированно преобразуется в int. Остальные нарушения дают INVALID, никакая цена тогда не печатается. Цена: local 300 центов, remote 700; дополнительно 100 за каждый начатый килограмм и 250 за fragile=yes. Выведи получателя, число килограммов и итоговые центы на трёх строках. Нулевой и отрицательный вес недопустимы. Повтор команд пока не нужен.

Исходные данные

JAVA

```
String sampleRecipient = "Лея";
int sampleGrams = 1201;
String sampleZone = "local";
String sampleFragile = "yes";
```

Стартовый файл: Project01.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 01: план и проверка

Этапы

1. Составь таблицу допустимых полей и цены каждой зоны.
2. Реализуй чтение по строкам без потери имени с пробелами.
3. До расчёта проверь все поля; раздели ошибку и корректный результат.
4. Добавь расчёт начатых килограммов и проверь точки 1000/1001.

Демонстрационный ввод

СТАНДАРТНЫЙ ВВОД

```
Лея
1201
local
yes
```

Критерии готовности

- Исходный пример -> Лея, 2, 750.
- Рин/1000/remote/no -> Рин,1,800; 1001 -> 2,900.
- Пустое имя, масса 0 или 30001, зона moon, fragile=maybe -> только INVALID.
- В README проекта опиши точный порядок строк и единицу цены.

Следующее расширение

- После главы 14 добавь обработку нечисловой массы и повтор ввода.
- После главы 9 выдели расчёт цены в чистый метод.

Твой следующий проверяемый шаг

ПОСЛЕ ГЛАВЫ 09 / МИНИ-ПРОЕКТ 02

Панель телеметрии

Цикл команд, массив измерений, методы и числовая статистика.

Договор приложения

Храни до 20 целых измерений -1000..1000 в массиве и отдельном size. Читай команды построчно: add N, stats, clear, exit. В add число корректно записано, но может быть вне диапазона. При успехе add -> ADDED, при заполнении -> FULL, неверном диапазоне -> INVALID. Stats на пустом наборе -> EMPTY, иначе одна строка COUNT:n MIN:a MAX:b AVG:v (с 2 знаками и Locale.ROOT). Clear -> CLEARED. Неизвестная команда -> UNKNOWN. Exit -> BYE; EOF завершает без BYE. Выдели вычисления в методы; среднее считай только по первым size ячейкам. Коллекции не используй.

Исходные данные

JAVA

```
int capacity = 20;
int[] readings = new int[capacity];
int size = 0;
```

Стартовый файл: Project02.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 02: план и проверка

Этапы

1. Сначала реализуй add и отдельный счётчик size.
2. Напиши методы min, max, sum для занятой части массива.
3. Добавь цикл команд и полный договор пустоты.
4. Прогони сценарии заполнения, очистки и повторного добавления.

Демонстрационный ввод

СТАНДАРТНЫЙ ВВОД

```
add 10
add -2
stats
clear
stats
exit
```

Критерии готовности

- Пример -> ADDED,ADDED,COUNT:2 MIN:-2 MAX:10 AVG:4.00,CLEARED,EMPTY,BYE.
- После 20 успешных add 21-я команда даёт FULL без изменения данных.
- Неиспользованные нулевые ячейки не влияют на статистику.
- После clear новый add начинает заполнение с позиции 0.

Следующее расширение

- После главы 14 обрабатывай испорченные команды без падения.
- После главы 15 замени хранение на List, сохранив договор.

Твой следующий проверяемый шаг

ПОСЛЕ ГЛАВЫ 11 / МИНИ-ПРОЕКТ 03

Бронирование рабочего пространства

Модель объектов, конструкторы, private и операции с инвариантами.

Договор приложения

Создай Room с неизменяемым id, capacity>0 и private reserved от 0 до capacity. Методы reserve(int), cancel(int), available(), id(). Аргументы операций неотрицательны; нехватка мест или отмена больше reserved -> IllegalArgumentException без изменения. BookingDesk хранит защитную копию массива Room, ищет по id и выполняет операции; id комнат уникальны и ненулевые, null-комнаты запрещены. Для неизвестного id возвращай false без изменения, для найденного корректная операция возвращает true. Комнаты изменяемы; явно документируй, что копия массива не клонирует Room. Main демонстрирует сценарий на фиксированных данных, консольный парсер не требуется. Методы стола: boolean reserve(String id,int n) и boolean cancel(String id,int n).

Исходные данные

```java
String[] ids = {"A", "B"};
int[] capacities = {4, 2};
```

Стартовый файл: Project03.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 03: план и проверка

Этапы

1. Опиши инвариант Room и проверь конструктор.
2. Реализуй reserve/cancel: сначала проверки, затем изменение.
3. Добавь BookingDesk с поиском и проверкой уникальности id.
4. Сделай проверочный main с успешными операциями и ожидаемыми отказами.

Критерии готовности

- Для A: reserve(3) -> свободно 1; cancel(1) -> свободно 2.
- Попытка reserve(3) после этого отвергнута, свободно всё ещё 2.
- Операции A не меняют B; неизвестный X -> false.
- Изменение исходного массива после создания не заменяет комнаты стола; null и повтор id в конструкторе запрещены.

Следующее расширение

- После главы 13 введи интерфейс источника комнат.
- После главы 18 сохраняй снимок бронирований в отдельный файл.

Твой следующий проверяемый шаг

ПОСЛЕ ГЛАВЫ 16 / МИНИ-ПРОЕКТ 04

Каталог мастерской

Объектная модель, Map/Set, уникальные ключи и история операций.

Договор приложения

Создай Item с final id,name и private nonnegative int stock. Id/name непустые после trim, id не меняется. Catalog хранит LinkedHashMap<String,Item>. Register запрещает повтор id, increase(id,n) и take(id,n) требуют n>=0, take не превышает stock; неизвестный id -> IllegalArgumentException. Увеличение, выходящее за int, запрещено до изменения. History - List<String> только успешных операций с точным форматом REGISTER:id, INCREASE:id:n, TAKE:id:n. List выдаётся неизменяемой копией. LowStock(limit) возвращает id с stock<=limit в порядке регистрации, limit>=0. Main работает с фиксированными операциями и проверками, без чтения файла.

Исходные данные

```java
String[] ids = {"iron", "wood"};
String[] names = {"Железо", "Доски"};
int[] stocks = {5, 2};
```

Стартовый файл: Project04.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 04: план и проверка

Этапы

1. Разработай Item с контролем остатка и переполнения.
2. Добавь Catalog и уникальность идентификатора.
3. Записывай историю только после успешной операции.
4. Проверь lowStock и невозможность изменить выданный список истории.

Критерии готовности

- После REGISTER iron=5, wood=2 и TAKE iron:3 lowStock(2) -> [iron,wood].
- TAKE wood:3 отвергнут: wood=2, история не выросла.
- Повтор REGISTER iron отвергнут, исходная запись не заменена.
- INCREASE на Integer.MAX_VALUE при ненулевом остатке отвергнут без переполнения.

Следующее расширение

- После главы 17 выдели обобщённое Repository<K,V> без raw types.
- Добавь отдельный адаптер команд после завершения модели.

Твой следующий проверяемый шаг

ПОСЛЕ ГЛАВЫ 18 / МИНИ-ПРОЕКТ 05

Файловый журнал расходов

Текстовый формат, разбор, ошибки и проверяемый финансовый расчёт в центах.

Договор приложения

Работай только в build/sandbox/project05. Создай входной UTF-8 файл expenses.txt из initialLines для демонстрации. Формат category;cents: category непустая, без краевых пробелов и без ;, cents целое long 0..1000000000. Ровно два поля. Прочитай все строки, собирай Map<String,Long> сумм в порядке первого появления; неверные строки пропускай с ERROR:номер:причина, пустая строка ошибочна. Номер с 1. Если суммарное сложение переполняется, откажись от отчёта явно, не допускай обёрнутого результата. Сохрани report.txt: по строке category;sum, затем TOTAL;sum. TOTAL запрещена как категория. IOException не маскируй ошибкой формата. Раздели parse, aggregate и file adapter.

Исходные данные

JAVA

```
List<String> initialLines = List.of("food;125", "travel;300", "food;75", "bad", "food;-1");
```

Стартовый файл: Project05.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 05: план и проверка

Этапы

1. Запиши точный формат и выбери тип результата разбора записи.
2. Проверь парсер в памяти до файловых операций.
3. Реализуй агрегирование допустимых строк и отдельный список ошибок.
4. Добавь запись отчёта и пользовательское сообщение об IOException.

Критерии готовности

- Пример: ошибки строк 4,5; report food;200, travel;300, TOTAL;500.
- Пустой файл -> только TOTAL;0; файл с одними ошибками также TOTAL;0 и полный отчёт об ошибках.
- Русская category сохраняется; отсутствие входного файла при режиме чтения даёт файловую ошибку.
- Повторный отчёт заменяет свой report.txt, вход не повреждается; TOTAL;5 отвергнута.

Следующее расширение

- Реализуй безопасную замену отчёта через временный файл и явную политику atomic move.
- Добавь режим запуска read без пересоздания демонстрационного входа.

Твой следующий проверяемый шаг

ПОСЛЕ ГЛАВЫ 20 / МИНИ-ПРОЕКТ 06

Симулятор исследовательского рюкзака

Итоговый проект: модель предметов, команды, интерфейсы, коллекции, generics и сохранение.

Договор приложения

Создай enum Kind { TOOL, MATERIAL, FOOD } и immutable record ItemType(String id,Kind kind,int unitWeight), где id соответствует [a-z][a-z0-9_]{0,15}, unitWeight>0. Backpack имеет maxWeight>0 и LinkedHashMap<ItemType,Integer> количества, неотрицательные int; общий вес long не превышает maxWeight. Add/take проверяют n>0, остаток и переполнение до изменения; нулевой остаток удаляет запись. Repository<T> задаёт save(T) и load(), файловая реализация хранит снимок. Команды: add id n, take id n, list, save, load, exit. Успех изменения -> OK; отказ -> ERROR:причина; list печатает id:count в порядке появления и WEIGHT:число. Save/load в UTF-8 только build/sandbox/project06/backpack.txt: id;count, id берётся из фиксированного каталога. Load сначала проверяет весь снимок, затем заменяет рюкзак целиком; при любой ошибке старое состояние остаётся. Повтор id в снимке запрещён. Формат команды неизвестен -> UNKNOWN, EOF завершает. Exit -> BYE. Конкретный Minecraft API не подключай. В снимке разрешены только положительные count. Методы файлового Repository могут объявлять IOException; граница команды обрабатывает её как ERROR:IO без изменения модели. Во время load отрицательные и нулевые count отвергаются.

Исходные данные

JAVA

```
String[] ids = {"pickaxe", "stone", "apple"};
String[] kinds = {"TOOL", "MATERIAL", "FOOD"};
int[] weights = {5, 2, 1};
long maxWeight = 15;
```

Стартовый файл: Project06.java. Сначала реализуй модель на этих данных. Этапы, проверки и расширения - на следующей странице.

Проект 06: план и проверка

Этапы

1. Сначала реализуй чистую модель и проверки веса/остатка.
2. Добавь интерфейс Repository<T> и MemoryRepository для тестов.
3. Добавь парсер команд и ReportOutput; проверь сценарий без диска.
4. Реализуй файловое хранилище и атомарную по смыслу загрузку через отдельный проверенный снимок.
5. Напиши README приложения и упакуй выбранный main в JAR.

Демонстрационный ввод

СТАНДАРТНЫЙ ВВОД

```
add stone 4
add apple 3
list
save
take stone 2
load
list
exit
```

Критерии готовности

- После add stone 4, add apple 3: stone:4, apple:3, WEIGHT:11.
- Добавление pickaxe 1 отвергнуто, вес остаётся 11; take неизвестного id отвергнут.
- После save, take stone 2, load восстановлены stone:4, apple:3 и вес 11.
- Повреждённый файл, неизвестный id, отрицательное количество, повтор id или превышение веса не меняют текущее состояние.
- Тесты модели запускаются без терминала и файла; Repository<T> не raw type; два рюкзака независимы.

Следующее расширение

- Добавь события изменения через интерфейс, чтобы внешний адаптер мог реагировать.
- Для будущего мода отдельно проверь JDK и API выбранного загрузчика; симулятор сам по себе модом не является.

Твой следующий проверяемый шаг

Памятки после первого прохода

Выбор структуры

Нужно	Подход
Фиксированное число однотипных элементов	Массив
Последовательность с добавлением и повторами	ArrayList через List
Уникальность без порядка	HashSet
Уникальность с порядком первого появления	LinkedHashSet
Поиск значения по ключу	Map
Отчёт по ключам в порядке регистрации	LinkedHashMap
Сортированный порядок ключей	TreeMap с осмысленным сравнением
Закрытый набор состояний	enum
Неизменяемое по компонентам значение	record с проверкой и защитой ссылочных компонентов
Изменяемый объект с правилами	Класс с private-полями и операциями

Прежде чем запускать

Проверь типы промежуточных выражений, границы индексов, причины остановки цикла, допустимость null и пустоты. Если операция меняет состояние, проверь отказ до записи. Если работаешь с файлом, уточни рабочую папку, UTF-8, режим перезаписи и закрытие ресурса.

Разбор проблемы

1. Зафиксируй точный вход, ожидаемое и полученное поведение.
2. Найди первый момент расхождения, а не последнюю видимую ошибку.
3. Уменьши вход. Нарисуй ссылки или таблицу итераций.
4. Проверь одну гипотезу одним изменением.
5. После исправления оставь проверку прежнего дефекта.

Минимальная матрица проверок

Вид операции	Проверки
Диапазон	Ниже, ровно нижняя, внутри, ровно верхняя, выше
Последовательность	Пустая, один элемент, несколько, повторы
Максимум	Все отрицательные, один, пустота по договору
Строка	Пустая, пробельная, разделитель отсутствует, пустое поле
Объект	Корректное создание, отказ создания, успех изменения, отказ без изменения
Мар	Новый ключ, повторный ключ, отсутствующий, равный новый ключ
Файл	Отсутствует, пустой, русские данные, повреждённая строка, повторный запуск
Сохранение	Успех, ошибка подготовки, ошибка замены, старые данные после отказа

Отсылка к следующим темам

После этого курса отдельно изучай Maven/Gradle и зависимости, JUnit, логирование, Git, сетевые протоколы и базы данных. Многопоточность требует отдельной модели памяти и синхронизации. Для Minecraft сначала выбери версию игры и загрузчик, проверь их официальные требования к JDK и создай минимальное наблюдаемое поведение.

Словарь

Термин	Смысл
JDK	Комплект разработки: компилятор, инструменты и среда выполнения
JVM	Виртуальная машина, загружающая и выполняющая байткод
Байткод	Представление скомпилированного кода в class-файлах
Пакет	Пространство имён классов, связанное с организацией исходников
Classpath	Корни поиска классов при компиляции или запуске
Тип	Договор допустимых значений и операций
Примитив	Непосредственное значение одного из восьми примитивных типов
Ссылка	Значение для обращения к объекту, либо null
Объект	Экземпляр класса или массив с собственной идентичностью
Поле	Часть состояния объекта или класса
Метод	Именованная операция класса с параметрами и договором результата
Параметр	Локальная переменная вызова в объявлении метода
Аргумент	Значение, переданное при вызове
Конструктор	Инициализация создаваемого экземпляра
Инвариант	Условие, сохраняемое после законных операций
Инкапсуляция	Контроль доступа и изменений через ясный договор
Наследование	Отношение типов с подстановкой подкласса вместо базового
Полиморфизм	Общий вызов выбирает поведение конкретного объекта
Перегрузка	Одно имя и разные списки параметров, выбор при компиляции

Термин	Смысл
Переопределение	Реализация метода экземпляра в подтипе
Интерфейс	Договор операций, который реализует класс
Композиция	Объект содержит и использует другие объекты
Исключение	Передача управления при невозможности обычного результата
Checked	Исключение, требующее обработки или объявления по правилам компилятора
Коллекция	Структура для работы с группой элементов
List	Последовательность с индексами и разрешёнными дубликатами
Set	Набор уникальных по договору равенства элементов
Map	Связи уникальных ключей со значениями
Generic	Объявление с параметрами типов, связывающими входы и результаты
Wildcard	Неизвестный аргумент типа, возможно с границей extends/super
Boxing	Преобразование примитива в объектную обёртку
Неизменяемость	Отсутствие изменения наблюдаемого состояния после создания
Защитная копия	Копирование для сохранения контроля над внутренним состоянием
UTF-8	Кодировка текста в байты, используемая для файлов курса
UTF-16	Модель кодовых единиц char в строках Java
Регрессия	Возвращение ранее исправленной ошибки
Контракт	Входы, результат, побочные эффекты и отказы операции

Таблица прогресса

Запиши даты решения и повторения. На повторении сначала объясни модель без просмотра своего кода. Если не получилось, отметь причину и назначь новый срок.

Глава 01

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex001	1				
Ex002	1				
Ex003	1				
Ex004	2				
Ex005	2				
Ex006	2				
Ex007	3				
Ex008	3				
Ex009	3				

Глава 02

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex010	1				
Ex011	1				
Ex012	1				
Ex013	2				
Ex014	2				
Ex015	2				
Ex016	3				
Ex017	3				
Ex018	3				

Глава 03

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex019	1				
Ex020	1				
Ex021	1				
Ex022	2				
Ex023	2				
Ex024	2				
Ex025	3				
Ex026	3				
Ex027	3				

Глава 04

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex028	1				
Ex029	1				
Ex030	1				
Ex031	2				
Ex032	2				
Ex033	2				
Ex034	3				
Ex035	3				
Ex036	3				

Глава 05

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex037	1				
Ex038	1				
Ex039	1				
Ex040	2				
Ex041	2				
Ex042	2				
Ex043	3				
Ex044	3				
Ex045	3				

Глава 06

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex046	1				
Ex047	1				
Ex048	1				
Ex049	2				
Ex050	2				
Ex051	2				
Ex052	3				
Ex053	3				
Ex054	3				

Глава 07

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex055	1				
Ex056	1				
Ex057	1				
Ex058	2				
Ex059	2				
Ex060	2				
Ex061	3				
Ex062	3				
Ex063	3				

Глава 08

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex064	1				
Ex065	1				
Ex066	1				
Ex067	2				
Ex068	2				
Ex069	2				
Ex070	3				
Ex071	3				
Ex072	3				

Глава 09

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex073	1				
Ex074	1				
Ex075	1				
Ex076	2				
Ex077	2				
Ex078	2				
Ex079	3				
Ex080	3				
Ex081	3				

Глава 10

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex082	1				
Ex083	1				
Ex084	1				
Ex085	2				
Ex086	2				
Ex087	2				
Ex088	3				
Ex089	3				
Ex090	3				

Глава 11

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex091	1				
Ex092	1				
Ex093	1				
Ex094	2				
Ex095	2				
Ex096	2				
Ex097	3				
Ex098	3				
Ex099	3				

Глава 12

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex100	1				
Ex101	1				
Ex102	1				
Ex103	2				
Ex104	2				
Ex105	2				
Ex106	3				
Ex107	3				
Ex108	3				

Глава 13

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex109	1				
Ex110	1				
Ex111	1				
Ex112	2				
Ex113	2				
Ex114	2				
Ex115	3				
Ex116	3				
Ex117	3				

Глава 14

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex118	1				
Ex119	1				
Ex120	1				
Ex121	2				
Ex122	2				
Ex123	2				
Ex124	3				
Ex125	3				
Ex126	3				

Глава 15

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex127	1				
Ex128	1				
Ex129	1				
Ex130	2				
Ex131	2				
Ex132	2				
Ex133	3				
Ex134	3				
Ex135	3				

Глава 16

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex136	1				
Ex137	1				
Ex138	1				
Ex139	2				
Ex140	2				
Ex141	2				
Ex142	3				
Ex143	3				
Ex144	3				

Глава 17

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex145	1				
Ex146	1				
Ex147	1				
Ex148	2				
Ex149	2				
Ex150	2				
Ex151	3				
Ex152	3				
Ex153	3				

Глава 18

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex154	1				
Ex155	1				
Ex156	1				
Ex157	2				
Ex158	2				
Ex159	2				
Ex160	3				
Ex161	3				
Ex162	3				

Глава 19

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex163	1				
Ex164	1				
Ex165	1				
Ex166	2				
Ex167	2				
Ex168	2				
Ex169	3				
Ex170	3				
Ex171	3				

Глава 20

Файл	Ур.	Решено	+1 день	+7 дней	+21 день
Ex172	1				
Ex173	1				
Ex174	1				
Ex175	2				
Ex176	2				
Ex177	2				
Ex178	3				
Ex179	3				
Ex180	3				

Прогресс мини-проектов

Отмечай готовность после проверки всех критериев. Запиши один дефект, который обнаружил сам, и объясни, какая проверка его поймала. Готовность модели, консольного интерфейса и файла проверяется отдельно.

Проект	Готово	Что улучшить
Project01		
Project02		
Project03		
Project04		
Project05		
Project06		

Что уже можешь объяснить

- Как модель сохраняет инварианты после успеха и отказа.
- Почему выбран именно этот контейнер и договор равенства.
- Как вычисление проверить без реальной консоли и диска.
- Что произойдёт при пустом или повреждённом файле.

План следующего этапа

Источники и как ими пользоваться

Материалы написаны на русском для самостоятельного обучения. Определения и особенности языка сверяются с Java 17, а инструменты - с актуальной официальной документацией. Дата редакторской сверки: 16 октября 2026 года. Версии интерфейса IDEA могут отличаться.

- Java Language Specification 17: формальный договор языка. Главы 4-5 - типы и преобразования, 6-8 - имена и классы, 9 - интерфейсы, 10 - массивы, 11 - исключения, 14-15 - инструкции и выражения.
- Java 17 API: контракт стандартных классов, а не только примеры их использования.
- String: неизменяемость, UTF-16, индексы и split.
- Scanner: токены, строки и локаль.
- List: дубликаты, порядок и ограничения неизменяемых фабрик.
- Map: ключи, операции и контракт равенства.
- Object: договор equals и hashCode.
- Files: файловые операции, кодировки, режимы и исключения.
- javac JDK 25: --release, -d, classpath и целевая версия.
- Первое Java-приложение в IntelliJ IDEA: настройка SDK, проект и запуск main.
- Maven Compiler Plugin: конфигурация release и сборка Maven.
- Python-гайд пользователя: педагогическая структура, а не источник Java-кода и готовых ответов.

Если краткая формулировка книги и точный контракт API кажутся несовместимыми, проверь версию Java, контекст и полный договор метода. Сообщай о дефекте с номером главы/упражнения и минимальным воспроизводимым примером.

Педагогическая отправная точка

В python-guide-ru сохранён удачный принцип: уровни, независимые файлы, чтение и исправление кода. В оригинале указана благодарность Nicolas Hinckxt за структуру PHP-сборника. Java-тексты, примеры и задания написаны отдельно; механического переноса Python-задач нет. Анализ и проектирование находятся в docs/ORIGINAL_REVIEW.md. Лицензия нового пособия и кода - MIT.

Чего книга не обещает

Это фундамент языка и консольной разработки. Конкретный Minecraft API, многопоточность, сеть, базы данных, GUI и фреймворки требуют следующего этапа. Проверяй требования выбранной платформы по её официальным источникам.